PROJECT 8 | SNOWFLAKE CAPSTONE | INSURANCE, CLAIMS & FRAUD

ClaimGuard 360

Step-by-Step Implementation Guide

AWS S3 → IDMC → Bronze → Silver → Gold → Snowpark → Streamlit

What this guide gives you

Every click in AWS and IDMC, written as numbered steps

Every SQL, Python and Streamlit script, ready to copy and run

The five stand-out features fully built: File Listener, Control File Check, Reject Reprocessing, Threshold Alerts, Region-Based Access

A “Done when” checklist after every phase, so you always know you are on track

Test cases, demo script, runbook and recovery steps

Companion to *ClaimGuard360_Pipeline_Flow.pdf* (concept) and *PROJECT_8_UPDATED_SNOWFLAKE_CAPSTONE_DOCUMENT.docx* (requirements)

Contents

Phase	What you build	Day
0	How to use this guide, prerequisites, naming standards	1
1	AWS S3 bucket, folders, IAM access for IDMC and Snowflake	1
2	Snowflake foundation: role, warehouse, database, schemas, integration, stages	1
3	CONTROL schema: audit, control-file, reject, alert and region tables	1–2
4	★ Control File Check (stored procedure)	2
5	IDMC connections, cleansing mappings and routing (all 7 sources)	2
6	★ IDMC File Listener and Taskflow (automatic start)	2
7	Bronze layer: tables, COPY, audit	2
8	Silver layer: typing, dedup, relationship checks	3
9	Gold layer: dimensions, facts, marts, 10 KPIs	3
10	Incremental claims: Stream, MERGE, Task graph, replay test	4
11	★ Reject Reprocessing loop	4
12	Snowpark fraud-risk score	4
13	Security: roles, masking, secure views, ★ Region-Based Access	4
14	★ Threshold Alerts (email)	4
15	Streamlit in Snowflake dashboard	5
16	Deployment, Git, runbook, recovery	5
17	Testing, reconciliation and demo script	5

Phase 0 – Before You Start

0.1 How to read this guide

- Work **phase by phase, in order**. Each phase depends on the previous one.
- Grey boxes are **code**. Copy them into a Snowsight worksheet (or SnowSQL) and run them top to bottom.
- Green boxes are **tips**, red boxes are **common mistakes**, amber boxes are **decisions you must confirm** with your own data (for example the exact date formats in your files).
- Every phase ends with a **"Done when"** checklist. Take a screenshot of each item – that becomes your evidence.
- Words in <angle brackets> such as <aws_account_id> must be replaced with your own values.

0.2 What you need

Item	Why	Who usually has it
AWS account with S3 + IAM access	Bucket, folders, roles	Trainer / cloud admin
Snowflake account (Enterprise edition or trial)	Masking and row access policies need Enterprise	Trainer
ACCOUNTADMIN for 30 minutes	Storage integration, email integration, EXECUTE TASK / ALERT grants	Trainer
IDMC org with Data Integration + Secure Agent	Mappings, taskflows, file listener	Trainer
Amazon S3 V2 and Snowflake Data Cloud connectors enabled in IDMC	Read/write S3 and Snowflake	IDMC admin
The 7 CSV files + a second claims file	Initial load and incremental test	Project kit
Git repository (GitHub / Azure Repos)	Version control of all scripts	Team

0.3 Team split (5 trainees)

Member	Owns phases	Hand-off to
T1 – Cloud & Platform	1, 2, 3, 16	Everyone (credentials, stages)
T2 – IDMC Engineer	4 (with T1), 5, 6	T3 (clean files in S3)
T3 – Snowflake Engineer	7, 8, 9, 10	T4, T5
T4 – Reprocessing, Snowpark & Alerts	11, 12, 14	T5
T5 – Security & Streamlit	13, 15, 17	Demo

0.4 Naming standards (use exactly these)

Object	Name
S3 bucket	claimguard360-<yourname> (bucket names must be globally unique, lower case)
Snowflake database / warehouse	INSURANCE_DB / CG_WH
Schemas	BRONZE, SILVER, GOLD, CONSUMPTION, CONTROL
Engineering role	CG_ENGINEER_ROLE (owns all objects)
Business roles	INSURANCE_ANALYST_ROLE, CLAIMS_ANALYST_ROLE, FRAUD_MANAGER_ROLE
Regional demo roles	INSURANCE_ANALYST_NORTH_ROLE, INSURANCE_ANALYST_SOUTH_ROLE

Object	Name
IDMC connections	CONN_S3_CG, CONN_SF_CG
IDMC mappings	m_clean_<source>, m_clean_<source>_resubmit, m_ctl_check
IDMC taskflows / listeners	tf_cg_ingest, tf_cg_resubmit / fl_cg_control, fl_cg_resubmit
Control file	<source>_<yyyymmdd>.ctl e.g. claims_20261103.ctl
Data file	<source>_<yyyymmdd>.csv e.g. claims_20261103.csv

✔ **Done when:**

- ■ Everyone has logins to AWS, Snowflake and IDMC.
- ■ Git repository created with folders: sql/, idmc/, snowpark/, streamlit/, tests/, docs/.
- ■ Team split agreed and written in the README.

Phase 1 – AWS S3 Setup

Goal: one bucket where every file has a clear place for every stage of its life, plus two sets of credentials – one for IDMC (read/write) and one for Snowflake (read only, through a role).

1.1 Create the bucket

1. Sign in to the AWS console → search **S3** → **Create bucket**.
2. Bucket name: **claimguard360-<yourname>**. Region: **ap-south-1 (Mumbai)** – keep the same region as your Snowflake account if possible (lower latency, no transfer cost).
3. Keep **Block all public access** ticked. Leave versioning **Enabled** (gives you an extra safety net).
4. Click **Create bucket**.

1.2 Create the folder structure

Open the bucket → **Create folder** for each path below. (S3 folders are just name prefixes – the trailing / matters.)

S3 layout

```
claimguard360-<yourname>/
├── landing/      customers/  insurance_products/  policies/  premium_payments/
│                 claims/  claim_settlements/  fraud_cases/
├── control/                      <- .ctl files land here (File Listener watches this)
├── clean/        <same 7 sub-folders>   <- IDMC writes clean + warning rows (loaded to Bronze)
├── warning/      <same 7 sub-folders>   <- copy of warning rows only (evidence)
├── duplicate/    <same 7 sub-folders>
├── reject/       <same 7 sub-folders>   <- rejected rows with reject_id + reason
├── resubmit/     <same 7 sub-folders>   <- corrected reject files are dropped here
├── quarantine/                   <- files that failed the control check
└── archive/                      <- processed source files (yyyy/mm/dd)
```

Shortcut

With the AWS CLI you can create all folders in one go: `for f in landing clean warning duplicate reject resubmit; do for s in customers insurance_products policies premium_payments claims claim_settlements fraud_cases; do aws s3api put-object --bucket claimguard360-<yourname> --key $f/$s/; done; done`

1.3 IAM user for IDMC (access key)

1. IAM → **Policies** → **Create policy** → JSON tab → paste the policy below → name it **CG_IDMC_S3_POLICY**.
2. IAM → **Users** → **Create user cg-idmc-user** → attach CG_IDMC_S3_POLICY.
3. Open the user → **Security credentials** → **Create access key** → choose “Application running outside AWS” → copy the **Access key** and **Secret key** (shown only once). Store them in your team password vault, never in Git.

CG_IDMC_S3_POLICY

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetBucketLocation"],
      "Resource": "arn:aws:s3:::claimguard360-<yourname>" },
    { "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::claimguard360-<yourname>/*" }
  ]
}
```

1.4 IAM role for Snowflake (no keys – trusted role)

1. IAM → Policies → Create policy → JSON below → name **CG_SNOWFLAKE_S3_POLICY** (read-only).
2. IAM → **Roles** → **Create role** → Trusted entity: **AWS account** → **This account** → tick **Require external ID** and type a temporary value **0000** → attach CG_SNOWFLAKE_S3_POLICY → name **cg-snowflake-role**.
3. Copy the **Role ARN** (arn:aws:iam::<aws_account_id>:role/cg-snowflake-role). You need it in Phase 2.
4. You will come back to this role in step 2.4 to replace the trust policy with Snowflake's real values.

CG_SNOWFLAKE_S3_POLICY

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:GetObjectVersion"],
      "Resource": "arn:aws:s3:::claimguard360-<yourname>/*" },
    { "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetBucketLocation"],
      "Resource": "arn:aws:s3:::claimguard360-<yourname>" }
  ]
}
```

1.5 Prepare the control files

Every data file gets a one-line partner file that states what the data file should contain. Create them in Excel/Notepad and save as **.ctl** (it is just CSV text).

claims_20261103.ctl (header + one line)

```
source_name,data_file_name,expected_row_count,amount_col_position,expected_amount_total,business_date
claims,claims_20261103.csv,1200,7,<sum of claim_amount>,2026-11-03
```

Source	amount_col_position (column number of the amount to total)	Column
customers	0 (no amount – only the row count is checked)	–
insurance_products	5	minimum_sum_insured
policies	9	premium_amount
premium_payments	5	payment_amount
claims	7	claim_amount
claim_settlements	8	net_settlement_amount
fraud_cases	0	–

How to get expected values for the kit files

Open the CSV in Excel: expected_row_count = number of data rows (exclude the header). expected_amount_total = SUM of the amount column. For your demo, the source system would normally produce these.

1.6 Upload order rule

Always upload the **data file first** and the **.ctl file last**. The File Listener watches only the control/ folder, so the pipeline starts only once the data file is fully in place.

✔ **Done when:**

- ■ Bucket and all folders visible in the S3 console.
- ■ IDMC access key stored safely; Snowflake role ARN copied.

- Seven .ctl files prepared (not uploaded yet).

Phase 2 – Snowflake Foundation

Goal: one engineering role that owns everything, a small warehouse, the database with five schemas, a secure link to S3, and stages that point at the S3 folders. Run each block in a Snowsight worksheet.

2.1 Role, warehouse, database and schemas

sql/01_foundation.sql

```
-- Run as ACCOUNTADMIN once (trainer) ------------------------------------------
USE ROLE ACCOUNTADMIN;
ALTER ACCOUNT SET TIMEZONE = 'Asia/Kolkata';      -- all timestamps, tasks and alerts in IST

USE ROLE SECURITYADMIN;
CREATE ROLE IF NOT EXISTS CG_ENGINEER_ROLE COMMENT = 'ClaimGuard360 engineering / owner role';
GRANT ROLE CG_ENGINEER_ROLE TO ROLE SYSADMIN;     -- keep the role hierarchy clean
GRANT ROLE CG_ENGINEER_ROLE TO USER <your_snowflake_user>;

USE ROLE SYSADMIN;
CREATE WAREHOUSE IF NOT EXISTS CG_WH
  WAREHOUSE_SIZE = 'XSMALL' AUTO_SUSPEND = 60 AUTO_RESUME = TRUE INITIALLY_SUSPENDED = TRUE;
CREATE DATABASE IF NOT EXISTS INSURANCE_DB;
GRANT OWNERSHIP ON DATABASE INSURANCE_DB TO ROLE CG_ENGINEER_ROLE COPY CURRENT GRANTS;
GRANT USAGE, OPERATE ON WAREHOUSE CG_WH TO ROLE CG_ENGINEER_ROLE;

USE ROLE ACCOUNTADMIN;
GRANT EXECUTE TASK  ON ACCOUNT TO ROLE CG_ENGINEER_ROLE;   -- needed to resume tasks
GRANT EXECUTE ALERT ON ACCOUNT TO ROLE CG_ENGINEER_ROLE;   -- needed for alerts (Phase 14)

-- From here on, always work as the engineering role ---------------------------
USE ROLE CG_ENGINEER_ROLE;
USE WAREHOUSE CG_WH;
USE DATABASE INSURANCE_DB;
CREATE SCHEMA IF NOT EXISTS BRONZE      COMMENT = 'Raw, source-aligned, all STRING + metadata';
CREATE SCHEMA IF NOT EXISTS SILVER      COMMENT = 'Typed, deduplicated, relationship-validated';
CREATE SCHEMA IF NOT EXISTS GOLD        COMMENT = 'Dimensions, facts, marts, fraud score';
CREATE SCHEMA IF NOT EXISTS CONSUMPTION COMMENT = 'Secure views + Streamlit app';
CREATE SCHEMA IF NOT EXISTS CONTROL     COMMENT = 'Stages, audit, DQ, rejects, alerts, policies';
```

> **Why one owner role?**
>
> If objects are created by different roles, grants become messy and tasks fail with “insufficient privileges”. Create **everything** as CG_ENGINEER_ROLE; only role creation and account-level grants use SECURITYADMIN / ACCOUNTADMIN, exactly as the project document requires.

2.2 Storage integration (secure link to S3)

sql/02_integration.sql

```
USE ROLE ACCOUNTADMIN;
CREATE OR REPLACE STORAGE INTEGRATION CG_S3_INT
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::<aws_account_id>:role/cg-snowflake-role'
  STORAGE_ALLOWED_LOCATIONS = ('s3://claimguard360-<yourname>/');
GRANT USAGE ON INTEGRATION CG_S3_INT TO ROLE CG_ENGINEER_ROLE;

DESC INTEGRATION CG_S3_INT;
-- Copy two values from the output:
--   STORAGE_AWS_IAM_USER_ARN   (e.g. arn:aws:iam::123456789012:user/abc1-s-xyz)
--   STORAGE_AWS_EXTERNAL_ID    (e.g. AB12345_SFCRole=2_abcdefg=)
```

2.3 Finish the AWS trust (back in AWS)

1. IAM → Roles → **cg-snowflake-role** → **Trust relationships** → **Edit trust policy**.
2. Replace the whole JSON with the one below, pasting the two values from DESC INTEGRATION.
3. Click **Update policy**. Wait one minute for AWS to apply it.

cg-snowflake-role trust policy

```
{
```

```
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Principal": { "AWS": "<STORAGE_AWS_IAM_USER_ARN>" },
      "Action": "sts:AssumeRole",
      "Condition": { "StringEquals": { "sts:ExternalId": "<STORAGE_AWS_EXTERNAL_ID>" } } }
  ]
}
```

Common mistake

Running **CREATE OR REPLACE STORAGE INTEGRATION** again generates a **new external ID** and breaks the trust. If you must recreate it, repeat step 2.3 with the new values.

2.4 File formats and stages

sql/03_stages.sql

```
USE ROLE CG_ENGINEER_ROLE;
USE SCHEMA INSURANCE_DB.CONTROL;

CREATE OR REPLACE FILE FORMAT CONTROL.FF_CSV
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  TRIM_SPACE = TRUE
  NULL_IF = ('', 'NULL', 'null', 'NA', 'N/A')
  EMPTY_FIELD_AS_NULL = TRUE
  ERROR_ON_COLUMN_COUNT_MISMATCH = TRUE     -- wrong column count = error row (header check)
  ENCODING = 'UTF8';

CREATE OR REPLACE STAGE CONTROL.STG_LANDING
  URL = 's3://claimguard360-<yourname>/landing/'
  STORAGE_INTEGRATION = CG_S3_INT  FILE_FORMAT = CONTROL.FF_CSV;

CREATE OR REPLACE STAGE CONTROL.STG_CONTROL
  URL = 's3://claimguard360-<yourname>/control/'
  STORAGE_INTEGRATION = CG_S3_INT  FILE_FORMAT = CONTROL.FF_CSV;

CREATE OR REPLACE STAGE CONTROL.STG_CLEAN
  URL = 's3://claimguard360-<yourname>/clean/'
  STORAGE_INTEGRATION = CG_S3_INT  FILE_FORMAT = CONTROL.FF_CSV;

-- Smoke test: upload any small file to landing/claims/ in S3, then:
LIST @CONTROL.STG_LANDING/claims/;
```

If LIST fails with “Access Denied”

(1) Check the bucket name in STORAGE_ALLOWED_LOCATIONS and the stage URL are identical. (2) Re-check the trust policy values. (3) Confirm CG_SNOWFLAKE_S3_POLICY is attached to the role.

Change from the project document

The document lists an **internal stage**. ClaimGuard 360 uses **external stages on S3** because the source is S3 and IDMC writes there. Write this in your architecture decision log.

2.5 Snowflake user for IDMC (key-pair)

IDMC needs its own Snowflake login. Snowflake is phasing out password-only logins for service users, so use a **key pair**.

Generate the key pair

```
# On your laptop (Git Bash / Linux / Mac):
openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out cg_idmc_key.p8 -nocrypt
openssl rsa -in cg_idmc_key.p8 -pubout -out cg_idmc_key.pub
# Copy the text between -----BEGIN PUBLIC KEY----- and -----END PUBLIC KEY----- (one line, no breaks)
```

sql/04_idmc_user.sql

```
USE ROLE SECURITYADMIN;
CREATE USER IF NOT EXISTS CG_IDMC_SVC
  TYPE = SERVICE
  DEFAULT_ROLE = CG_ENGINEER_ROLE
```

```
  DEFAULT_WAREHOUSE = CG_WH
  DEFAULT_NAMESPACE = INSURANCE_DB.CONTROL
  RSA_PUBLIC_KEY = '<paste public key body here>';
GRANT ROLE CG_ENGINEER_ROLE TO USER CG_IDMC_SVC;
```

✔ **Done when:**

- ■ LIST @CONTROL.STG_LANDING/claims/ shows your test file.
- ■ SHOW SCHEMAS IN DATABASE INSURANCE_DB shows the five schemas owned by CG_ENGINEER_ROLE.
- ■ CG_IDMC_SVC exists and has CG_ENGINEER_ROLE.

Phase 3 – CONTROL Tables (the brain of the pipeline)

These tables store every check, reject, alert and access rule. The stand-out features read and write them.

sql/05_control_tables.sql

```
USE ROLE CG_ENGINEER_ROLE;  USE SCHEMA INSURANCE_DB.CONTROL;

-- ★ Control File Check: one row per .ctl file checked
CREATE TABLE IF NOT EXISTS CONTROL.CONTROL_FILE_CHECK (
  CHECK_ID               NUMBER AUTOINCREMENT,
  CTL_FILE_NAME          STRING,
  SOURCE_NAME            STRING,
  DATA_FILE_NAME         STRING,
  BUSINESS_DATE          DATE,
  EXPECTED_ROW_COUNT     NUMBER,
  ACTUAL_ROW_COUNT       NUMBER,
  EXPECTED_AMOUNT_TOTAL  NUMBER(20,2),
  ACTUAL_AMOUNT_TOTAL    NUMBER(20,2),
  STATUS                 STRING,          -- MATCH / MISMATCH
  MESSAGE                STRING,
  CHECK_TS               TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- ★ Gate: latest decision per source, read by the IDMC taskflow
CREATE TABLE IF NOT EXISTS CONTROL.CTL_GATE (
  SOURCE_NAME  STRING,
  GATE_CODE    NUMBER,          -- 0 = NO_FILE, 1 = MATCH, 2 = MISMATCH
  GATE_STATUS  STRING,
  CTL_FILES    STRING,
  UPDATED_TS   TIMESTAMP_NTZ
);
CREATE TABLE IF NOT EXISTS CONTROL.CTL_GATE_LOG (
  SOURCE_NAME STRING, GATE_CODE NUMBER, GATE_STATUS STRING,
  LOGGED_TS TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Bronze load audit (filled from COPY_HISTORY)
CREATE TABLE IF NOT EXISTS CONTROL.LOAD_AUDIT (
  TABLE_NAME STRING, FILE_NAME STRING, STATUS STRING,
  ROW_PARSED NUMBER, ROW_LOADED NUMBER, ERROR_COUNT NUMBER,
  FIRST_ERROR_MESSAGE STRING, LAST_LOAD_TIME TIMESTAMP_LTZ,
  BATCH_ID STRING, LOGGED_TS TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Silver relationship / rule failures
CREATE TABLE IF NOT EXISTS CONTROL.DQ_EXCEPTIONS (
  LAYER STRING, TABLE_NAME STRING, RECORD_KEY STRING, RULE_NAME STRING,
  SEVERITY STRING, DETAILS STRING,
  DETECTED_TS TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Written by IDMC: every duplicate and every reject attempt
CREATE TABLE IF NOT EXISTS CONTROL.DUPLICATE_LOG (
  SOURCE_NAME STRING, RECORD_KEY STRING, RAW_RECORD STRING,
  LOGGED_TS TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);
CREATE TABLE IF NOT EXISTS CONTROL.REJECT_LOG (
  REJECT_ID STRING, SOURCE_NAME STRING, RECORD_KEY STRING,
  REJECT_REASON STRING, RAW_RECORD STRING,
  ATTEMPT_TYPE STRING,             -- INITIAL / RESUBMIT
  LOGGED_TS TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- ★ Reject register: one row per reject, life-cycle status
CREATE TABLE IF NOT EXISTS CONTROL.REJECT_REGISTER (
  REJECT_ID STRING, SOURCE_NAME STRING, RECORD_KEY STRING,
  FIRST_REASON STRING, LAST_REASON STRING, ATTEMPT_COUNT NUMBER,
  STATUS STRING,                   -- OPEN / RESUBMITTED_FAILED / RESOLVED / CLOSED_NOT_FIXABLE
  FIRST_SEEN_TS TIMESTAMP_NTZ, LAST_ATTEMPT_TS TIMESTAMP_NTZ,
  RESOLVED_TS TIMESTAMP_NTZ, UPDATED_TS TIMESTAMP_NTZ
);

-- ★ Threshold alerts
CREATE TABLE IF NOT EXISTS CONTROL.ALERT_THRESHOLDS (
  METRIC_NAME STRING, DESCRIPTION STRING, METRIC_SQL STRING,
```

```
  OPERATOR STRING, THRESHOLD_VALUE NUMBER(20,2), SEVERITY STRING,
  RECIPIENTS STRING, IS_ACTIVE BOOLEAN DEFAULT TRUE
);
CREATE TABLE IF NOT EXISTS CONTROL.ALERT_HISTORY (
  METRIC_NAME STRING, METRIC_VALUE NUMBER(20,2), OPERATOR STRING,
  THRESHOLD_VALUE NUMBER(20,2), SEVERITY STRING, MESSAGE STRING,
  EMAIL_SENT BOOLEAN, ALERT_TS TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- ⋆ Region-based access: who may see which region
CREATE TABLE IF NOT EXISTS CONTROL.USER_REGION_MAP (
  PRINCIPAL_TYPE STRING,          -- ROLE or USER
  PRINCIPAL_NAME STRING,          -- e.g. INSURANCE_ANALYST_SOUTH_ROLE or JDOE
  REGION STRING,                  -- upper case region value, or ALL
  IS_ACTIVE BOOLEAN DEFAULT TRUE
);
```

✔ Done when:

■ SHOW TABLES IN SCHEMA CONTROL returns 11 tables.

Phase 4 – ★ Control File Check (stored procedure)

In simple words: before IDMC touches a data file, Snowflake opens the matching .ctl file and the data file directly in S3, counts the rows, adds up the amount column and compares both with what the .ctl promised. The answer is saved in CONTROL_FILE_CHECK and a one-line “gate” decision per source is saved in CTL_GATE. IDMC then reads the gate and decides: clean the file, quarantine it, or skip (no new file).

4.1 How it decides

Situation	STATUS	Gate
No new .ctl file for the source	–	0 = NO_FILE → IDMC skips the source
Row count and amount total both equal	MATCH	1 = MATCH → IDMC cleans the file
Data file missing, count differs, amount differs (> 0.01), or bad .ctl	MISMATCH	2 = MISMATCH → quarantine + alert
.ctl file already checked before (same name)	ignored	Duplicate-batch protection: a replayed .ctl is never re-processed

4.2 Create the procedure

sql/06_sp_control_check.sql

```
CREATE OR REPLACE PROCEDURE CONTROL.SP_CONTROL_CHECK(P_SOURCE STRING)
RETURNS STRING
LANGUAGE PYTHON
RUNTIME_VERSION = '3.11'
PACKAGES = ('snowflake-snowpark-python')
HANDLER = 'run'
EXECUTE AS CALLER
AS
$$
import re

SOURCES = {'customers', 'insurance_products', 'policies', 'premium_payments',
           'claims', 'claim_settlements', 'fraud_cases'}
SAFE = re.compile(r'^[A-Za-z0-9_.\-]+$')


def check_one(session, src, ctl):
    """Returns (status, message, data_file, business_date, exp_rows, act_rows, exp_amt, act_amt)."""
    rows = session.sql(
        f"SELECT $1, $2, TRY_TO_NUMBER($3), TRY_TO_NUMBER($4), TRY_TO_NUMBER($5, 20, 2), "
        f"TRY_TO_DATE($6) FROM @CONTROL.STG_CONTROL/{ctl} (FILE_FORMAT => 'CONTROL.FF_CSV')").collect()
    if len(rows) != 1:
        return ('MISMATCH', 'Control file must contain exactly one data line',
                None, None, None, None, None, None)
    _, data_file, exp_rows, pos, exp_amt, bdate = rows[0]
    if not data_file or not SAFE.match(data_file):
        return ('MISMATCH', 'Invalid data_file_name in control file',
                data_file, bdate, exp_rows, None, exp_amt, None)
    if not session.sql(f"LIST @CONTROL.STG_LANDING/{src}/{data_file}").collect():
        return ('MISMATCH', 'Data file not found in landing/' + src,
                data_file, bdate, exp_rows, 0, exp_amt, None)
    pos = int(pos or 0)
    amt_expr = (f"SUM(TRY_TO_NUMBER(REPLACE(${pos}, ',', ''), 20, 2))" if pos > 0 else "NULL")
    act_rows, act_amt = session.sql(
        f"SELECT COUNT(*), {amt_expr} FROM @CONTROL.STG_LANDING/{src}/{data_file} "
        f"(FILE_FORMAT => 'CONTROL.FF_CSV')").collect()[0]
    problems = []
    if exp_rows is None or int(exp_rows) != int(act_rows):
        problems.append(f'row count expected {exp_rows}, found {act_rows}')
    if pos > 0 and (exp_amt is None or act_amt is None or abs(float(exp_amt) - float(act_amt)) > 0.01):
        problems.append(f'amount total expected {exp_amt}, found {act_amt}')
    status = 'MISMATCH' if problems else 'MATCH'
    msg = '; '.join(problems) if problems else 'Row count and amount total match'
    return (status, msg, data_file, bdate, exp_rows, act_rows, exp_amt, act_amt)


def set_gate(session, src, code, status, files):
```

```
    session.sql("DELETE FROM CONTROL.CTL_GATE WHERE SOURCE_NAME = ?", params=[src]).collect()
    session.sql("INSERT INTO CONTROL.CTL_GATE (SOURCE_NAME, GATE_CODE, GATE_STATUS, CTL_FILES, UPDATED_TS) "
                "VALUES (?, ?, ?, ?, CURRENT_TIMESTAMP())", params=[src, code, status, files]).collect()


def run(session, p_source):
    src = (p_source or '').strip().lower()
    if src not in SOURCES:
        return 'ERROR: unknown source ' + src
    listed = session.sql(
        f"LIST @CONTROL.STG_CONTROL PATTERN = '.*{src}_[0-9]{{8}}[.]ctl'").collect()
    names = {r['name'].split('/')[-1] for r in listed}
    names = {n for n in names if re.match(rf'^{src}_[0-9]{{8}}[.]ctl$', n)}   # exact source only
    done = {r[0] for r in session.sql(
        "SELECT CTL_FILE_NAME FROM CONTROL.CONTROL_FILE_CHECK WHERE SOURCE_NAME = ?",
        params=[src]).collect()}
    new_files = sorted(names - done)
    if not new_files:
        set_gate(session, src, 0, 'NO_FILE', '')
        return 'NO_FILE'
    worst = 1
    for ctl in new_files:
        st, msg, dfile, bdate, er, ar, ea, aa = check_one(session, src, ctl)

        session.sql(
            "INSERT INTO CONTROL.CONTROL_FILE_CHECK (CTL_FILE_NAME, SOURCE_NAME, DATA_FILE_NAME, "
            "BUSINESS_DATE, EXPECTED_ROW_COUNT, ACTUAL_ROW_COUNT, EXPECTED_AMOUNT_TOTAL, "
            "ACTUAL_AMOUNT_TOTAL, STATUS, MESSAGE) VALUES (?,?,?,?,?,?,?,?,?,?)",
            params=[ctl, src, dfile, bdate, er, ar, ea, aa, st, msg]).collect()
        if st != 'MATCH':
            worst = 2
    set_gate(session, src, worst, 'MATCH' if worst == 1 else 'MISMATCH', ','.join(new_files))
    return 'MATCH' if worst == 1 else 'MISMATCH'
$$;
```

> **Why EXECUTE AS CALLER?**
>
> The procedure runs LIST on stages. Running it with the caller's rights (IDMC logs in as CG_ENGINEER_ROLE) avoids owner's-rights restrictions and keeps all privileges in one role.

4.3 Test it by hand before wiring IDMC

1. Upload **claims_20261103.csv** to landing/claims/ and **claims_20261103.ctl** to control/.
2. Run the statements below. Expected: MATCH, and one row in each table.
3. Edit the .ctl to a wrong row count, save it as **claims_20261104.ctl** (new name), upload, run again. Expected: MISMATCH with the message “row count expected …, found …”.
4. Run the CALL a third time. Expected: NO_FILE (both .ctl files already checked).

```
CALL CONTROL.SP_CONTROL_CHECK('claims');
SELECT * FROM CONTROL.CONTROL_FILE_CHECK ORDER BY CHECK_ID DESC;
SELECT * FROM CONTROL.CTL_GATE;
-- reset after testing:
-- DELETE FROM CONTROL.CONTROL_FILE_CHECK;  DELETE FROM CONTROL.CTL_GATE;
```

✔ **Done when:**

- ■ MATCH, MISMATCH and NO_FILE all demonstrated (screenshot each).
- ■ Test rows deleted and test files removed from S3 before the real run.

Phase 5 – IDMC: Connections, Cleansing and Routing

Goal: one cleansing mapping per source that standardises formats, applies the data-quality rules from the project document and writes every record to exactly one route: **CLEAN**, **WARNING** (kept), **DUPLICATE** or **REJECT**.

5.1 Create the two connections

IDMC → **Administrator** → **Connections** → **New Connection**.

Property	CONN_S3_CG (type: Amazon S3 V2)	CONN_SF_CG (type: Snowflake Data Cloud)
Runtime environment	Your Secure Agent group	Your Secure Agent group
Authentication	Access Key / Secret Key from Phase 1.3	KeyPair – user CG_IDMC_SVC, private key file cg_idmc_key.p8
Location	Folder Path: claimguard360-<yourname> • Region: ap-south-1	Account: <orgname-accountname> • Warehouse: CG_WH • Role: CG_ENGINEER_ROLE
Extra	–	Additional JDBC URL parameters: db=INSURANCE_DB&schema=CONTROL
Test	Click **Test Connection** → “successful”	Click **Test Connection** → “successful”

> **Tip**
>
> The private key file must be on the Secure Agent machine (e.g. /opt/infa/keys/cg_idmc_key.p8). Give the agent's OS user read permission on it.

5.2 Prepare S3 helper files (one-time)

IDMC needs a sample file at design time to see the fields, and existing targets give you an exact column order (an acceptance criterion). Upload these **header-only** CSV files:

Upload to	Content	Why
clean/<source>/<source>_clean.csv	documented columns + dq_status, dq_reason, reject_id	Target for CLEAN + WARNING rows (loaded to Bronze)
clean/<source>/<source>_resubmit_clean.csv	same as above	Target for fixed rejects (Phase 11)
warning/<source>/<source>_warning.csv	documented columns + dq_reason	Evidence of warning rows
duplicate/<source>/<source>_duplicate.csv	documented columns	Evidence of duplicates
reject/<source>/<source>_reject.csv	documented columns + reject_id, reject_reason	Rows the business must fix
resubmit/<source>/_placeholder.csv	reject layout header only	Keeps the folder readable when empty

Example header: clean/claims/claims_clean.csv (one line in the real file)

```
claim_id,policy_id,customer_id,claim_registration_date,loss_date,claim_type,claim_amount,approved_amount,
claim_status,assessor_id,branch_id,source_updated_ts,dq_status,dq_reason,reject_id
```

5.3 The data-quality rules per source

> **Confirm with profiling (Day 1)**
>
> Run IDMC **Data Profiling** (or open the files in Excel) and confirm: (a) the exact 4 date/timestamp formats, (b) which columns hold the 8 nulls – they must be optional columns, (c) the real status values (e.g. ACTIVE / LAPSED, APPROVED / REJECTED). Adjust the rules below if your data differs. The document fixes the expected totals: **8 nulls, 3 duplicates, 3 anomalies, 2 outliers, 4 date formats**.

Source	Key (dedupe)	Mandatory → REJECT if null	Optional → WARNING if null	Business rules
customers	customer_id	customer_id, customer_name	email, phone, address_line, occupation, gender	–
insurance_products	product_id	product_id, product_name	coverage_category	–
policies	policy_id	policy_id, customer_id, product_id, policy_start_date	agent_id, nominee_name	**REJECT**: policy_expiry_date < policy_start_date
premium_payments	payment_id	payment_id, policy_id, payment_date	reference_number, payment_method	–
claims	claim_id	claim_id, policy_id, loss_date	approved_amount, assessor_id, branch_id	**REJECT**: loss_date < policy_start_date (lookup) • **WARNING**: claim_amount ≥ 10,000,000
claim_settlements	settlement_id	settlement_id, claim_id	payment_reference	**REJECT**: |net − (gross − deductible − recovery)| > 1
fraud_cases	fraud_case_id	fraud_case_id, claim_id	investigation_end_date, investigation_outcome, investigator_id	**WARNING**: fraud_score < 0 or > 100

All sources: every date and timestamp is standardised to **YYYY-MM-DD** / **YYYY-MM-DD HH24:MI:SS**; a value that cannot be parsed in any known format is a **REJECT** (“Unparseable date”). Route priority when several rules hit one row: **REJECT → DUPLICATE → WARNING → CLEAN**.

5.4 Build m_clean_claims (the full worked example)

Data Integration → **New** → **Mappings** → **Mapping** → name **m_clean_claims**. Build the flow below left to right. The other six mappings follow the same pattern (5.5).

Mapping flow

```
SRC_CLAIMS -> EXP_STANDARDISE -> LKP_POLICY_START -> SRT_DEDUP -> EXP_DQ -> RTR_ROUTE
                                                                              |
     +--------------------------------------------------------------------------+
     |-> CLEAN_OR_WARN -> TGT_CLEAN   (S3)
     |-> WARNING       -> TGT_WARNING (S3)
     |-> DUPLICATE     -> TGT_DUP_S3 (S3)  + TGT_DUP_LOG (Snowflake CONTROL.DUPLICATE_LOG)
     |-> REJECT        -> TGT_REJ_S3 (S3)  + TGT_REJ_LOG (Snowflake CONTROL.REJECT_LOG)
```

Step A – Source SRC_CLAIMS

1. Connection **CONN_S3_CG**, Source Type **Single Object**, Object: browse to **landing/claims/claims_20261103.csv** (sample, gives the fields).
2. **Formatting Options**: Delimited, delimiter comma, text qualifier double quote, **Import column names from first line**, first data row 2.
3. **Advanced** → Source Type: **Directory**, Folder Path: **claimguard360-<yourname>/landing/claims**. The mapping now reads *every* file in the folder,

so you never hard-code a file name.

4. **Fields** tab: keep all fields as **string** (precision 255). Converting happens in the next step.

> **Fixes the problem "fields disappear when the source is a parameter"**
>
> Do not parameterise the object. Use a sample object for design time and **Source Type = Directory** for run time. Fields stay visible and you can still process any file name.

Step B – Expression EXP_STANDARDISE

Add an Expression. Create the fields below (type **v** = variable, **o** = output). Repeat the date pattern for every date column; the four formats shown are examples – use the four your profiling found.

EXP_STANDARDISE fields

```
-- ---------- dates: try each known format, NULL if none fits ----------
v_loss_date  (date/time) =
  IIF(IS_DATE(LTRIM(RTRIM(loss_date)),'YYYY-MM-DD'), TO_DATE(LTRIM(RTRIM(loss_date)),'YYYY-MM-DD'),
  IIF(IS_DATE(LTRIM(RTRIM(loss_date)),'DD-MM-YYYY'), TO_DATE(LTRIM(RTRIM(loss_date)),'DD-MM-YYYY'),
  IIF(IS_DATE(LTRIM(RTRIM(loss_date)),'DD/MM/YYYY'), TO_DATE(LTRIM(RTRIM(loss_date)),'DD/MM/YYYY'),
  IIF(IS_DATE(LTRIM(RTRIM(loss_date)),'DD-MON-YYYY'),TO_DATE(LTRIM(RTRIM(loss_date)),'DD-MON-YYYY'),
  NULL))))
v_reg_date   (date/time) = same pattern on claim_registration_date
v_upd_ts     (date/time) =
  IIF(IS_DATE(source_updated_ts,'YYYY-MM-DD HH24:MI:SS'), TO_DATE(source_updated_ts,'YYYY-MM-DD HH24:MI:SS'),
  IIF(IS_DATE(source_updated_ts,'DD-MM-YYYY HH24:MI:SS'), TO_DATE(source_updated_ts,'DD-MM-YYYY HH24:MI:SS'),
  IIF(IS_DATE(source_updated_ts,'DD/MM/YYYY HH24:MI'),    TO_DATE(source_updated_ts,'DD/MM/YYYY HH24:MI'),
  IIF(IS_DATE(source_updated_ts,'MM/DD/YYYY HH24:MI:SS'), TO_DATE(source_updated_ts,'MM/DD/YYYY HH24:MI:SS'),
  NULL))))

-- ---------- outputs (all string, in standard format) ----------
o_claim_id                (string)  = UPPER(LTRIM(RTRIM(claim_id)))
o_policy_id               (string)  = UPPER(LTRIM(RTRIM(policy_id)))
o_customer_id             (string)  = UPPER(LTRIM(RTRIM(customer_id)))
o_claim_registration_date (string)  = TO_CHAR(v_reg_date,'YYYY-MM-DD')
o_loss_date               (string)  = TO_CHAR(v_loss_date,'YYYY-MM-DD')
o_claim_type              (string)  = UPPER(LTRIM(RTRIM(claim_type)))
o_claim_amount            (string)  = IIF(IS_NUMBER(REPLACECHR(0,claim_amount,',','')),
                                        REPLACECHR(0,claim_amount,',',''), NULL)
o_approved_amount         (string)  = IIF(IS_NUMBER(REPLACECHR(0,approved_amount,',','')),
                                        REPLACECHR(0,approved_amount,',',''), NULL)
o_claim_status            (string)  = UPPER(LTRIM(RTRIM(claim_status)))
o_assessor_id             (string)  = LTRIM(RTRIM(assessor_id))
o_branch_id               (string)  = LTRIM(RTRIM(branch_id))
o_source_updated_ts       (string)  = TO_CHAR(v_upd_ts,'YYYY-MM-DD HH24:MI:SS')
o_loss_date_dt            (date)    = v_loss_date        -- kept for comparisons
o_bad_date_flag           (integer) = IIF((NOT ISNULL(loss_date) AND ISNULL(v_loss_date)) OR
                                        (NOT ISNULL(claim_registration_date) AND ISNULL(v_reg_date)), 1, 0)
o_raw_record              (string)  = claim_id||'|'||policy_id||'|'||customer_id||'|'||
                                      claim_registration_date||'|'||loss_date||'|'||claim_type||'|'||
                                      claim_amount||'|'||approved_amount||'|'||claim_status||'|'||
                                      assessor_id||'|'||branch_id||'|'||source_updated_ts
```

> **Reuse tip**
>
> Create the 4-format date logic once as an IDMC **User-Defined Function** (New → Components → User-Defined Function, e.g. *udf_to_std_date(in_str)*) and call it in every mapping. One change then fixes all seven mappings.

Step C – Lookup LKP_POLICY_START

1. Add a **Lookup**: connection **CONN_SF_CG**, object **SILVER.POLICIES**.
2. Lookup condition: **POLICY_ID = o_policy_id**. Return field: **POLICY_START_DATE**. Multiple matches: **Return any row**.
3. Why Snowflake and not the S3 file? Silver already has the latest valid version of every policy, whatever batch it came in.

> **Load order rule**
>
> Because of this lookup, the **very first** run must load the masters first: upload customers, insurance_products and policies (wave 1), let the Snowflake task graph build Silver (Phase 10), then upload the transactional files (wave 2). Silver also repeats this check as a safety net (Phase 8).

Step D – Sorter SRT_DEDUP

Sort by **o_claim_id ascending**, then **o_source_updated_ts descending**. The newest version of each claim now comes first; any later row with the same key is a duplicate.

Step E – Expression EXP_DQ (route decision)

Variable fields are evaluated **top to bottom**, so keep them in exactly this order.

EXP_DQ fields

```
v_row_no      (integer) = v_row_no + 1
v_is_dup      (integer) = IIF(o_claim_id = v_prev_key, 1, 0)
v_prev_key    (string)  = o_claim_id                        -- must come AFTER v_is_dup
v_reject_reason (string) =
   IIF(ISNULL(o_claim_id) OR ISNULL(o_policy_id) OR ISNULL(o_loss_date), 'Mandatory field missing',
   IIF(o_bad_date_flag = 1, 'Unparseable date',
   IIF(NOT ISNULL(POLICY_START_DATE) AND o_loss_date_dt < POLICY_START_DATE,
       'Loss date before policy start', NULL)))
v_warn_reason (string) =
   IIF(TO_DECIMAL(o_claim_amount,2) >= 10000000, 'Extreme claim amount (outlier); ', '') ||
   IIF(ISNULL(o_approved_amount), 'approved_amount null; ', '') ||
   IIF(ISNULL(o_assessor_id), 'assessor_id null; ', '') ||
   IIF(ISNULL(o_branch_id),   'branch_id null; ', '')

o_dq_route  (string) = IIF(NOT ISNULL(v_reject_reason), 'REJECT',
                       IIF(v_is_dup = 1, 'DUPLICATE',
                       IIF(LENGTH(v_warn_reason) > 0, 'WARNING', 'CLEAN')))
o_dq_status (string) = IIF(o_dq_route = 'WARNING', 'WARNING', 'CLEAN')
o_dq_reason (string) = IIF(o_dq_route = 'REJECT', v_reject_reason,
                       IIF(o_dq_route = 'DUPLICATE', 'Duplicate of ' || o_claim_id,
                       IIF(LENGTH(v_warn_reason) > 0, v_warn_reason, NULL)))
o_reject_id (string) = IIF(o_dq_route = 'REJECT',
                       'REJ-CLAIMS-' || TO_CHAR(SYSDATE,'YYYYMMDDHH24MISS') || '-' || TO_CHAR(v_row_no), NULL)
o_source_name  (string) = 'claims'
o_attempt_type (string) = 'INITIAL'
```

Step F – Router RTR_ROUTE

Group	Condition	Goes to
CLEAN_OR_WARN	IN(o_dq_route,'CLEAN','WARNING')	TGT_CLEAN
WARNING	o_dq_route = 'WARNING'	TGT_WARNING (evidence copy)
DUPLICATE	o_dq_route = 'DUPLICATE'	TGT_DUP_S3 and TGT_DUP_LOG
REJECT	o_dq_route = 'REJECT'	TGT_REJ_S3 and TGT_REJ_LOG

A warning row deliberately goes to two groups: it is **kept** (CLEAN_OR_WARN) and **reported** (WARNING).

Step G – Targets

Target	Connection / object	Operation	Field mapping
TGT_CLEAN	CONN_S3_CG • existing object clean/claims/claims_clean.csv	Insert (overwrite file)	12 o_ business fields → documented columns in order; o_dq_status → dq_status; o_dq_reason → dq_reason; reject_id left **unmapped** (empty)
TGT_WARNING	S3 • warning/claims/claims_warning.csv	Insert	business fields + o_dq_reason
TGT_DUP_S3	S3 • duplicate/claims/claims_duplicate.csv	Insert	business fields
TGT_DUP_LOG	CONN_SF_CG • CONTROL.DUPLICATE_LOG	Insert	o_source_name→SOURCE_NAME, o_claim_id→RECORD_KEY, o_raw_record→RAW_RECORD
TGT_REJ_S3	S3 • reject/claims/claims_reject.csv	Insert	business fields + o_reject_id + o_dq_reason

Target	Connection / object	Operation	Field mapping
TGT_REJ_LOG	CONN_SF_CG • CONTROL.REJECT_LOG	Insert	o_reject_id, o_source_name, o_claim_id→RECORD_KEY, o_dq_reason→REJECT_REASON, o_raw_record, o_attempt_type

Common mistake

Mapping fields **by name** to a CSV target can silently re-order columns. Use the header-only target file from 5.2 and check the **Field Mapping** tab top to bottom against the document order.

Save → **Validate** (top right, must be green) → **Run** once against the sample file and open the output files in S3.

5.5 The other six mappings (same pattern, Save As)

Open m_clean_claims → **Save As** → rename → change the source object/folder, the fields in EXP_STANDARDISE, the rules in EXP_DQ (use table 5.3) and the targets. Differences:

Mapping	What changes
m_clean_customers	No lookup. Dates: date_of_birth, registration_date. Warnings: email/phone/address_line/occupation null. Tip: UPPER the region so region security works later.
m_clean_insurance_products	No lookup. Numbers: minimum/maximum_sum_insured, grace_period_days. active_flag → 'TRUE'/'FALSE' with IIF(UPPER(x) IN ('Y','YES','1','TRUE'),'TRUE','FALSE').
m_clean_policies	No lookup. Reject: TO_DATE(expiry) < TO_DATE(start). Warnings: agent_id, nominee_name null.
m_clean_premium_payments	Optional lookup on SILVER.POLICIES only to flag unknown policy as WARNING (Silver also checks).
m_clean_claim_settlements	Reject: ABS(TO_DECIMAL(net,2) − (TO_DECIMAL(gross,2) − NVL(TO_DECIMAL(deductible,2),0) − NVL(TO_DECIMAL(recovery,2),0))) > 1.
m_clean_fraud_cases	Warning: TO_DECIMAL(fraud_score,2) > 100 OR < 0 → 'Fraud score outside 0–100 (outlier)'.

5.6 Mapping tasks

For each mapping: New → **Tasks** → **Mapping Task** → name **mt_clean_<source>** → select the mapping → runtime environment → Save. Run each once by hand and check the counts.

✔ **Done when:**

- ■ 7 mappings validate (green) and 7 mapping tasks run successfully on the sample files.
- ■ Across all sources: 3 rows in REJECT_LOG, 3 rows in DUPLICATE_LOG, 10 warning rows (8 nulls + 2 outliers) unless one row has two warnings.
- ■ clean/ files open in Excel with columns in the documented order + dq_status, dq_reason, reject_id.

Phase 6 – ★ IDMC File Listener and Taskflow

In simple words: the source system drops the data files, then the .ctl files, then one tiny **_BATCH_READY.done** file. The File Listener sees the .done file and starts the taskflow. The taskflow walks through the seven sources in dependency order; for each one it asks Snowflake "is the control file OK?" and then cleans, quarantines or skips that source. Finally it archives everything it processed.

6.1 The gate mapping m_ctl_check (one mapping for all sources)

1. New mapping **m_ctl_check**. Create two **in-out parameters** (Parameters panel → In-Out Parameters):
   • **P_SOURCE** – String, default *claims*
   • **P_GATE_CODE** – Integer, default 0, Aggregation type **Max**
2. Source: connection CONN_SF_CG, Source Type **Query**:
   `SELECT SOURCE_NAME, GATE_CODE, GATE_STATUS FROM INSURANCE_DB.CONTROL.CTL_GATE WHERE SOURCE_NAME = '$$P_SOURCE'`
3. Source → Advanced → **Pre SQL**:
   `CALL INSURANCE_DB.CONTROL.SP_CONTROL_CHECK('$$P_SOURCE')`
   Pre SQL runs *before* the query, so the gate row is always fresh.
4. Expression EXP_GATE: output field **o_set_gate** (integer) = `SETVARIABLE($$P_GATE_CODE, GATE_CODE)`.
5. Target: CONN_SF_CG → **CONTROL.CTL_GATE_LOG** (Insert): SOURCE_NAME, GATE_CODE, GATE_STATUS.
6. Validate → create mapping task **mt_ctl_check**.

Why reset P_GATE_CODE every time?

In-out parameters remember their last value between runs. The taskflow therefore passes **P_GATE_CODE = 0** on every call, so a MISMATCH from yesterday can never leak into today.

6.2 Move/archive script on the Secure Agent

Snowflake cannot move S3 files, so the taskflow uses a small script through a **Command Task**. The Secure Agent machine needs the AWS CLI configured with the cg-idmc-user keys (`aws configure`).

cg_move.sh (chmod +x)

```
#!/bin/bash
# /opt/cg/scripts/cg_move.sh  <from_prefix>  <to_prefix>
# example: cg_move.sh landing/claims/  archive/claims/
set -e
BUCKET="claimguard360-<yourname>"
DAY=$(date +%Y/%m/%d)
aws s3 mv "s3://$BUCKET/$1" "s3://$BUCKET/$2$DAY/" --recursive --exclude "_placeholder.csv"
echo "moved s3://$BUCKET/$1 -> s3://$BUCKET/$2$DAY/"
```

No AWS CLI on the agent?

Use a **File Ingestion and Replication** task (Amazon S3 V2 → Amazon S3 V2) with the option to delete source files after transfer, and call it from the taskflow instead of the Command Task. Check the exact option name in your IDMC org.

6.3 Build the taskflow tf_cg_ingest

New → **Taskflows** → **Taskflow** → name **tf_cg_ingest**. For **each source in this order**: customers, insurance_products, policies, premium_payments, claims, claim_settlements, fraud_cases, add the block below.

Taskflow design (repeat block × 7)

```
[Data Task]  Gate_<source>      task = mt_ctl_check
             Input fields:  P_SOURCE = '<source>'     P_GATE_CODE = 0
                  |
[Decision]   Is_<source>_ok?    field = Gate_<source>.P_GATE_CODE   (output in-out parameter)
     |-- = 1 (MATCH)    -> [Data Task] Clean_<source>    task = mt_clean_<source>
     |                      -> [Command Task] Archive_<source>
     |                           script: /opt/cg/scripts/cg_move.sh   args: landing/<source>/ archive/<source>/
     |-- = 2 (MISMATCH) -> [Command Task] Quarantine_<source>
     |                           script: /opt/cg/scripts/cg_move.sh   args: landing/<source>/ quarantine/<source>/
     |-- otherwise (0)  -> continue (nothing to do)
                  |
             (all three paths join and continue to the next source)

After fraud_cases:
[Command Task] Archive_control   args: control/  archive/control/
[End]
```

1. Drag a **Data Task** → Task: mt_ctl_check → **Input Fields** → set P_SOURCE and P_GATE_CODE as shown.
2. Drag a **Decision** → Field: pick **Gate_<source>** → **P_GATE_CODE** from the output fields → add paths “Equals 1”, “Equals 2”, and **Otherwise**.
3. Leave the Data Task error handling at its default (a failed task **stops the taskflow**), so a broken mapping can never be followed by archiving a file that was never cleaned.
4. Command Task: Script File Name **/opt/cg/scripts/cg_move.sh**, Input Arguments as above, Runtime Environment = your agent group.
5. Use **Jump** or simply connect the end of each path to the next source's Gate step.

6.4 Create the File Listener fl_cg_batch

1. New → **Components** → **File Listener** → name **fl_cg_batch**.
2. Connection Type: **Amazon S3 V2** → Connection **CONN_S3_CG**. Runtime environment: your agent.
3. Folder Path: **control**. File Pattern: ***.done** (pattern type: wildcard).
4. Events to listen for: **Arrive** only.
5. Schedule: run **daily**, between **08:00 and 20:00**, check every **2 minutes**, time zone **Asia/Kolkata**.
6. Tick **Check file stability** (waits until the file size stops changing) – protects against half-uploaded files.
7. Save → in the list view, select the listener → **Start**.

6.5 Bind the taskflow to the listener

1. Open tf_cg_ingest → click the **Start** step → **Binding**: **Event**.
2. **Event Source Name**: select **fl_cg_batch**.
3. Save → **Publish** (top right). A taskflow bound to an event runs only after it is published.
4. Last step of the taskflow: also move the .done file to archive (it is in control/, so Archive_control already does it).

Why a .done file instead of listening to every .ctl?

Seven .ctl files would start seven overlapping runs. One .done file = one batch = one run, and masters are always processed before the claims that depend on them.

6.6 Test the automatic start

1. Upload customers_20261103.csv → landing/customers/, then customers_20261103.ctl → control/.
2. Upload an empty file named **_BATCH_READY_20261103.done** → control/.
3. Within ~2 minutes: Monitor → **All Jobs** shows tf_cg_ingest running. Gate_customers = 1, Clean_customers runs, Archive_customers moves the file; all other sources take the “otherwise” path.
4. In Snowflake: `SELECT * FROM CONTROL.CTL_GATE_LOG ORDER BY LOGGED_TS DESC;`

✔ **Done when:**

■ File Listener status = Running; taskflow starts by itself (screenshot of Monitor with start time).

■ A wrong .ctl moves the data file to quarantine/ and is logged as MISMATCH.

■ Processed files are in archive/<source>/yyyy/mm/dd/ and landing/ is empty.

Phase 7 – Bronze Layer (raw copy + audit)

In simple words: Bronze is an exact copy of the clean files. Every column is STRING so loading never fails on a type problem, and four extra columns record *where* and *when* each row came from. The documented columns come first, in the documented order (acceptance criterion).

7.1 Create the seven Bronze tables

sql/10_bronze_tables.sql

```
USE ROLE CG_ENGINEER_ROLE;  USE WAREHOUSE CG_WH;  USE SCHEMA INSURANCE_DB.BRONZE;

CREATE TABLE IF NOT EXISTS BRONZE.CUSTOMERS (
  CUSTOMER_ID             STRING,
  CUSTOMER_NAME           STRING,
  DATE_OF_BIRTH           STRING,
  GENDER                  STRING,
  EMAIL                   STRING,
  PHONE                   STRING,
  ADDRESS_LINE            STRING,
  CITY                    STRING,
  STATE                   STRING,
  REGION                  STRING,
  OCCUPATION              STRING,
  CUSTOMER_RISK_CATEGORY  STRING,
  REGISTRATION_DATE       STRING,
  SOURCE_UPDATED_TS       STRING,
  DQ_STATUS               STRING,
  DQ_REASON               STRING,
  REJECT_ID               STRING,
  _FILE_NAME              STRING,
  _FILE_ROW_NUMBER        NUMBER,
  _LOAD_TS                TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  _BATCH_ID               STRING
);

CREATE TABLE IF NOT EXISTS BRONZE.INSURANCE_PRODUCTS (
  PRODUCT_ID           STRING,
  PRODUCT_NAME         STRING,
  INSURANCE_TYPE       STRING,
  COVERAGE_CATEGORY    STRING,
  MINIMUM_SUM_INSURED  STRING,
  MAXIMUM_SUM_INSURED  STRING,
  PREMIUM_FREQUENCY    STRING,
  GRACE_PERIOD_DAYS    STRING,
  ACTIVE_FLAG          STRING,
  SOURCE_UPDATED_TS    STRING,
  DQ_STATUS            STRING,
  DQ_REASON            STRING,
  REJECT_ID            STRING,
  _FILE_NAME           STRING,
  _FILE_ROW_NUMBER     NUMBER,
  _LOAD_TS             TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  _BATCH_ID            STRING
);

CREATE TABLE IF NOT EXISTS BRONZE.POLICIES (
  POLICY_ID               STRING,
  CUSTOMER_ID             STRING,
  PRODUCT_ID              STRING,
  AGENT_ID                STRING,
  POLICY_ISSUE_DATE       STRING,
  POLICY_START_DATE       STRING,
  POLICY_EXPIRY_DATE      STRING,
  SUM_INSURED             STRING,
  PREMIUM_AMOUNT          STRING,
  PREMIUM_FREQUENCY       STRING,
  POLICY_STATUS           STRING,
  NOMINEE_NAME            STRING,
  UNDERWRITING_RISK_BAND  STRING,
  SOURCE_UPDATED_TS       STRING,
  DQ_STATUS               STRING,
  DQ_REASON               STRING,
  REJECT_ID               STRING,
  _FILE_NAME              STRING,
  _FILE_ROW_NUMBER        NUMBER,
  _LOAD_TS                TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  _BATCH_ID               STRING
);
```

```
CREATE TABLE IF NOT EXISTS BRONZE.PREMIUM_PAYMENTS (
  PAYMENT_ID           STRING,
  POLICY_ID            STRING,
  PREMIUM_SCHEDULE_ID  STRING,
  PAYMENT_DATE         STRING,
  PAYMENT_AMOUNT       STRING,
  PAYMENT_METHOD       STRING,
  PAYMENT_STATUS       STRING,
  REFERENCE_NUMBER     STRING,
  SOURCE_UPDATED_TS    STRING,
  DQ_STATUS            STRING,
  DQ_REASON            STRING,
  REJECT_ID            STRING,
  _FILE_NAME           STRING,
  _FILE_ROW_NUMBER     NUMBER,
  _LOAD_TS             TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  _BATCH_ID            STRING
);

CREATE TABLE IF NOT EXISTS BRONZE.CLAIMS (
  CLAIM_ID                 STRING,
  POLICY_ID                STRING,
  CUSTOMER_ID              STRING,
  CLAIM_REGISTRATION_DATE  STRING,
  LOSS_DATE                STRING,
  CLAIM_TYPE               STRING,
  CLAIM_AMOUNT             STRING,
  APPROVED_AMOUNT          STRING,
  CLAIM_STATUS             STRING,
  ASSESSOR_ID              STRING,
  BRANCH_ID                STRING,
  SOURCE_UPDATED_TS        STRING,
  DQ_STATUS                STRING,
  DQ_REASON                STRING,
  REJECT_ID                STRING,
  _FILE_NAME               STRING,
  _FILE_ROW_NUMBER         NUMBER,
  _LOAD_TS                 TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  _BATCH_ID                STRING
);

CREATE TABLE IF NOT EXISTS BRONZE.CLAIM_SETTLEMENTS (
  SETTLEMENT_ID            STRING,
  CLAIM_ID                 STRING,
  SETTLEMENT_DATE          STRING,
  SETTLEMENT_TYPE          STRING,
  GROSS_SETTLEMENT_AMOUNT  STRING,
  DEDUCTIBLE_AMOUNT        STRING,
  RECOVERY_AMOUNT          STRING,
  NET_SETTLEMENT_AMOUNT    STRING,
  PAYMENT_STATUS           STRING,
  PAYMENT_REFERENCE        STRING,
  SOURCE_UPDATED_TS        STRING,
  DQ_STATUS                STRING,
  DQ_REASON                STRING,
  REJECT_ID                STRING,
  _FILE_NAME               STRING,
  _FILE_ROW_NUMBER         NUMBER,
  _LOAD_TS                 TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  _BATCH_ID                STRING
);

CREATE TABLE IF NOT EXISTS BRONZE.FRAUD_CASES (
  FRAUD_CASE_ID             STRING,
  CLAIM_ID                  STRING,
  POLICY_ID                 STRING,
  CUSTOMER_ID               STRING,
  FRAUD_SCORE               STRING,
  TRIGGER_REASON            STRING,
  INVESTIGATION_START_DATE  STRING,

  INVESTIGATION_END_DATE    STRING,
  INVESTIGATION_STATUS      STRING,
  INVESTIGATION_OUTCOME     STRING,
  INVESTIGATOR_ID           STRING,
  SOURCE_UPDATED_TS         STRING,
  DQ_STATUS                 STRING,
  DQ_REASON                 STRING,
  REJECT_ID                 STRING,
  _FILE_NAME                STRING,
  _FILE_ROW_NUMBER          NUMBER,
  _LOAD_TS                  TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  _BATCH_ID                 STRING
```

```
);
```

7.2 COPY statements (one per source)

METADATA$FILENAME and METADATA$FILE_ROW_NUMBER give perfect lineage. _LOAD_TS fills itself through the DEFAULT. COPY remembers which files it already loaded (64 days), so running it again never duplicates data.

COPY – masters and payments

```
COPY INTO BRONZE.CUSTOMERS (
    CUSTOMER_ID, CUSTOMER_NAME, DATE_OF_BIRTH, GENDER, EMAIL, PHONE, ADDRESS_LINE, CITY,
    STATE, REGION, OCCUPATION, CUSTOMER_RISK_CATEGORY, REGISTRATION_DATE,
    SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON, REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, $16, $17,
    METADATA$FILENAME, METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/customers/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';

COPY INTO BRONZE.INSURANCE_PRODUCTS (
    PRODUCT_ID, PRODUCT_NAME, INSURANCE_TYPE, COVERAGE_CATEGORY, MINIMUM_SUM_INSURED,
    MAXIMUM_SUM_INSURED, PREMIUM_FREQUENCY, GRACE_PERIOD_DAYS, ACTIVE_FLAG,
    SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON, REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/insurance_products/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';

COPY INTO BRONZE.POLICIES (
    POLICY_ID, CUSTOMER_ID, PRODUCT_ID, AGENT_ID, POLICY_ISSUE_DATE, POLICY_START_DATE,
    POLICY_EXPIRY_DATE, SUM_INSURED, PREMIUM_AMOUNT, PREMIUM_FREQUENCY, POLICY_STATUS,
    NOMINEE_NAME, UNDERWRITING_RISK_BAND, SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON,
    REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, $16, $17,
    METADATA$FILENAME, METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/policies/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';

COPY INTO BRONZE.PREMIUM_PAYMENTS (
    PAYMENT_ID, POLICY_ID, PREMIUM_SCHEDULE_ID, PAYMENT_DATE, PAYMENT_AMOUNT,
    PAYMENT_METHOD, PAYMENT_STATUS, REFERENCE_NUMBER, SOURCE_UPDATED_TS, DQ_STATUS,
    DQ_REASON, REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/premium_payments/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';
```

COPY – claims, settlements, fraud

```
COPY INTO BRONZE.CLAIMS (
    CLAIM_ID, POLICY_ID, CUSTOMER_ID, CLAIM_REGISTRATION_DATE, LOSS_DATE, CLAIM_TYPE,
    CLAIM_AMOUNT, APPROVED_AMOUNT, CLAIM_STATUS, ASSESSOR_ID, BRANCH_ID, SOURCE_UPDATED_TS,
    DQ_STATUS, DQ_REASON, REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/claims/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';

COPY INTO BRONZE.CLAIM_SETTLEMENTS (
    SETTLEMENT_ID, CLAIM_ID, SETTLEMENT_DATE, SETTLEMENT_TYPE, GROSS_SETTLEMENT_AMOUNT,
```

```
    DEDUCTIBLE_AMOUNT, RECOVERY_AMOUNT, NET_SETTLEMENT_AMOUNT, PAYMENT_STATUS,
    PAYMENT_REFERENCE, SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON, REJECT_ID, _FILE_NAME,
    _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/claim_settlements/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';

COPY INTO BRONZE.FRAUD_CASES (
    FRAUD_CASE_ID, CLAIM_ID, POLICY_ID, CUSTOMER_ID, FRAUD_SCORE, TRIGGER_REASON,
    INVESTIGATION_START_DATE, INVESTIGATION_END_DATE, INVESTIGATION_STATUS,
    INVESTIGATION_OUTCOME, INVESTIGATOR_ID, SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON,
    REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/fraud_cases/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';
```

7.3 Wrap the loads in one procedure with audit

sql/11_sp_load_bronze.sql

```
CREATE OR REPLACE PROCEDURE CONTROL.SP_LOAD_BRONZE()
RETURNS STRING
LANGUAGE SQL
AS
$$
DECLARE
  v_batch STRING        DEFAULT 'B' || TO_CHAR(CURRENT_TIMESTAMP(), 'YYYYMMDDHH24MISS');
  v_start TIMESTAMP_LTZ DEFAULT DATEADD(second, -5, CURRENT_TIMESTAMP());
  v_files NUMBER        DEFAULT 0;
BEGIN
  COPY INTO BRONZE.CUSTOMERS (
      CUSTOMER_ID, CUSTOMER_NAME, DATE_OF_BIRTH, GENDER, EMAIL, PHONE, ADDRESS_LINE, CITY,
      STATE, REGION, OCCUPATION, CUSTOMER_RISK_CATEGORY, REGISTRATION_DATE,
      SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON, REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
  FROM (
    SELECT
      $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, $16, $17,
      METADATA$FILENAME, METADATA$FILE_ROW_NUMBER
    FROM @CONTROL.STG_CLEAN/customers/
  )
  PATTERN = '.*[.]csv'
  ON_ERROR = 'CONTINUE';

  COPY INTO BRONZE.INSURANCE_PRODUCTS (
      PRODUCT_ID, PRODUCT_NAME, INSURANCE_TYPE, COVERAGE_CATEGORY, MINIMUM_SUM_INSURED,
      MAXIMUM_SUM_INSURED, PREMIUM_FREQUENCY, GRACE_PERIOD_DAYS, ACTIVE_FLAG,
      SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON, REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
  FROM (
    SELECT
      $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, METADATA$FILENAME,
      METADATA$FILE_ROW_NUMBER
    FROM @CONTROL.STG_CLEAN/insurance_products/
  )
  PATTERN = '.*[.]csv'
  ON_ERROR = 'CONTINUE';

  COPY INTO BRONZE.POLICIES (
      POLICY_ID, CUSTOMER_ID, PRODUCT_ID, AGENT_ID, POLICY_ISSUE_DATE, POLICY_START_DATE,
      POLICY_EXPIRY_DATE, SUM_INSURED, PREMIUM_AMOUNT, PREMIUM_FREQUENCY, POLICY_STATUS,
      NOMINEE_NAME, UNDERWRITING_RISK_BAND, SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON,
      REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
  FROM (
    SELECT
      $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, $16, $17,
      METADATA$FILENAME, METADATA$FILE_ROW_NUMBER
    FROM @CONTROL.STG_CLEAN/policies/
  )
  PATTERN = '.*[.]csv'
  ON_ERROR = 'CONTINUE';

  COPY INTO BRONZE.PREMIUM_PAYMENTS (
```

```
    PAYMENT_ID, POLICY_ID, PREMIUM_SCHEDULE_ID, PAYMENT_DATE, PAYMENT_AMOUNT,
    PAYMENT_METHOD, PAYMENT_STATUS, REFERENCE_NUMBER, SOURCE_UPDATED_TS, DQ_STATUS,
    DQ_REASON, REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/premium_payments/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';

COPY INTO BRONZE.CLAIMS (
    CLAIM_ID, POLICY_ID, CUSTOMER_ID, CLAIM_REGISTRATION_DATE, LOSS_DATE, CLAIM_TYPE,
    CLAIM_AMOUNT, APPROVED_AMOUNT, CLAIM_STATUS, ASSESSOR_ID, BRANCH_ID, SOURCE_UPDATED_TS,
    DQ_STATUS, DQ_REASON, REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, METADATA$FILENAME,

    METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/claims/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';

COPY INTO BRONZE.CLAIM_SETTLEMENTS (
    SETTLEMENT_ID, CLAIM_ID, SETTLEMENT_DATE, SETTLEMENT_TYPE, GROSS_SETTLEMENT_AMOUNT,
    DEDUCTIBLE_AMOUNT, RECOVERY_AMOUNT, NET_SETTLEMENT_AMOUNT, PAYMENT_STATUS,
    PAYMENT_REFERENCE, SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON, REJECT_ID, _FILE_NAME,
    _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/claim_settlements/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';

COPY INTO BRONZE.FRAUD_CASES (
    FRAUD_CASE_ID, CLAIM_ID, POLICY_ID, CUSTOMER_ID, FRAUD_SCORE, TRIGGER_REASON,
    INVESTIGATION_START_DATE, INVESTIGATION_END_DATE, INVESTIGATION_STATUS,
    INVESTIGATION_OUTCOME, INVESTIGATOR_ID, SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON,
    REJECT_ID, _FILE_NAME, _FILE_ROW_NUMBER)
FROM (
  SELECT
    $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER
  FROM @CONTROL.STG_CLEAN/fraud_cases/
)
PATTERN = '.*[.]csv'
ON_ERROR = 'CONTINUE';

-- stamp the batch id on the rows that were just loaded
UPDATE BRONZE.CUSTOMERS SET _BATCH_ID = :v_batch WHERE _BATCH_ID IS NULL;
UPDATE BRONZE.INSURANCE_PRODUCTS SET _BATCH_ID = :v_batch WHERE _BATCH_ID IS NULL;
UPDATE BRONZE.POLICIES SET _BATCH_ID = :v_batch WHERE _BATCH_ID IS NULL;
UPDATE BRONZE.PREMIUM_PAYMENTS SET _BATCH_ID = :v_batch WHERE _BATCH_ID IS NULL;
UPDATE BRONZE.CLAIMS SET _BATCH_ID = :v_batch WHERE _BATCH_ID IS NULL;
UPDATE BRONZE.CLAIM_SETTLEMENTS SET _BATCH_ID = :v_batch WHERE _BATCH_ID IS NULL;
UPDATE BRONZE.FRAUD_CASES SET _BATCH_ID = :v_batch WHERE _BATCH_ID IS NULL;

-- audit: one row per file loaded in this run
INSERT INTO CONTROL.LOAD_AUDIT (TABLE_NAME, FILE_NAME, STATUS, ROW_PARSED, ROW_LOADED,
       ERROR_COUNT, FIRST_ERROR_MESSAGE, LAST_LOAD_TIME, BATCH_ID)
SELECT TABLE_SCHEMA_NAME || '.' || TABLE_NAME, FILE_NAME, STATUS, ROW_PARSED, ROW_COUNT,
       ERROR_COUNT, FIRST_ERROR_MESSAGE, LAST_LOAD_TIME, :v_batch
FROM TABLE(INSURANCE_DB.INFORMATION_SCHEMA.COPY_HISTORY(
       TABLE_NAME => 'INSURANCE_DB.BRONZE.CUSTOMERS', START_TIME => :v_start))
UNION ALL
SELECT TABLE_SCHEMA_NAME || '.' || TABLE_NAME, FILE_NAME, STATUS, ROW_PARSED, ROW_COUNT,
       ERROR_COUNT, FIRST_ERROR_MESSAGE, LAST_LOAD_TIME, :v_batch
FROM TABLE(INSURANCE_DB.INFORMATION_SCHEMA.COPY_HISTORY(
       TABLE_NAME => 'INSURANCE_DB.BRONZE.INSURANCE_PRODUCTS', START_TIME => :v_start))
UNION ALL
SELECT TABLE_SCHEMA_NAME || '.' || TABLE_NAME, FILE_NAME, STATUS, ROW_PARSED, ROW_COUNT,
       ERROR_COUNT, FIRST_ERROR_MESSAGE, LAST_LOAD_TIME, :v_batch
FROM TABLE(INSURANCE_DB.INFORMATION_SCHEMA.COPY_HISTORY(
       TABLE_NAME => 'INSURANCE_DB.BRONZE.POLICIES', START_TIME => :v_start))
UNION ALL
SELECT TABLE_SCHEMA_NAME || '.' || TABLE_NAME, FILE_NAME, STATUS, ROW_PARSED, ROW_COUNT,
       ERROR_COUNT, FIRST_ERROR_MESSAGE, LAST_LOAD_TIME, :v_batch
```

```
  FROM TABLE(INSURANCE_DB.INFORMATION_SCHEMA.COPY_HISTORY(
         TABLE_NAME => 'INSURANCE_DB.BRONZE.PREMIUM_PAYMENTS', START_TIME => :v_start))
  UNION ALL
  SELECT TABLE_SCHEMA_NAME || '.' || TABLE_NAME, FILE_NAME, STATUS, ROW_PARSED, ROW_COUNT,
         ERROR_COUNT, FIRST_ERROR_MESSAGE, LAST_LOAD_TIME, :v_batch
  FROM TABLE(INSURANCE_DB.INFORMATION_SCHEMA.COPY_HISTORY(
         TABLE_NAME => 'INSURANCE_DB.BRONZE.CLAIMS', START_TIME => :v_start))

  UNION ALL
  SELECT TABLE_SCHEMA_NAME || '.' || TABLE_NAME, FILE_NAME, STATUS, ROW_PARSED, ROW_COUNT,
         ERROR_COUNT, FIRST_ERROR_MESSAGE, LAST_LOAD_TIME, :v_batch
  FROM TABLE(INSURANCE_DB.INFORMATION_SCHEMA.COPY_HISTORY(
         TABLE_NAME => 'INSURANCE_DB.BRONZE.CLAIM_SETTLEMENTS', START_TIME => :v_start))
  UNION ALL
  SELECT TABLE_SCHEMA_NAME || '.' || TABLE_NAME, FILE_NAME, STATUS, ROW_PARSED, ROW_COUNT,
         ERROR_COUNT, FIRST_ERROR_MESSAGE, LAST_LOAD_TIME, :v_batch
  FROM TABLE(INSURANCE_DB.INFORMATION_SCHEMA.COPY_HISTORY(
         TABLE_NAME => 'INSURANCE_DB.BRONZE.FRAUD_CASES', START_TIME => :v_start));

  SELECT COUNT(*) INTO :v_files FROM CONTROL.LOAD_AUDIT WHERE BATCH_ID = :v_batch;
  IF (v_files = 0) THEN
    RETURN 'NOTHING_NEW';
  END IF;
  RETURN v_files || ' file(s) loaded, batch ' || v_batch;
END;
$$;

CALL CONTROL.SP_LOAD_BRONZE();
SELECT * FROM CONTROL.LOAD_AUDIT ORDER BY LOGGED_TS DESC;
```

7.4 Check Bronze

```
-- rows per table and per file
SELECT 'CUSTOMERS' T, _FILE_NAME, COUNT(*) FROM BRONZE.CUSTOMERS GROUP BY 1,2
UNION ALL SELECT 'CLAIMS', _FILE_NAME, COUNT(*) FROM BRONZE.CLAIMS GROUP BY 1,2;   -- add the other 5

-- warnings that were kept
SELECT DQ_STATUS, COUNT(*) FROM BRONZE.CLAIMS GROUP BY 1;

-- source-to-Bronze reconciliation (per source):
--   control file rows  =  Bronze rows  +  duplicates  +  rejects
SELECT c.SOURCE_NAME, c.EXPECTED_ROW_COUNT,
       (SELECT COUNT(*) FROM CONTROL.DUPLICATE_LOG d WHERE d.SOURCE_NAME = c.SOURCE_NAME) AS DUPLICATES,
       (SELECT COUNT(*) FROM CONTROL.REJECT_LOG r
         WHERE r.SOURCE_NAME = c.SOURCE_NAME AND r.ATTEMPT_TYPE = 'INITIAL') AS REJECTS
FROM CONTROL.CONTROL_FILE_CHECK c WHERE c.STATUS = 'MATCH';
```

Expected totals

If the 3 duplicates are extra copies inside the 6,080 rows, Bronze holds 6,080 − 3 duplicates − 3 rejects = **6,074** rows across the seven tables after wave 1 + wave 2.

✔ **Done when:**

- ■ 7 Bronze tables, columns in documented order + dq_status, dq_reason, reject_id + 4 metadata columns.
- ■ LOAD_AUDIT shows one LOADED row per clean file with ERROR_COUNT = 0.
- ■ Re-running CALL CONTROL.SP_LOAD_BRONZE() returns NOTHING_NEW (no duplicates).

Phase 8 – Silver Layer (typed, deduplicated, validated)

In simple words: Silver turns text into real dates and numbers, keeps only the newest version of each record, and checks that every child record has a valid parent. Each Silver table is defined once as a **build view** (BLD_...). The table is then filled from that view. This keeps the logic in one place.

8.1 Build views (all seven)

sql/20_silver_views.sql (part 1)

```
USE SCHEMA INSURANCE_DB.SILVER;

CREATE OR REPLACE VIEW SILVER.BLD_CUSTOMERS AS
SELECT
    NULLIF(TRIM(CUSTOMER_ID), '') AS CUSTOMER_ID,
    NULLIF(TRIM(CUSTOMER_NAME), '') AS CUSTOMER_NAME,
    TRY_TO_DATE(DATE_OF_BIRTH, 'YYYY-MM-DD') AS DATE_OF_BIRTH,
    NULLIF(TRIM(GENDER), '') AS GENDER,
    NULLIF(TRIM(EMAIL), '') AS EMAIL,
    NULLIF(TRIM(PHONE), '') AS PHONE,
    NULLIF(TRIM(ADDRESS_LINE), '') AS ADDRESS_LINE,
    NULLIF(TRIM(CITY), '') AS CITY,
    NULLIF(TRIM(STATE), '') AS STATE,
    UPPER(NULLIF(TRIM(REGION), '')) AS REGION,
    NULLIF(TRIM(OCCUPATION), '') AS OCCUPATION,
    NULLIF(TRIM(CUSTOMER_RISK_CATEGORY), '') AS CUSTOMER_RISK_CATEGORY,
    TRY_TO_DATE(REGISTRATION_DATE, 'YYYY-MM-DD') AS REGISTRATION_DATE,
    TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') AS SOURCE_UPDATED_TS,
    DQ_STATUS, DQ_REASON,
    _FILE_NAME AS SRC_FILE_NAME, _LOAD_TS AS BRONZE_LOAD_TS
FROM BRONZE.CUSTOMERS
WHERE NULLIF(TRIM(CUSTOMER_ID), '') IS NOT NULL
QUALIFY ROW_NUMBER() OVER (PARTITION BY CUSTOMER_ID
        ORDER BY TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') DESC NULLS LAST,
                 _LOAD_TS DESC) = 1;

CREATE OR REPLACE VIEW SILVER.BLD_INSURANCE_PRODUCTS AS
SELECT
    NULLIF(TRIM(PRODUCT_ID), '') AS PRODUCT_ID,
    NULLIF(TRIM(PRODUCT_NAME), '') AS PRODUCT_NAME,
    NULLIF(TRIM(INSURANCE_TYPE), '') AS INSURANCE_TYPE,
    NULLIF(TRIM(COVERAGE_CATEGORY), '') AS COVERAGE_CATEGORY,
    TRY_TO_NUMBER(MINIMUM_SUM_INSURED, 18, 2) AS MINIMUM_SUM_INSURED,
    TRY_TO_NUMBER(MAXIMUM_SUM_INSURED, 18, 2) AS MAXIMUM_SUM_INSURED,
    TRY_TO_NUMBER(PREMIUM_FREQUENCY, 18, 2) AS PREMIUM_FREQUENCY,
    TRY_TO_NUMBER(GRACE_PERIOD_DAYS) AS GRACE_PERIOD_DAYS,
    TRY_TO_BOOLEAN(ACTIVE_FLAG) AS ACTIVE_FLAG,
    TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') AS SOURCE_UPDATED_TS,
    DQ_STATUS, DQ_REASON,
    _FILE_NAME AS SRC_FILE_NAME, _LOAD_TS AS BRONZE_LOAD_TS
FROM BRONZE.INSURANCE_PRODUCTS
WHERE NULLIF(TRIM(PRODUCT_ID), '') IS NOT NULL
QUALIFY ROW_NUMBER() OVER (PARTITION BY PRODUCT_ID
        ORDER BY TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') DESC NULLS LAST,
                 _LOAD_TS DESC) = 1;

CREATE OR REPLACE VIEW SILVER.BLD_POLICIES AS
SELECT
    NULLIF(TRIM(POLICY_ID), '') AS POLICY_ID,
    NULLIF(TRIM(CUSTOMER_ID), '') AS CUSTOMER_ID,
    NULLIF(TRIM(PRODUCT_ID), '') AS PRODUCT_ID,
    NULLIF(TRIM(AGENT_ID), '') AS AGENT_ID,
    TRY_TO_DATE(POLICY_ISSUE_DATE, 'YYYY-MM-DD') AS POLICY_ISSUE_DATE,
    TRY_TO_DATE(POLICY_START_DATE, 'YYYY-MM-DD') AS POLICY_START_DATE,
    TRY_TO_DATE(POLICY_EXPIRY_DATE, 'YYYY-MM-DD') AS POLICY_EXPIRY_DATE,
    TRY_TO_NUMBER(SUM_INSURED, 18, 2) AS SUM_INSURED,
    TRY_TO_NUMBER(PREMIUM_AMOUNT, 18, 2) AS PREMIUM_AMOUNT,
    TRY_TO_NUMBER(PREMIUM_FREQUENCY, 18, 2) AS PREMIUM_FREQUENCY,
    NULLIF(TRIM(POLICY_STATUS), '') AS POLICY_STATUS,
    NULLIF(TRIM(NOMINEE_NAME), '') AS NOMINEE_NAME,
    NULLIF(TRIM(UNDERWRITING_RISK_BAND), '') AS UNDERWRITING_RISK_BAND,
    TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') AS SOURCE_UPDATED_TS,
    DQ_STATUS, DQ_REASON,
    _FILE_NAME AS SRC_FILE_NAME, _LOAD_TS AS BRONZE_LOAD_TS
FROM BRONZE.POLICIES
WHERE NULLIF(TRIM(POLICY_ID), '') IS NOT NULL
QUALIFY ROW_NUMBER() OVER (PARTITION BY POLICY_ID
```

```
        ORDER BY TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') DESC NULLS LAST,
                 _LOAD_TS DESC) = 1;

CREATE OR REPLACE VIEW SILVER.BLD_PREMIUM_PAYMENTS AS
SELECT
    NULLIF(TRIM(PAYMENT_ID), '') AS PAYMENT_ID,
    NULLIF(TRIM(POLICY_ID), '') AS POLICY_ID,
    TRY_TO_NUMBER(PREMIUM_SCHEDULE_ID, 18, 2) AS PREMIUM_SCHEDULE_ID,
    TRY_TO_DATE(PAYMENT_DATE, 'YYYY-MM-DD') AS PAYMENT_DATE,
    TRY_TO_NUMBER(PAYMENT_AMOUNT, 18, 2) AS PAYMENT_AMOUNT,
    NULLIF(TRIM(PAYMENT_METHOD), '') AS PAYMENT_METHOD,
    NULLIF(TRIM(PAYMENT_STATUS), '') AS PAYMENT_STATUS,
    NULLIF(TRIM(REFERENCE_NUMBER), '') AS REFERENCE_NUMBER,
    TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') AS SOURCE_UPDATED_TS,
    DQ_STATUS, DQ_REASON,
    _FILE_NAME AS SRC_FILE_NAME, _LOAD_TS AS BRONZE_LOAD_TS
FROM BRONZE.PREMIUM_PAYMENTS
WHERE NULLIF(TRIM(PAYMENT_ID), '') IS NOT NULL
QUALIFY ROW_NUMBER() OVER (PARTITION BY PAYMENT_ID
        ORDER BY TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') DESC NULLS LAST,
                 _LOAD_TS DESC) = 1;
```

sql/20_silver_views.sql (part 2)

```
CREATE OR REPLACE VIEW SILVER.BLD_CLAIMS AS
SELECT
    NULLIF(TRIM(CLAIM_ID), '') AS CLAIM_ID,
    NULLIF(TRIM(POLICY_ID), '') AS POLICY_ID,
    NULLIF(TRIM(CUSTOMER_ID), '') AS CUSTOMER_ID,
    TRY_TO_DATE(CLAIM_REGISTRATION_DATE, 'YYYY-MM-DD') AS CLAIM_REGISTRATION_DATE,
    TRY_TO_DATE(LOSS_DATE, 'YYYY-MM-DD') AS LOSS_DATE,
    NULLIF(TRIM(CLAIM_TYPE), '') AS CLAIM_TYPE,
    TRY_TO_NUMBER(CLAIM_AMOUNT, 18, 2) AS CLAIM_AMOUNT,
    TRY_TO_NUMBER(APPROVED_AMOUNT, 18, 2) AS APPROVED_AMOUNT,
    NULLIF(TRIM(CLAIM_STATUS), '') AS CLAIM_STATUS,
    NULLIF(TRIM(ASSESSOR_ID), '') AS ASSESSOR_ID,
    NULLIF(TRIM(BRANCH_ID), '') AS BRANCH_ID,
    TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') AS SOURCE_UPDATED_TS,
    DQ_STATUS, DQ_REASON,
    _FILE_NAME AS SRC_FILE_NAME, _LOAD_TS AS BRONZE_LOAD_TS
FROM BRONZE.CLAIMS
WHERE NULLIF(TRIM(CLAIM_ID), '') IS NOT NULL
QUALIFY ROW_NUMBER() OVER (PARTITION BY CLAIM_ID
        ORDER BY TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') DESC NULLS LAST,
                 _LOAD_TS DESC) = 1;

CREATE OR REPLACE VIEW SILVER.BLD_CLAIM_SETTLEMENTS AS
SELECT
    NULLIF(TRIM(SETTLEMENT_ID), '') AS SETTLEMENT_ID,
    NULLIF(TRIM(CLAIM_ID), '') AS CLAIM_ID,
    TRY_TO_DATE(SETTLEMENT_DATE, 'YYYY-MM-DD') AS SETTLEMENT_DATE,
    NULLIF(TRIM(SETTLEMENT_TYPE), '') AS SETTLEMENT_TYPE,
    TRY_TO_NUMBER(GROSS_SETTLEMENT_AMOUNT, 18, 2) AS GROSS_SETTLEMENT_AMOUNT,
    TRY_TO_NUMBER(DEDUCTIBLE_AMOUNT, 18, 2) AS DEDUCTIBLE_AMOUNT,
    TRY_TO_NUMBER(RECOVERY_AMOUNT, 18, 2) AS RECOVERY_AMOUNT,
    TRY_TO_NUMBER(NET_SETTLEMENT_AMOUNT, 18, 2) AS NET_SETTLEMENT_AMOUNT,
    NULLIF(TRIM(PAYMENT_STATUS), '') AS PAYMENT_STATUS,
    NULLIF(TRIM(PAYMENT_REFERENCE), '') AS PAYMENT_REFERENCE,
    TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') AS SOURCE_UPDATED_TS,
    DQ_STATUS, DQ_REASON,
    _FILE_NAME AS SRC_FILE_NAME, _LOAD_TS AS BRONZE_LOAD_TS
FROM BRONZE.CLAIM_SETTLEMENTS
WHERE NULLIF(TRIM(SETTLEMENT_ID), '') IS NOT NULL
QUALIFY ROW_NUMBER() OVER (PARTITION BY SETTLEMENT_ID
        ORDER BY TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') DESC NULLS LAST,
                 _LOAD_TS DESC) = 1;

CREATE OR REPLACE VIEW SILVER.BLD_FRAUD_CASES AS
SELECT
    NULLIF(TRIM(FRAUD_CASE_ID), '') AS FRAUD_CASE_ID,
    NULLIF(TRIM(CLAIM_ID), '') AS CLAIM_ID,
    NULLIF(TRIM(POLICY_ID), '') AS POLICY_ID,
    NULLIF(TRIM(CUSTOMER_ID), '') AS CUSTOMER_ID,
    TRY_TO_NUMBER(FRAUD_SCORE, 18, 2) AS FRAUD_SCORE,
    NULLIF(TRIM(TRIGGER_REASON), '') AS TRIGGER_REASON,
    TRY_TO_DATE(INVESTIGATION_START_DATE, 'YYYY-MM-DD') AS INVESTIGATION_START_DATE,
    TRY_TO_DATE(INVESTIGATION_END_DATE, 'YYYY-MM-DD') AS INVESTIGATION_END_DATE,
    NULLIF(TRIM(INVESTIGATION_STATUS), '') AS INVESTIGATION_STATUS,
    NULLIF(TRIM(INVESTIGATION_OUTCOME), '') AS INVESTIGATION_OUTCOME,
    NULLIF(TRIM(INVESTIGATOR_ID), '') AS INVESTIGATOR_ID,
    TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') AS SOURCE_UPDATED_TS,
    DQ_STATUS, DQ_REASON,
    _FILE_NAME AS SRC_FILE_NAME, _LOAD_TS AS BRONZE_LOAD_TS
```

```
FROM BRONZE.FRAUD_CASES
WHERE NULLIF(TRIM(FRAUD_CASE_ID), '') IS NOT NULL
QUALIFY ROW_NUMBER() OVER (PARTITION BY FRAUD_CASE_ID
        ORDER BY TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') DESC NULLS LAST,
                 _LOAD_TS DESC) = 1;
```

Check premium_frequency

The document types premium_frequency as NUMBER. If profiling shows text such as “Monthly”, change its cast to NULLIF(TRIM(PREMIUM_FREQUENCY), '') in both views – otherwise TRY_TO_NUMBER turns it into NULL.

8.2 Create the Silver tables and the refresh procedure

sql/21_silver_tables.sql

```
-- one-time creation (structure + first data)
CREATE TABLE IF NOT EXISTS SILVER.CUSTOMERS          AS SELECT * FROM SILVER.BLD_CUSTOMERS;
CREATE TABLE IF NOT EXISTS SILVER.INSURANCE_PRODUCTS AS SELECT * FROM SILVER.BLD_INSURANCE_PRODUCTS;
CREATE TABLE IF NOT EXISTS SILVER.POLICIES           AS SELECT * FROM SILVER.BLD_POLICIES;
CREATE TABLE IF NOT EXISTS SILVER.PREMIUM_PAYMENTS   AS SELECT * FROM SILVER.BLD_PREMIUM_PAYMENTS;
CREATE TABLE IF NOT EXISTS SILVER.CLAIM_SETTLEMENTS  AS SELECT * FROM SILVER.BLD_CLAIM_SETTLEMENTS;
CREATE TABLE IF NOT EXISTS SILVER.FRAUD_CASES        AS SELECT * FROM SILVER.BLD_FRAUD_CASES;
-- claims is filled incrementally by the Stream + MERGE (Phase 10): structure only
CREATE TABLE IF NOT EXISTS SILVER.CLAIMS AS
  SELECT *, CURRENT_TIMESTAMP()::TIMESTAMP_NTZ AS SILVER_UPDATED_TS FROM SILVER.BLD_CLAIMS WHERE 1 = 0;

-- refresh procedure for the six full-refresh tables
CREATE OR REPLACE PROCEDURE SILVER.SP_BUILD_SILVER()
RETURNS STRING
LANGUAGE SQL
AS
$$
BEGIN
  BEGIN TRANSACTION;
  INSERT OVERWRITE INTO SILVER.CUSTOMERS          SELECT * FROM SILVER.BLD_CUSTOMERS;
  INSERT OVERWRITE INTO SILVER.INSURANCE_PRODUCTS SELECT * FROM SILVER.BLD_INSURANCE_PRODUCTS;
  INSERT OVERWRITE INTO SILVER.POLICIES           SELECT * FROM SILVER.BLD_POLICIES;
  INSERT OVERWRITE INTO SILVER.PREMIUM_PAYMENTS   SELECT * FROM SILVER.BLD_PREMIUM_PAYMENTS;
  INSERT OVERWRITE INTO SILVER.CLAIM_SETTLEMENTS  SELECT * FROM SILVER.BLD_CLAIM_SETTLEMENTS;
  INSERT OVERWRITE INTO SILVER.FRAUD_CASES        SELECT * FROM SILVER.BLD_FRAUD_CASES;
  COMMIT;
  RETURN 'Silver masters refreshed';
END;
$$;
```

Why INSERT OVERWRITE and not CREATE OR REPLACE TABLE?

CREATE OR REPLACE drops the table – and with it every grant, masking policy and row access policy you attached. INSERT OVERWRITE replaces only the rows. Use this rule in Silver **and** Gold.

8.3 Relationship and sequence checks (safety net)

sql/22_silver_dq.sql

```
CREATE OR REPLACE PROCEDURE SILVER.SP_SILVER_DQ()
RETURNS STRING
LANGUAGE SQL
AS
$$
BEGIN
  DELETE FROM CONTROL.DQ_EXCEPTIONS WHERE LAYER = 'SILVER';      -- always reflect the current state
  INSERT INTO CONTROL.DQ_EXCEPTIONS (LAYER, TABLE_NAME, RECORD_KEY, RULE_NAME, SEVERITY, DETAILS)
  SELECT 'SILVER','POLICIES', p.POLICY_ID, 'ORPHAN_CUSTOMER', 'ERROR', 'customer ' || p.CUSTOMER_ID || ' not found'
    FROM SILVER.POLICIES p LEFT JOIN SILVER.CUSTOMERS c ON c.CUSTOMER_ID = p.CUSTOMER_ID
   WHERE c.CUSTOMER_ID IS NULL
  UNION ALL
  SELECT 'SILVER','POLICIES', p.POLICY_ID, 'ORPHAN_PRODUCT', 'ERROR', 'product ' || p.PRODUCT_ID || ' not found'
    FROM SILVER.POLICIES p LEFT JOIN SILVER.INSURANCE_PRODUCTS x ON x.PRODUCT_ID = p.PRODUCT_ID
   WHERE x.PRODUCT_ID IS NULL
  UNION ALL
  SELECT 'SILVER','POLICIES', POLICY_ID, 'EXPIRY_BEFORE_START', 'ERROR', 'expiry before start'
    FROM SILVER.POLICIES WHERE POLICY_EXPIRY_DATE < POLICY_START_DATE
  UNION ALL
  SELECT 'SILVER','PREMIUM_PAYMENTS', pp.PAYMENT_ID, 'ORPHAN_POLICY', 'ERROR', 'policy ' || pp.POLICY_ID
    FROM SILVER.PREMIUM_PAYMENTS pp LEFT JOIN SILVER.POLICIES p ON p.POLICY_ID = pp.POLICY_ID
   WHERE p.POLICY_ID IS NULL
  UNION ALL
```

```
  SELECT 'SILVER','CLAIMS', cl.CLAIM_ID, 'ORPHAN_POLICY', 'ERROR', 'policy ' || cl.POLICY_ID
    FROM SILVER.CLAIMS cl LEFT JOIN SILVER.POLICIES p ON p.POLICY_ID = cl.POLICY_ID
   WHERE p.POLICY_ID IS NULL
  UNION ALL
  SELECT 'SILVER','CLAIMS', cl.CLAIM_ID, 'LOSS_BEFORE_POLICY_START', 'ERROR',
         'loss ' || cl.LOSS_DATE || ' < start ' || p.POLICY_START_DATE
    FROM SILVER.CLAIMS cl JOIN SILVER.POLICIES p ON p.POLICY_ID = cl.POLICY_ID
   WHERE cl.LOSS_DATE < p.POLICY_START_DATE
  UNION ALL
  SELECT 'SILVER','CLAIMS', cl.CLAIM_ID, 'CUSTOMER_POLICY_MISMATCH', 'WARNING',
         'claim customer ' || cl.CUSTOMER_ID || ' <> policy customer ' || p.CUSTOMER_ID
    FROM SILVER.CLAIMS cl JOIN SILVER.POLICIES p ON p.POLICY_ID = cl.POLICY_ID
   WHERE cl.CUSTOMER_ID <> p.CUSTOMER_ID
  UNION ALL
  SELECT 'SILVER','CLAIM_SETTLEMENTS', s.SETTLEMENT_ID, 'ORPHAN_CLAIM', 'ERROR', 'claim ' || s.CLAIM_ID
    FROM SILVER.CLAIM_SETTLEMENTS s LEFT JOIN SILVER.CLAIMS cl ON cl.CLAIM_ID = s.CLAIM_ID
   WHERE cl.CLAIM_ID IS NULL
  UNION ALL
  SELECT 'SILVER','CLAIM_SETTLEMENTS', SETTLEMENT_ID, 'NET_AMOUNT_MISMATCH', 'ERROR', 'net <> gross - ded - rec'
    FROM SILVER.CLAIM_SETTLEMENTS
   WHERE ABS(NET_SETTLEMENT_AMOUNT - (GROSS_SETTLEMENT_AMOUNT - COALESCE(DEDUCTIBLE_AMOUNT,0)
             - COALESCE(RECOVERY_AMOUNT,0))) > 1
  UNION ALL
  SELECT 'SILVER','FRAUD_CASES', f.FRAUD_CASE_ID, 'ORPHAN_CLAIM', 'ERROR', 'claim ' || f.CLAIM_ID
    FROM SILVER.FRAUD_CASES f LEFT JOIN SILVER.CLAIMS cl ON cl.CLAIM_ID = f.CLAIM_ID
   WHERE cl.CLAIM_ID IS NULL;
  RETURN 'Silver DQ checked';
END;
$$;

CALL SILVER.SP_SILVER_DQ();
SELECT TABLE_NAME, RULE_NAME, SEVERITY, COUNT(*) FROM CONTROL.DQ_EXCEPTIONS GROUP BY 1,2,3;
```

Rows with an **ERROR** exception are kept in Silver (for investigation) but are **excluded from Gold**. WARNING exceptions flow to Gold and are visible on the dashboard.

✔ **Done when:**

- Silver row counts = distinct keys in Bronze (SELECT COUNT(DISTINCT CLAIM_ID) FROM BRONZE.CLAIMS).
- No NULL in any date column that had a value in Bronze (proves the 4 formats were standardised).
- DQ_EXCEPTIONS has 0 ERROR rows after a clean run (IDMC already rejected the 3 anomalies).

Phase 9 – Gold Layer (dimensions, facts, marts, KPIs)

In simple words: Gold reshapes Silver into the star schema from the document. Every object is again a build view + table pair. Region is carried from the customer into every fact and mart – this is what makes region-based security possible later.

9.1 DIM_DATE (one-time)

sql/30_dim_date.sql

```
USE SCHEMA INSURANCE_DB.GOLD;
CREATE TABLE IF NOT EXISTS GOLD.DIM_DATE AS
SELECT TO_NUMBER(TO_CHAR(D, 'YYYYMMDD')) AS DATE_KEY,
       D AS FULL_DATE, YEAR(D) AS YEAR_NO, QUARTER(D) AS QUARTER_NO, MONTH(D) AS MONTH_NO,
       MONTHNAME(D) AS MONTH_NAME, DAY(D) AS DAY_NO, DAYNAME(D) AS DAY_NAME,
       IFF(DAYOFWEEKISO(D) IN (6, 7), TRUE, FALSE) AS IS_WEEKEND,
       DATE_TRUNC('month', D) AS MONTH_START
FROM (SELECT DATEADD(day, ROW_NUMBER() OVER (ORDER BY SEQ4()) - 1, '2010-01-01'::DATE) AS D
      FROM TABLE(GENERATOR(ROWCOUNT => 7670)));          -- 2010-01-01 .. 2030-12-31
```

9.2 Dimensions

sql/31_gold_dimensions.sql

```
CREATE OR REPLACE VIEW GOLD.BLD_DIM_CUSTOMER AS
SELECT CUSTOMER_ID, CUSTOMER_NAME, DATE_OF_BIRTH,
       DATEDIFF(year, DATE_OF_BIRTH, CURRENT_DATE()) AS AGE_YEARS,
       GENDER, EMAIL, PHONE, ADDRESS_LINE, CITY, STATE,
       COALESCE(REGION, 'UNKNOWN') AS REGION,
       OCCUPATION, CUSTOMER_RISK_CATEGORY, REGISTRATION_DATE, DQ_STATUS
FROM SILVER.CUSTOMERS;
CREATE TABLE IF NOT EXISTS GOLD.DIM_CUSTOMER AS SELECT * FROM GOLD.BLD_DIM_CUSTOMER;

CREATE OR REPLACE VIEW GOLD.BLD_DIM_INSURANCE_PRODUCT AS
SELECT PRODUCT_ID, PRODUCT_NAME, INSURANCE_TYPE, COVERAGE_CATEGORY, MINIMUM_SUM_INSURED,
       MAXIMUM_SUM_INSURED, PREMIUM_FREQUENCY, GRACE_PERIOD_DAYS, ACTIVE_FLAG
FROM SILVER.INSURANCE_PRODUCTS;
CREATE TABLE IF NOT EXISTS GOLD.DIM_INSURANCE_PRODUCT AS SELECT * FROM GOLD.BLD_DIM_INSURANCE_PRODUCT;

CREATE OR REPLACE VIEW GOLD.BLD_DIM_POLICY AS
SELECT p.POLICY_ID, p.CUSTOMER_ID, p.PRODUCT_ID, pr.INSURANCE_TYPE, c.REGION, p.AGENT_ID,
       p.POLICY_ISSUE_DATE, p.POLICY_START_DATE, p.POLICY_EXPIRY_DATE,
       p.SUM_INSURED, p.PREMIUM_AMOUNT, p.PREMIUM_FREQUENCY, p.POLICY_STATUS, p.UNDERWRITING_RISK_BAND,
       IFF(UPPER(p.POLICY_STATUS) IN ('ACTIVE', 'IN FORCE', 'INFORCE'), TRUE, FALSE) AS IS_ACTIVE,
       p.DQ_STATUS
FROM SILVER.POLICIES p
JOIN GOLD.DIM_CUSTOMER c           ON c.CUSTOMER_ID = p.CUSTOMER_ID
JOIN GOLD.DIM_INSURANCE_PRODUCT pr ON pr.PRODUCT_ID = p.PRODUCT_ID
WHERE NOT EXISTS (SELECT 1 FROM CONTROL.DQ_EXCEPTIONS e WHERE e.TABLE_NAME = 'POLICIES'
                  AND e.RECORD_KEY = p.POLICY_ID AND e.SEVERITY = 'ERROR');
CREATE TABLE IF NOT EXISTS GOLD.DIM_POLICY AS SELECT * FROM GOLD.BLD_DIM_POLICY;
```

Confirm status values

IS_ACTIVE, IS_SUCCESSFUL, IS_APPROVED, IS_OPEN and IS_CONFIRMED use typical status words. Run SELECT DISTINCT POLICY_STATUS / PAYMENT_STATUS / CLAIM_STATUS / INVESTIGATION_OUTCOME in Silver and put your real values into the IN (...) lists. nominee_name is intentionally not carried to Gold (PII not needed).

9.3 Facts

sql/32_gold_facts.sql

```
CREATE OR REPLACE VIEW GOLD.BLD_FACT_PREMIUM_PAYMENT AS
SELECT pp.PAYMENT_ID, pp.POLICY_ID, d.CUSTOMER_ID, d.PRODUCT_ID, d.INSURANCE_TYPE, d.REGION,
       pp.PREMIUM_SCHEDULE_ID, pp.PAYMENT_DATE,
       TO_NUMBER(TO_CHAR(pp.PAYMENT_DATE, 'YYYYMMDD')) AS PAYMENT_DATE_KEY,
       pp.PAYMENT_AMOUNT, pp.PAYMENT_METHOD, pp.PAYMENT_STATUS,
       IFF(UPPER(pp.PAYMENT_STATUS) IN ('PAID','SUCCESS','SUCCESSFUL','COMPLETED'), TRUE, FALSE) AS IS_SUCCESSFUL
FROM SILVER.PREMIUM_PAYMENTS pp
JOIN GOLD.DIM_POLICY d ON d.POLICY_ID = pp.POLICY_ID
WHERE NOT EXISTS (SELECT 1 FROM CONTROL.DQ_EXCEPTIONS e WHERE e.TABLE_NAME = 'PREMIUM_PAYMENTS'
                  AND e.RECORD_KEY = pp.PAYMENT_ID AND e.SEVERITY = 'ERROR');
```

```
CREATE TABLE IF NOT EXISTS GOLD.FACT_PREMIUM_PAYMENT AS SELECT * FROM GOLD.BLD_FACT_PREMIUM_PAYMENT;

CREATE OR REPLACE VIEW GOLD.BLD_FACT_CLAIM AS
SELECT cl.CLAIM_ID, cl.POLICY_ID, cl.CUSTOMER_ID, d.PRODUCT_ID, d.INSURANCE_TYPE, d.REGION,
       cl.CLAIM_REGISTRATION_DATE,
       TO_NUMBER(TO_CHAR(cl.CLAIM_REGISTRATION_DATE, 'YYYYMMDD')) AS REG_DATE_KEY,
       cl.LOSS_DATE, TO_NUMBER(TO_CHAR(cl.LOSS_DATE, 'YYYYMMDD')) AS LOSS_DATE_KEY,
       cl.CLAIM_TYPE, cl.CLAIM_AMOUNT, cl.APPROVED_AMOUNT, cl.CLAIM_STATUS,
       IFF(COALESCE(cl.APPROVED_AMOUNT, 0) > 0
           OR UPPER(cl.CLAIM_STATUS) IN ('APPROVED','SETTLED','PARTIALLY APPROVED'), TRUE, FALSE) AS IS_APPROVED,
       IFF(UPPER(cl.CLAIM_STATUS) IN ('OPEN','REGISTERED','PENDING','UNDER REVIEW','IN PROGRESS'),
           TRUE, FALSE) AS IS_OPEN,
       DATEDIFF(day, d.POLICY_START_DATE, cl.LOSS_DATE) AS DAYS_POLICY_TO_LOSS,
       cl.ASSESSOR_ID, cl.BRANCH_ID, cl.DQ_STATUS
FROM SILVER.CLAIMS cl
JOIN GOLD.DIM_POLICY d ON d.POLICY_ID = cl.POLICY_ID
WHERE NOT EXISTS (SELECT 1 FROM CONTROL.DQ_EXCEPTIONS e WHERE e.TABLE_NAME = 'CLAIMS'
                  AND e.RECORD_KEY = cl.CLAIM_ID AND e.SEVERITY = 'ERROR');
CREATE TABLE IF NOT EXISTS GOLD.FACT_CLAIM AS SELECT * FROM GOLD.BLD_FACT_CLAIM;

CREATE OR REPLACE VIEW GOLD.BLD_FACT_CLAIM_SETTLEMENT AS
SELECT s.SETTLEMENT_ID, s.CLAIM_ID, f.POLICY_ID, f.CUSTOMER_ID, f.INSURANCE_TYPE, f.REGION,
       s.SETTLEMENT_DATE, TO_NUMBER(TO_CHAR(s.SETTLEMENT_DATE, 'YYYYMMDD')) AS SETTLEMENT_DATE_KEY,
       s.SETTLEMENT_TYPE, s.GROSS_SETTLEMENT_AMOUNT, s.DEDUCTIBLE_AMOUNT, s.RECOVERY_AMOUNT,
       s.NET_SETTLEMENT_AMOUNT, s.PAYMENT_STATUS,
       f.APPROVED_AMOUNT AS CLAIM_APPROVED_AMOUNT,
       DATEDIFF(day, f.CLAIM_REGISTRATION_DATE, s.SETTLEMENT_DATE) AS TURNAROUND_DAYS
FROM SILVER.CLAIM_SETTLEMENTS s
JOIN GOLD.FACT_CLAIM f ON f.CLAIM_ID = s.CLAIM_ID
WHERE NOT EXISTS (SELECT 1 FROM CONTROL.DQ_EXCEPTIONS e WHERE e.TABLE_NAME = 'CLAIM_SETTLEMENTS'
                  AND e.RECORD_KEY = s.SETTLEMENT_ID AND e.SEVERITY = 'ERROR');
CREATE TABLE IF NOT EXISTS GOLD.FACT_CLAIM_SETTLEMENT AS SELECT * FROM GOLD.BLD_FACT_CLAIM_SETTLEMENT;

CREATE OR REPLACE VIEW GOLD.BLD_FACT_FRAUD_CASE AS
SELECT fc.FRAUD_CASE_ID, fc.CLAIM_ID, fc.POLICY_ID, fc.CUSTOMER_ID, f.REGION, f.INSURANCE_TYPE,
       fc.FRAUD_SCORE AS SOURCE_FRAUD_SCORE, fc.TRIGGER_REASON,
       fc.INVESTIGATION_START_DATE, fc.INVESTIGATION_END_DATE,
       fc.INVESTIGATION_STATUS, fc.INVESTIGATION_OUTCOME, fc.INVESTIGATOR_ID,
       IFF(fc.INVESTIGATION_END_DATE IS NULL
           AND UPPER(COALESCE(fc.INVESTIGATION_STATUS, '')) NOT IN ('CLOSED','COMPLETED'), TRUE, FALSE) AS IS_OPEN,
       IFF(UPPER(COALESCE(fc.INVESTIGATION_OUTCOME, '')) IN
           ('CONFIRMED','CONFIRMED FRAUD','FRAUD CONFIRMED','FRAUD'), TRUE, FALSE) AS IS_CONFIRMED,
       DATEDIFF(day, fc.INVESTIGATION_START_DATE,
                COALESCE(fc.INVESTIGATION_END_DATE, CURRENT_DATE())) AS INVESTIGATION_DAYS,
       fc.DQ_STATUS
FROM SILVER.FRAUD_CASES fc
JOIN GOLD.FACT_CLAIM f ON f.CLAIM_ID = fc.CLAIM_ID
WHERE NOT EXISTS (SELECT 1 FROM CONTROL.DQ_EXCEPTIONS e WHERE e.TABLE_NAME = 'FRAUD_CASES'
                  AND e.RECORD_KEY = fc.FRAUD_CASE_ID AND e.SEVERITY = 'ERROR');
CREATE TABLE IF NOT EXISTS GOLD.FACT_FRAUD_CASE AS SELECT * FROM GOLD.BLD_FACT_FRAUD_CASE;
```

9.4 Marts

sql/33_gold_marts.sql

```
CREATE OR REPLACE VIEW GOLD.BLD_MART_POLICY_PORTFOLIO AS
WITH pol AS (
  SELECT REGION, INSURANCE_TYPE, COUNT(*) AS POLICY_COUNT, COUNT_IF(IS_ACTIVE) AS ACTIVE_POLICY_COUNT,
         SUM(SUM_INSURED) AS TOTAL_SUM_INSURED, SUM(IFF(IS_ACTIVE, SUM_INSURED, 0)) AS ACTIVE_SUM_INSURED,
         SUM(PREMIUM_AMOUNT) AS WRITTEN_PREMIUM
  FROM GOLD.DIM_POLICY GROUP BY 1, 2),
pay AS (
  SELECT REGION, INSURANCE_TYPE, COUNT(*) AS PAYMENT_COUNT,
         SUM(IFF(IS_SUCCESSFUL, PAYMENT_AMOUNT, 0)) AS PREMIUM_COLLECTED
  FROM GOLD.FACT_PREMIUM_PAYMENT GROUP BY 1, 2)
SELECT pol.REGION, pol.INSURANCE_TYPE, pol.POLICY_COUNT, pol.ACTIVE_POLICY_COUNT,
       pol.TOTAL_SUM_INSURED, pol.ACTIVE_SUM_INSURED, pol.WRITTEN_PREMIUM,
       COALESCE(pay.PAYMENT_COUNT, 0) AS PAYMENT_COUNT,
       COALESCE(pay.PREMIUM_COLLECTED, 0) AS PREMIUM_COLLECTED
FROM pol
LEFT JOIN pay ON EQUAL_NULL(pay.REGION, pol.REGION) AND EQUAL_NULL(pay.INSURANCE_TYPE, pol.INSURANCE_TYPE);
CREATE TABLE IF NOT EXISTS GOLD.MART_POLICY_PORTFOLIO AS SELECT * FROM GOLD.BLD_MART_POLICY_PORTFOLIO;

CREATE OR REPLACE VIEW GOLD.BLD_MART_CLAIMS_PERFORMANCE AS
SELECT REGION, INSURANCE_TYPE, CLAIM_TYPE,
       DATE_TRUNC('month', CLAIM_REGISTRATION_DATE) AS CLAIM_MONTH,
       COUNT(*) AS CLAIM_COUNT, SUM(CLAIM_AMOUNT) AS TOTAL_CLAIM_AMOUNT,
       ROUND(AVG(CLAIM_AMOUNT), 2) AS AVG_CLAIM_AMOUNT,
       COUNT_IF(IS_APPROVED) AS APPROVED_CLAIM_COUNT,
       SUM(COALESCE(APPROVED_AMOUNT, 0)) AS TOTAL_APPROVED_AMOUNT,
       COUNT_IF(IS_OPEN) AS OPEN_CLAIM_COUNT,
```

```
       ROUND(100 * COUNT_IF(IS_APPROVED) / NULLIF(COUNT(*), 0), 2) AS APPROVAL_RATIO_PCT
FROM GOLD.FACT_CLAIM
GROUP BY 1, 2, 3, 4;
CREATE TABLE IF NOT EXISTS GOLD.MART_CLAIMS_PERFORMANCE AS SELECT * FROM GOLD.BLD_MART_CLAIMS_PERFORMANCE;

CREATE OR REPLACE VIEW GOLD.BLD_MART_SETTLEMENT_RECONCILIATION AS
WITH s AS (
  SELECT CLAIM_ID, COUNT(*) AS SETTLEMENT_COUNT,
         SUM(GROSS_SETTLEMENT_AMOUNT) AS TOTAL_GROSS, SUM(NET_SETTLEMENT_AMOUNT) AS TOTAL_NET,
         MAX(SETTLEMENT_DATE) AS LAST_SETTLEMENT_DATE, ROUND(AVG(TURNAROUND_DAYS), 1) AS AVG_TURNAROUND_DAYS
  FROM GOLD.FACT_CLAIM_SETTLEMENT GROUP BY 1)
SELECT c.CLAIM_ID, c.POLICY_ID, c.CUSTOMER_ID, c.REGION, c.INSURANCE_TYPE, c.CLAIM_STATUS,
       c.APPROVED_AMOUNT, COALESCE(s.SETTLEMENT_COUNT, 0) AS SETTLEMENT_COUNT,
       COALESCE(s.TOTAL_GROSS, 0) AS TOTAL_GROSS_SETTLED, COALESCE(s.TOTAL_NET, 0) AS TOTAL_NET_SETTLED,
       COALESCE(c.APPROVED_AMOUNT, 0) - COALESCE(s.TOTAL_GROSS, 0) AS DIFFERENCE_AMOUNT,
       s.LAST_SETTLEMENT_DATE, s.AVG_TURNAROUND_DAYS,
       CASE WHEN s.CLAIM_ID IS NULL AND COALESCE(c.APPROVED_AMOUNT, 0) > 0 THEN 'NOT_SETTLED'
            WHEN ABS(COALESCE(c.APPROVED_AMOUNT, 0) - s.TOTAL_GROSS) <= 1 THEN 'MATCHED'
            WHEN s.TOTAL_GROSS < COALESCE(c.APPROVED_AMOUNT, 0) THEN 'UNDER_SETTLED'
            ELSE 'OVER_SETTLED' END AS RECON_STATUS
FROM GOLD.FACT_CLAIM c
LEFT JOIN s ON s.CLAIM_ID = c.CLAIM_ID
WHERE c.IS_APPROVED OR s.CLAIM_ID IS NOT NULL;
CREATE TABLE IF NOT EXISTS GOLD.MART_SETTLEMENT_RECONCILIATION
  AS SELECT * FROM GOLD.BLD_MART_SETTLEMENT_RECONCILIATION;

CREATE OR REPLACE VIEW GOLD.BLD_MART_FRAUD_RISK AS
WITH hist AS (
  SELECT CLAIM_ID,
         COUNT(*) OVER (PARTITION BY CUSTOMER_ID ORDER BY CLAIM_REGISTRATION_DATE, CLAIM_ID
                        ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING) AS PRIOR_CLAIMS_COUNT
  FROM GOLD.FACT_CLAIM),
fc AS (
  SELECT CLAIM_ID, COUNT(*) AS FRAUD_CASE_COUNT, COUNT_IF(IS_OPEN) AS OPEN_CASE_COUNT,
         COUNT_IF(IS_CONFIRMED) AS CONFIRMED_CASE_COUNT, MAX(SOURCE_FRAUD_SCORE) AS MAX_SOURCE_FRAUD_SCORE
  FROM GOLD.FACT_FRAUD_CASE GROUP BY 1)
SELECT c.CLAIM_ID, c.CUSTOMER_ID, c.POLICY_ID, c.REGION, c.INSURANCE_TYPE, c.CLAIM_TYPE, c.CLAIM_STATUS,
       c.CLAIM_REGISTRATION_DATE, c.CLAIM_AMOUNT, c.APPROVED_AMOUNT, c.DAYS_POLICY_TO_LOSS,
       COALESCE(h.PRIOR_CLAIMS_COUNT, 0) AS PRIOR_CLAIMS_COUNT,
       COALESCE(fc.FRAUD_CASE_COUNT, 0) AS FRAUD_CASE_COUNT,
       COALESCE(fc.OPEN_CASE_COUNT, 0) AS OPEN_CASE_COUNT,
       COALESCE(fc.CONFIRMED_CASE_COUNT, 0) AS CONFIRMED_CASE_COUNT,
       fc.MAX_SOURCE_FRAUD_SCORE
FROM GOLD.FACT_CLAIM c

LEFT JOIN hist h ON h.CLAIM_ID = c.CLAIM_ID
LEFT JOIN fc     ON fc.CLAIM_ID = c.CLAIM_ID;
CREATE TABLE IF NOT EXISTS GOLD.MART_FRAUD_RISK AS SELECT * FROM GOLD.BLD_MART_FRAUD_RISK;
```

Reconciliation rule

The settlement mart compares the claim's **approved amount** with the **gross** settlement (net = gross − deductible − recovery is already validated). If your data shows approved = net, switch TOTAL_GROSS to TOTAL_NET in the CASE.

9.5 Fraud score table (filled by Snowpark in Phase 12)

```
CREATE TABLE IF NOT EXISTS GOLD.FRAUD_RISK_SCORE (
  CLAIM_ID STRING, CUSTOMER_ID STRING, POLICY_ID STRING, REGION STRING, INSURANCE_TYPE STRING,
  CLAIM_AMOUNT NUMBER(18,2), DAYS_POLICY_TO_LOSS NUMBER, PRIOR_CLAIMS_COUNT NUMBER,
  OPEN_CASE_COUNT NUMBER, CONFIRMED_CASE_COUNT NUMBER,
  AMOUNT_SCORE NUMBER, EARLY_CLAIM_SCORE NUMBER, HISTORY_SCORE NUMBER, INVESTIGATION_SCORE NUMBER,
  TOTAL_SCORE NUMBER, RISK_CATEGORY STRING, SCORE_REASON STRING,
  MODEL_VERSION STRING, SCORING_TS TIMESTAMP_NTZ
);
CREATE TABLE IF NOT EXISTS GOLD.FRAUD_RISK_SCORE_HISTORY LIKE GOLD.FRAUD_RISK_SCORE;
```

9.6 KPI view – the 10 required KPIs (per region + total)

sql/34_kpi_view.sql

```
CREATE OR REPLACE VIEW GOLD.V_KPI_SUMMARY AS
WITH r AS (SELECT DISTINCT REGION FROM GOLD.DIM_POLICY UNION SELECT 'ALL REGIONS'),
pol AS (SELECT IFF(GROUPING(REGION) = 1, 'ALL REGIONS', REGION) AS REGION,
               COUNT(*) AS TOTAL_POLICIES, COUNT_IF(IS_ACTIVE) AS ACTIVE_POLICIES,
               SUM(IFF(IS_ACTIVE, SUM_INSURED, 0)) AS ACTIVE_SUM_INSURED
        FROM GOLD.DIM_POLICY GROUP BY ROLLUP (REGION)),
pay AS (SELECT IFF(GROUPING(REGION) = 1, 'ALL REGIONS', REGION) AS REGION,
```

```
               SUM(IFF(IS_SUCCESSFUL, PAYMENT_AMOUNT, 0)) AS PREMIUM_COLLECTED
        FROM GOLD.FACT_PREMIUM_PAYMENT GROUP BY ROLLUP (REGION)),
clm AS (SELECT IFF(GROUPING(REGION) = 1, 'ALL REGIONS', REGION) AS REGION,
               COUNT(*) AS CLAIM_COUNT, SUM(CLAIM_AMOUNT) AS TOTAL_CLAIM_AMOUNT,
               COUNT_IF(IS_APPROVED) AS APPROVED_CLAIMS
        FROM GOLD.FACT_CLAIM GROUP BY ROLLUP (REGION)),
stl AS (SELECT IFF(GROUPING(REGION) = 1, 'ALL REGIONS', REGION) AS REGION,
               SUM(NET_SETTLEMENT_AMOUNT) AS NET_SETTLEMENT_AMOUNT,
               ROUND(AVG(TURNAROUND_DAYS), 1) AS AVG_SETTLEMENT_TAT_DAYS
        FROM GOLD.FACT_CLAIM_SETTLEMENT GROUP BY ROLLUP (REGION)),
frd AS (SELECT IFF(GROUPING(REGION) = 1, 'ALL REGIONS', REGION) AS REGION,
               COUNT(*) AS FRAUD_CASES, COUNT_IF(IS_OPEN) AS OPEN_FRAUD_INVESTIGATIONS,
               COUNT_IF(IS_CONFIRMED) AS CONFIRMED_FRAUD_CASES
        FROM GOLD.FACT_FRAUD_CASE GROUP BY ROLLUP (REGION)),
rsk AS (SELECT IFF(GROUPING(REGION) = 1, 'ALL REGIONS', REGION) AS REGION,
               COUNT_IF(RISK_CATEGORY = 'HIGH') AS HIGH_RISK_CLAIMS
        FROM GOLD.FRAUD_RISK_SCORE GROUP BY ROLLUP (REGION))
SELECT r.REGION,
  pol.ACTIVE_POLICIES, pol.ACTIVE_SUM_INSURED,                                          -- KPI 1
  pay.PREMIUM_COLLECTED,                                                                -- KPI 2
  clm.CLAIM_COUNT,
  ROUND(100 * clm.CLAIM_COUNT / NULLIF(pol.TOTAL_POLICIES, 0), 2) AS CLAIM_FREQUENCY_PER_100, -- KPI 3a
  ROUND(clm.TOTAL_CLAIM_AMOUNT / NULLIF(clm.CLAIM_COUNT, 0), 2) AS AVG_CLAIM_AMOUNT,          -- KPI 3b
  ROUND(100 * clm.APPROVED_CLAIMS / NULLIF(clm.CLAIM_COUNT, 0), 2) AS APPROVAL_RATIO_PCT,     -- KPI 4
  stl.AVG_SETTLEMENT_TAT_DAYS,                                                          -- KPI 5
  stl.NET_SETTLEMENT_AMOUNT,                                                            -- KPI 6
  ROUND(100 * stl.NET_SETTLEMENT_AMOUNT / NULLIF(pay.PREMIUM_COLLECTED, 0), 2) AS LOSS_RATIO_PCT, -- KPI 7
  frd.OPEN_FRAUD_INVESTIGATIONS,                                                        -- KPI 8
  ROUND(100 * frd.CONFIRMED_FRAUD_CASES / NULLIF(frd.FRAUD_CASES, 0), 2) AS CONFIRMED_FRAUD_RATE_PCT, -- KPI 9
  COALESCE(rsk.HIGH_RISK_CLAIMS, 0) AS HIGH_RISK_CLAIMS,                                -- KPI 10
  pol.TOTAL_POLICIES, clm.APPROVED_CLAIMS, frd.FRAUD_CASES, frd.CONFIRMED_FRAUD_CASES
FROM r
LEFT JOIN pol ON pol.REGION = r.REGION   LEFT JOIN pay ON pay.REGION = r.REGION
LEFT JOIN clm ON clm.REGION = r.REGION   LEFT JOIN stl ON stl.REGION = r.REGION
LEFT JOIN frd ON frd.REGION = r.REGION   LEFT JOIN rsk ON rsk.REGION = r.REGION;
```

KPI (document)	Formula used
Active policies and sum insured	count / sum of policies where IS_ACTIVE
Premium collected	sum of successful payment_amount
Claim frequency and average claim	claims ÷ policies × 100 • total claim amount ÷ claims
Approval ratio	approved claims ÷ all claims × 100
Settlement turnaround time	average days from claim registration to settlement
Net settlement amount	sum of net_settlement_amount
Loss ratio proxy	net settlement ÷ premium collected × 100
Open fraud investigations	fraud cases with no end date and status not closed
Confirmed fraud rate	confirmed cases ÷ all fraud cases × 100
High-risk claims	claims with RISK_CATEGORY = HIGH (Snowpark)

9.7 Gold refresh procedure

sql/35_sp_refresh_gold.sql

```
CREATE OR REPLACE PROCEDURE GOLD.SP_REFRESH_GOLD()
RETURNS STRING
LANGUAGE SQL
AS
$$
BEGIN
  BEGIN TRANSACTION;          -- dashboards never see a half-refreshed Gold
  INSERT OVERWRITE INTO GOLD.DIM_CUSTOMER SELECT * FROM GOLD.BLD_DIM_CUSTOMER;
  INSERT OVERWRITE INTO GOLD.DIM_INSURANCE_PRODUCT SELECT * FROM GOLD.BLD_DIM_INSURANCE_PRODUCT;
  INSERT OVERWRITE INTO GOLD.DIM_POLICY SELECT * FROM GOLD.BLD_DIM_POLICY;
  INSERT OVERWRITE INTO GOLD.FACT_PREMIUM_PAYMENT SELECT * FROM GOLD.BLD_FACT_PREMIUM_PAYMENT;
  INSERT OVERWRITE INTO GOLD.FACT_CLAIM SELECT * FROM GOLD.BLD_FACT_CLAIM;
  INSERT OVERWRITE INTO GOLD.FACT_CLAIM_SETTLEMENT SELECT * FROM GOLD.BLD_FACT_CLAIM_SETTLEMENT;
  INSERT OVERWRITE INTO GOLD.FACT_FRAUD_CASE SELECT * FROM GOLD.BLD_FACT_FRAUD_CASE;
  INSERT OVERWRITE INTO GOLD.MART_POLICY_PORTFOLIO SELECT * FROM GOLD.BLD_MART_POLICY_PORTFOLIO;
  INSERT OVERWRITE INTO GOLD.MART_CLAIMS_PERFORMANCE SELECT * FROM GOLD.BLD_MART_CLAIMS_PERFORMANCE;
  INSERT OVERWRITE INTO GOLD.MART_SETTLEMENT_RECONCILIATION SELECT * FROM GOLD.BLD_MART_SETTLEMENT_RECONCILIATION;
  INSERT OVERWRITE INTO GOLD.MART_FRAUD_RISK SELECT * FROM GOLD.BLD_MART_FRAUD_RISK;
  COMMIT;
```

```
  RETURN 'Gold refreshed';
END;
$$;
```

Order matters

Dimensions first, then facts (they join to the dimension *tables*), then marts. The procedure above already runs in the right order.

✔ **Done when:**

- ■ 11 Gold tables + DIM_DATE + FRAUD_RISK_SCORE exist; V_KPI_SUMMARY returns one row per region plus ALL REGIONS.
- ■ Grain test: SELECT CLAIM_ID, COUNT(*) FROM GOLD.FACT_CLAIM GROUP BY 1 HAVING COUNT(*) > 1 returns nothing (repeat for every key).
- ■ No orphan in Gold: every FACT_CLAIM.POLICY_ID exists in DIM_POLICY.

Phase 10 – Incremental Claims: Stream, MERGE and Task Graph

In simple words: a **Stream** is a bookmark on BRONZE.CLAIMS – it shows only rows added since the last time we read it. A **MERGE** inserts new claims and updates corrected ones (only if the new version is newer). A **Task graph** runs everything automatically in the right order.

10.1 Stream and MERGE procedure

sql/40_stream_merge.sql

```
CREATE STREAM IF NOT EXISTS BRONZE.CLAIMS_STRM
  ON TABLE BRONZE.CLAIMS
  APPEND_ONLY = TRUE           -- Bronze is insert-only; we only need new rows
  SHOW_INITIAL_ROWS = TRUE;    -- rows already in Bronze are delivered on the first read

CREATE OR REPLACE PROCEDURE SILVER.SP_MERGE_CLAIMS()
RETURNS STRING
LANGUAGE SQL
AS
$$
DECLARE
  n_ins NUMBER DEFAULT 0;
  n_upd NUMBER DEFAULT 0;
BEGIN
  MERGE INTO SILVER.CLAIMS t
  USING (
    SELECT
      NULLIF(TRIM(CLAIM_ID), '') AS CLAIM_ID,
      NULLIF(TRIM(POLICY_ID), '') AS POLICY_ID,
      NULLIF(TRIM(CUSTOMER_ID), '') AS CUSTOMER_ID,
      TRY_TO_DATE(CLAIM_REGISTRATION_DATE, 'YYYY-MM-DD') AS CLAIM_REGISTRATION_DATE,
      TRY_TO_DATE(LOSS_DATE, 'YYYY-MM-DD') AS LOSS_DATE,
      NULLIF(TRIM(CLAIM_TYPE), '') AS CLAIM_TYPE,
      TRY_TO_NUMBER(CLAIM_AMOUNT, 18, 2) AS CLAIM_AMOUNT,
      TRY_TO_NUMBER(APPROVED_AMOUNT, 18, 2) AS APPROVED_AMOUNT,
      NULLIF(TRIM(CLAIM_STATUS), '') AS CLAIM_STATUS,
      NULLIF(TRIM(ASSESSOR_ID), '') AS ASSESSOR_ID,
      NULLIF(TRIM(BRANCH_ID), '') AS BRANCH_ID,
      TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') AS SOURCE_UPDATED_TS,
      DQ_STATUS, DQ_REASON, _FILE_NAME AS SRC_FILE_NAME, _LOAD_TS AS BRONZE_LOAD_TS
    FROM BRONZE.CLAIMS_STRM
    WHERE NULLIF(TRIM(CLAIM_ID), '') IS NOT NULL
    QUALIFY ROW_NUMBER() OVER (
              PARTITION BY NULLIF(TRIM(CLAIM_ID), '')
              ORDER BY TRY_TO_TIMESTAMP_NTZ(SOURCE_UPDATED_TS, 'YYYY-MM-DD HH24:MI:SS') DESC NULLS LAST,
                       _LOAD_TS DESC) = 1
  ) s
  ON t.CLAIM_ID = s.CLAIM_ID
  WHEN MATCHED AND COALESCE(s.SOURCE_UPDATED_TS, '1900-01-01'::TIMESTAMP_NTZ)
                 > COALESCE(t.SOURCE_UPDATED_TS, '1900-01-01'::TIMESTAMP_NTZ) THEN UPDATE SET
     t.POLICY_ID = s.POLICY_ID,
     t.CUSTOMER_ID = s.CUSTOMER_ID,
     t.CLAIM_REGISTRATION_DATE = s.CLAIM_REGISTRATION_DATE,
     t.LOSS_DATE = s.LOSS_DATE,
     t.CLAIM_TYPE = s.CLAIM_TYPE,
     t.CLAIM_AMOUNT = s.CLAIM_AMOUNT,
     t.APPROVED_AMOUNT = s.APPROVED_AMOUNT,
     t.CLAIM_STATUS = s.CLAIM_STATUS,
     t.ASSESSOR_ID = s.ASSESSOR_ID,
     t.BRANCH_ID = s.BRANCH_ID,
     t.SOURCE_UPDATED_TS = s.SOURCE_UPDATED_TS,
     t.DQ_STATUS = s.DQ_STATUS,
     t.DQ_REASON = s.DQ_REASON,
     t.SRC_FILE_NAME = s.SRC_FILE_NAME,
     t.BRONZE_LOAD_TS = s.BRONZE_LOAD_TS,
     t.SILVER_UPDATED_TS = CURRENT_TIMESTAMP()
  WHEN NOT MATCHED THEN INSERT (
     CLAIM_ID, POLICY_ID, CUSTOMER_ID, CLAIM_REGISTRATION_DATE, LOSS_DATE, CLAIM_TYPE,
     CLAIM_AMOUNT, APPROVED_AMOUNT, CLAIM_STATUS, ASSESSOR_ID, BRANCH_ID,
     SOURCE_UPDATED_TS, DQ_STATUS, DQ_REASON, SRC_FILE_NAME, BRONZE_LOAD_TS,
     SILVER_UPDATED_TS)
  VALUES (
     s.CLAIM_ID, s.POLICY_ID, s.CUSTOMER_ID, s.CLAIM_REGISTRATION_DATE, s.LOSS_DATE,
     s.CLAIM_TYPE, s.CLAIM_AMOUNT, s.APPROVED_AMOUNT, s.CLAIM_STATUS, s.ASSESSOR_ID,
     s.BRANCH_ID, s.SOURCE_UPDATED_TS, s.DQ_STATUS, s.DQ_REASON, s.SRC_FILE_NAME,
     s.BRONZE_LOAD_TS, CURRENT_TIMESTAMP());
```

```
  SELECT $1, $2 INTO :n_ins, :n_upd FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
  RETURN 'claims inserted=' || n_ins || ', updated=' || n_upd;
END;

$$;
```

Why this is idempotent

(1) COPY never reloads a file it already loaded. (2) Even if the same rows arrive again, the MERGE updates only when SOURCE_UPDATED_TS is **newer**, and never inserts an existing CLAIM_ID. (3) The control check ignores a .ctl name it has already seen. Three layers of protection.

10.2 The task graph (runs the whole Snowflake side)

sql/41_task_graph.sql

```
-- Root: every 30 minutes during working hours (adjust to save credits)
CREATE OR REPLACE TASK CONTROL.T_ROOT_LOAD_BRONZE
  WAREHOUSE = CG_WH
  SCHEDULE  = 'USING CRON */30 8-20 * * MON-SAT Asia/Kolkata'
AS CALL CONTROL.SP_LOAD_BRONZE();

CREATE OR REPLACE TASK SILVER.T_SILVER_MASTERS  WAREHOUSE = CG_WH
  AFTER CONTROL.T_ROOT_LOAD_BRONZE  AS CALL SILVER.SP_BUILD_SILVER();

CREATE OR REPLACE TASK SILVER.T_CLAIMS_MERGE    WAREHOUSE = CG_WH
  AFTER CONTROL.T_ROOT_LOAD_BRONZE  AS CALL SILVER.SP_MERGE_CLAIMS();

CREATE OR REPLACE TASK CONTROL.T_REJECT_REGISTER WAREHOUSE = CG_WH        -- Phase 11
  AFTER CONTROL.T_ROOT_LOAD_BRONZE  AS CALL CONTROL.SP_UPDATE_REJECT_REGISTER();

CREATE OR REPLACE TASK SILVER.T_SILVER_DQ       WAREHOUSE = CG_WH
  AFTER SILVER.T_SILVER_MASTERS, SILVER.T_CLAIMS_MERGE  AS CALL SILVER.SP_SILVER_DQ();

CREATE OR REPLACE TASK GOLD.T_GOLD_REFRESH      WAREHOUSE = CG_WH
  AFTER SILVER.T_SILVER_DQ          AS CALL GOLD.SP_REFRESH_GOLD();

CREATE OR REPLACE TASK GOLD.T_FRAUD_SCORE       WAREHOUSE = CG_WH        -- Phase 12
  AFTER GOLD.T_GOLD_REFRESH         AS CALL GOLD.SP_FRAUD_SCORE();

CREATE OR REPLACE TASK CONTROL.T_CHECK_THRESHOLDS WAREHOUSE = CG_WH      -- Phase 14
  AFTER GOLD.T_FRAUD_SCORE          AS CALL CONTROL.SP_CHECK_THRESHOLDS();

-- Create the procedures of Phases 11, 12 and 14 BEFORE resuming. Then enable everything at once:
SELECT SYSTEM$TASK_DEPENDENTS_ENABLE('INSURANCE_DB.CONTROL.T_ROOT_LOAD_BRONZE');

-- Run the whole graph now instead of waiting for the schedule:
EXECUTE TASK CONTROL.T_ROOT_LOAD_BRONZE;

-- Watch it:
SELECT NAME, STATE, SCHEDULED_TIME, COMPLETED_TIME, RETURN_VALUE, ERROR_MESSAGE
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(SCHEDULED_TIME_RANGE_START => DATEADD(hour, -2, CURRENT_TIMESTAMP())))
ORDER BY SCHEDULED_TIME DESC;
```

Task graph shape

```
T_ROOT_LOAD_BRONZE (schedule)
 |-- T_SILVER_MASTERS --+
 |-- T_CLAIMS_MERGE ----+--> T_SILVER_DQ --> T_GOLD_REFRESH --> T_FRAUD_SCORE --> T_CHECK_THRESHOLDS
 |-- T_REJECT_REGISTER
```

Cost control

A suspended graph costs nothing. After the demo: `ALTER TASK CONTROL.T_ROOT_LOAD_BRONZE SUSPEND;` (suspending the root stops the whole graph).

10.3 The incremental and replay test (required by the document)

tests/incremental_replay.sql

```
-- STEP 0: baseline after the initial load
SELECT COUNT(*) AS CLAIMS, HASH_AGG(*) AS FINGERPRINT FROM SILVER.CLAIMS;

-- STEP 1: second file. Upload claims_20261104.csv (new + corrected claims) to landing/claims/,
--         claims_20261104.ctl to control/, then _BATCH_READY_20261104.done to control/.
--         IDMC runs by itself; then:
EXECUTE TASK CONTROL.T_ROOT_LOAD_BRONZE;
SELECT * FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY()) WHERE NAME = 'T_CLAIMS_MERGE'
ORDER BY SCHEDULED_TIME DESC LIMIT 1;          -- RETURN_VALUE shows inserted / updated counts
SELECT CLAIM_ID, CLAIM_STATUS, APPROVED_AMOUNT, SOURCE_UPDATED_TS, SILVER_UPDATED_TS
FROM SILVER.CLAIMS WHERE CLAIM_ID IN ('<a corrected claim id>', '<a new claim id>');

-- STEP 2: replay A — run the load again: nothing changes
CALL CONTROL.SP_LOAD_BRONZE();                 -- returns NOTHING_NEW (COPY skipped the files)

-- STEP 3: replay B — force the same file into Bronze a second time (simulates a bad re-send)
SELECT COUNT(*) AS CLAIMS, HASH_AGG(*) AS FINGERPRINT FROM SILVER.CLAIMS;      -- note values
-- Run the Phase 7 'COPY INTO BRONZE.CLAIMS ...' statement again, adding FORCE = TRUE
-- just before the final semicolon. Bronze now holds every claim row twice.
SELECT COUNT(*) FROM BRONZE.CLAIMS;            -- went up
CALL SILVER.SP_MERGE_CLAIMS();                 -- inserted=0, updated=0
SELECT COUNT(*) AS CLAIMS, HASH_AGG(*) AS FINGERPRINT FROM SILVER.CLAIMS;      -- same values = idempotent
```

Simplest way to run replay B

Copy the Phase 7 COPY INTO BRONZE.CLAIMS statement and add FORCE = TRUE before the semicolon. Bronze row count goes up, Silver count and HASH_AGG stay identical – that screenshot is your idempotency evidence.

✔ **Done when:**

- Second file: new claims inserted, corrected claims updated (SILVER_UPDATED_TS changed).
- Replay A returns NOTHING_NEW; replay B leaves Silver count and HASH_AGG unchanged.
- Gold marts and KPI view reflect the second file after the graph runs.

Phase 11 – ★ Reject Reprocessing Loop

In simple words: a rejected record is treated like a support ticket. It gets an ID, a reason and a status. The business fixes the row in the reject file and drops it into **resubmit/**. The pipeline checks it again with the same rules. If it passes, it flows into Bronze carrying its original reject_id and the ticket closes itself as **RESOLVED**. After three failed attempts it becomes **CLOSED_NOT_FIXABLE**.

11.1 Life cycle

Status	Meaning	Set by
OPEN	Rejected once, waiting for a fix	SP_UPDATE_REJECT_REGISTER (from REJECT_LOG)
RESUBMITTED_FAILED	Fixed file sent but still fails a rule (attempt 2)	same procedure
RESOLVED	reject_id now present in a Bronze table	same procedure
CLOSED_NOT_FIXABLE	3 failed attempts – needs business decision	same procedure

11.2 Register procedure

sql/50_reject_register.sql

```
CREATE OR REPLACE PROCEDURE CONTROL.SP_UPDATE_REJECT_REGISTER()
RETURNS STRING
LANGUAGE SQL
AS
$$
BEGIN
  -- 1. add new rejects and refresh attempt counts from the log written by IDMC
  MERGE INTO CONTROL.REJECT_REGISTER r
  USING (
    SELECT REJECT_ID,
           ANY_VALUE(SOURCE_NAME)            AS SOURCE_NAME,
           ANY_VALUE(RECORD_KEY)             AS RECORD_KEY,
           MIN_BY(REJECT_REASON, LOGGED_TS)  AS FIRST_REASON,
           MAX_BY(REJECT_REASON, LOGGED_TS)  AS LAST_REASON,
           COUNT(*)                          AS ATTEMPTS,
           MIN(LOGGED_TS) AS FIRST_TS, MAX(LOGGED_TS) AS LAST_TS
    FROM CONTROL.REJECT_LOG
    GROUP BY REJECT_ID
  ) l
  ON r.REJECT_ID = l.REJECT_ID
  WHEN MATCHED AND r.ATTEMPT_COUNT <> l.ATTEMPTS THEN UPDATE SET
       r.LAST_REASON = l.LAST_REASON, r.ATTEMPT_COUNT = l.ATTEMPTS,
       r.LAST_ATTEMPT_TS = l.LAST_TS, r.UPDATED_TS = CURRENT_TIMESTAMP()
  WHEN NOT MATCHED THEN INSERT (REJECT_ID, SOURCE_NAME, RECORD_KEY, FIRST_REASON, LAST_REASON,
       ATTEMPT_COUNT, STATUS, FIRST_SEEN_TS, LAST_ATTEMPT_TS, UPDATED_TS)
  VALUES (l.REJECT_ID, l.SOURCE_NAME, l.RECORD_KEY, l.FIRST_REASON, l.LAST_REASON,
       l.ATTEMPTS, 'OPEN', l.FIRST_TS, l.LAST_TS, CURRENT_TIMESTAMP());

  -- 2. RESOLVED: the corrected row reached Bronze with its reject_id
  UPDATE CONTROL.REJECT_REGISTER r
     SET STATUS = 'RESOLVED', RESOLVED_TS = CURRENT_TIMESTAMP(), UPDATED_TS = CURRENT_TIMESTAMP()
    FROM (SELECT REJECT_ID FROM BRONZE.CUSTOMERS          WHERE NULLIF(TRIM(REJECT_ID),'') IS NOT NULL
          UNION SELECT REJECT_ID FROM BRONZE.INSURANCE_PRODUCTS WHERE NULLIF(TRIM(REJECT_ID),'') IS NOT NULL
          UNION SELECT REJECT_ID FROM BRONZE.POLICIES     WHERE NULLIF(TRIM(REJECT_ID),'') IS NOT NULL
          UNION SELECT REJECT_ID FROM BRONZE.PREMIUM_PAYMENTS WHERE NULLIF(TRIM(REJECT_ID),'') IS NOT NULL
          UNION SELECT REJECT_ID FROM BRONZE.CLAIMS       WHERE NULLIF(TRIM(REJECT_ID),'') IS NOT NULL
          UNION SELECT REJECT_ID FROM BRONZE.CLAIM_SETTLEMENTS WHERE NULLIF(TRIM(REJECT_ID),'') IS NOT NULL
          UNION SELECT REJECT_ID FROM BRONZE.FRAUD_CASES  WHERE NULLIF(TRIM(REJECT_ID),'') IS NOT NULL) b
   WHERE r.REJECT_ID = TRIM(b.REJECT_ID) AND r.STATUS <> 'RESOLVED';

  -- 3. failed again
  UPDATE CONTROL.REJECT_REGISTER
     SET STATUS = IFF(ATTEMPT_COUNT >= 3, 'CLOSED_NOT_FIXABLE', 'RESUBMITTED_FAILED'),
         UPDATED_TS = CURRENT_TIMESTAMP()
   WHERE STATUS IN ('OPEN', 'RESUBMITTED_FAILED') AND ATTEMPT_COUNT > 1;

  RETURN 'Reject register updated';
END;
$$;

-- what the business must fix, oldest first
CREATE OR REPLACE VIEW CONTROL.V_REJECTS_TO_FIX AS
```

```
SELECT r.REJECT_ID, r.SOURCE_NAME, r.RECORD_KEY, r.LAST_REASON, r.ATTEMPT_COUNT, r.STATUS,
       DATEDIFF(day, r.FIRST_SEEN_TS, CURRENT_TIMESTAMP()) AS AGE_DAYS,
       l.RAW_RECORD
FROM CONTROL.REJECT_REGISTER r
JOIN CONTROL.REJECT_LOG l ON l.REJECT_ID = r.REJECT_ID AND l.LOGGED_TS = r.LAST_ATTEMPT_TS
WHERE r.STATUS IN ('OPEN', 'RESUBMITTED_FAILED');
```

11.3 IDMC resubmit mappings

Rejects in the kit come from policies, claims and claim_settlements, so build resubmit mappings for those three (the same steps work for any source).

1. Open m_clean_claims → **Save As** → **m_clean_claims_resubmit**.
2. Source: sample object **reject/claims/claims_reject.csv** (it has reject_id and reject_reason); Advanced → Source Type **Directory**, Folder Path **…/resubmit/claims**.
3. EXP_DQ: change **o_reject_id** to keep the original ID:
   `o_reject_id = IIF(o_dq_route = 'REJECT', LTRIM(RTRIM(reject_id)), NULL)`
   and add `o_keep_reject_id = LTRIM(RTRIM(reject_id));` set `o_attempt_type = 'RESUBMIT'`.
4. TGT_CLEAN → object **clean/claims/claims_resubmit_clean.csv**, and now **map o_keep_reject_id → reject_id**. This is the link that lets Snowflake mark the ticket RESOLVED.
5. TGT_REJ_S3 → **reject/claims/claims_reject_again.csv**; TGT_REJ_LOG unchanged (it appends attempt 2, 3 …).
6. Validate → mapping task **mt_clean_claims_resubmit**. Repeat for policies and claim_settlements.

11.4 Resubmit taskflow and listener

1. Taskflow **tf_cg_resubmit**: Data Task mt_clean_policies_resubmit → mt_clean_claims_resubmit → mt_clean_claim_settlements_resubmit → Command Tasks **cg_move.sh resubmit/<source>/ archive/resubmit/<source>/** for each → End. (The _placeholder.csv files stay behind thanks to --exclude.)
2. File Listener **fl_cg_resubmit**: connection CONN_S3_CG, Folder Path **resubmit**, pattern ***.done**, event Arrive, same schedule and stability check as fl_cg_batch.
3. Bind: tf_cg_resubmit Start → Binding **Event** → fl_cg_resubmit → **Publish**. Start the listener.

11.5 Business user procedure (put this in the runbook)

1. Download **reject/claims/claims_reject.csv** from S3 (or query CONTROL.V_REJECTS_TO_FIX).
2. Correct the bad value (e.g. the loss_date). **Do not change reject_id.** Delete rows you cannot fix.
3. Save as **claims_fix_20261105.csv** and upload to **resubmit/claims/**.
4. Upload an empty **_RESUBMIT_READY_20261105.done** to **resubmit/**.
5. After the next task-graph run the register shows **RESOLVED**.

Evidence queries

```
SELECT STATUS, COUNT(*) FROM CONTROL.REJECT_REGISTER GROUP BY 1;
SELECT * FROM CONTROL.REJECT_REGISTER WHERE SOURCE_NAME = 'claims' ORDER BY UPDATED_TS DESC;
SELECT CLAIM_ID, REJECT_ID FROM BRONZE.CLAIMS WHERE NULLIF(REJECT_ID,'') IS NOT NULL;  -- the fixed row
```

✔ **Done when:**

- ■ 3 OPEN rejects after the first run.
- ■ One fixed claim → RESOLVED, and the claim appears in SILVER.CLAIMS and GOLD.FACT_CLAIM.
- ■ A deliberately still-wrong row resubmitted twice more → CLOSED_NOT_FIXABLE.

Phase 12 – Snowpark Fraud-Risk Score

In simple words: a Python procedure running *inside* Snowflake gives every claim points for four warning signs, adds them up (0–100) and labels the claim LOW, MEDIUM or HIGH. Every component is stored, so anyone can see *why* a claim is HIGH – no black box, exactly as the document asks.

12.1 Scoring rules

Component	Rule (points)	Max
AMOUNT_SCORE	claim_amount ≥ 1,000,000 → 30 • ≥ 500,000 → 20 • ≥ 200,000 → 10 • else 0	30
EARLY_CLAIM_SCORE	days policy start → loss ≤ 30 → 25 • ≤ 90 → 15 • ≤ 180 → 5 • else 0	25
HISTORY_SCORE	earlier claims by the same customer: ≥ 3 → 20 • 2 → 12 • 1 → 6 • 0 → 0	20
INVESTIGATION_SCORE	confirmed fraud case → 25 • open case → 15 • any case or source score ≥ 70 → 10	25
TOTAL_SCORE → RISK_CATEGORY	≥ 60 HIGH • 30–59 MEDIUM • < 30 LOW	100

Tune with your data

Look at the distribution first: `SELECT APPROX_PERCENTILE(CLAIM_AMOUNT, 0.5), APPROX_PERCENTILE(CLAIM_AMOUNT, 0.9) FROM GOLD.FACT_CLAIM;` Set the amount bands near the 75th / 90th / 97th percentiles. Record the final rules in the code comment and bump MODEL_VERSION whenever you change them.

12.2 The procedure

sql/60_sp_fraud_score.sql

```
CREATE OR REPLACE PROCEDURE GOLD.SP_FRAUD_SCORE()
RETURNS STRING
LANGUAGE PYTHON
RUNTIME_VERSION = '3.11'
PACKAGES = ('snowflake-snowpark-python')
HANDLER = 'main'
AS
$$
from snowflake.snowpark import functions as F

MODEL_VERSION = 'CG-RISK-v1.0'


def main(session):
    src = session.table('GOLD.MART_FRAUD_RISK')

    amount = (F.when(F.col('CLAIM_AMOUNT') >= 1000000, 30)
               .when(F.col('CLAIM_AMOUNT') >= 500000, 20)
               .when(F.col('CLAIM_AMOUNT') >= 200000, 10)
               .otherwise(0))
    early = (F.when(F.col('DAYS_POLICY_TO_LOSS').is_null(), 0)
              .when(F.col('DAYS_POLICY_TO_LOSS') <= 30, 25)
              .when(F.col('DAYS_POLICY_TO_LOSS') <= 90, 15)
              .when(F.col('DAYS_POLICY_TO_LOSS') <= 180, 5)
              .otherwise(0))
    history = (F.when(F.col('PRIOR_CLAIMS_COUNT') >= 3, 20)
                .when(F.col('PRIOR_CLAIMS_COUNT') == 2, 12)
                .when(F.col('PRIOR_CLAIMS_COUNT') == 1, 6)
                .otherwise(0))
    invest = (F.when(F.col('CONFIRMED_CASE_COUNT') > 0, 25)
               .when(F.col('OPEN_CASE_COUNT') > 0, 15)
               .when((F.col('FRAUD_CASE_COUNT') > 0) |
                     (F.coalesce(F.col('MAX_SOURCE_FRAUD_SCORE'), F.lit(0)) >= 70), 10)
               .otherwise(0))

    df = (src.with_column('AMOUNT_SCORE', amount)
             .with_column('EARLY_CLAIM_SCORE', early)
             .with_column('HISTORY_SCORE', history)
             .with_column('INVESTIGATION_SCORE', invest))
    df = df.with_column('TOTAL_SCORE', F.col('AMOUNT_SCORE') + F.col('EARLY_CLAIM_SCORE')
                                     + F.col('HISTORY_SCORE') + F.col('INVESTIGATION_SCORE'))
```

```
    df = df.with_column('RISK_CATEGORY',
                        F.when(F.col('TOTAL_SCORE') >= 60, 'HIGH')
                         .when(F.col('TOTAL_SCORE') >= 30, 'MEDIUM')
                         .otherwise('LOW'))
    reason = F.array_to_string(F.array_construct_compact(
        F.iff(F.col('AMOUNT_SCORE') > 0,
              F.concat(F.lit('High claim amount +'), F.col('AMOUNT_SCORE').cast('string')), F.lit(None)),
        F.iff(F.col('EARLY_CLAIM_SCORE') > 0,
              F.concat(F.lit('Early claim +'), F.col('EARLY_CLAIM_SCORE').cast('string')), F.lit(None)),
        F.iff(F.col('HISTORY_SCORE') > 0,
              F.concat(F.lit('Claim history +'), F.col('HISTORY_SCORE').cast('string')), F.lit(None)),
        F.iff(F.col('INVESTIGATION_SCORE') > 0,
              F.concat(F.lit('Fraud investigation +'), F.col('INVESTIGATION_SCORE').cast('string')),
              F.lit(None))), F.lit('; '))

    out = df.select(
        'CLAIM_ID', 'CUSTOMER_ID', 'POLICY_ID', 'REGION', 'INSURANCE_TYPE', 'CLAIM_AMOUNT',
        'DAYS_POLICY_TO_LOSS', 'PRIOR_CLAIMS_COUNT', 'OPEN_CASE_COUNT', 'CONFIRMED_CASE_COUNT',
        'AMOUNT_SCORE', 'EARLY_CLAIM_SCORE', 'HISTORY_SCORE', 'INVESTIGATION_SCORE',
        'TOTAL_SCORE', 'RISK_CATEGORY',
        reason.alias('SCORE_REASON'),
        F.lit(MODEL_VERSION).alias('MODEL_VERSION'),
        F.current_timestamp().cast('timestamp_ntz').alias('SCORING_TS'))

    # 'truncate' keeps the table (and its security policies); column_order='name' matches by name
    out.write.save_as_table('GOLD.FRAUD_RISK_SCORE', mode='truncate', column_order='name')
    session.table('GOLD.FRAUD_RISK_SCORE').write.save_as_table(
        'GOLD.FRAUD_RISK_SCORE_HISTORY', mode='append', column_order='name')

    counts = session.table('GOLD.FRAUD_RISK_SCORE').group_by('RISK_CATEGORY').count().collect()
    return MODEL_VERSION + ' scored: ' + ', '.join(f"{r['RISK_CATEGORY']}={r['COUNT']}" for r in counts)
$$;

CALL GOLD.SP_FRAUD_SCORE();
SELECT RISK_CATEGORY, COUNT(*), MIN(TOTAL_SCORE), MAX(TOTAL_SCORE)
FROM GOLD.FRAUD_RISK_SCORE GROUP BY 1 ORDER BY 2 DESC;
```

12.3 Prove it is reproducible

tests/snowpark_reproducible.sql

```
CALL GOLD.SP_FRAUD_SCORE();
CREATE OR REPLACE TEMPORARY TABLE RUN1 AS
  SELECT CLAIM_ID, TOTAL_SCORE, RISK_CATEGORY FROM GOLD.FRAUD_RISK_SCORE;
CALL GOLD.SP_FRAUD_SCORE();
SELECT COUNT(*) AS DIFFERENCES
FROM RUN1 a JOIN GOLD.FRAUD_RISK_SCORE b USING (CLAIM_ID)
WHERE a.TOTAL_SCORE <> b.TOTAL_SCORE OR a.RISK_CATEGORY <> b.RISK_CATEGORY;     -- expected 0

-- explain one HIGH claim
SELECT CLAIM_ID, AMOUNT_SCORE, EARLY_CLAIM_SCORE, HISTORY_SCORE, INVESTIGATION_SCORE,
       TOTAL_SCORE, RISK_CATEGORY, SCORE_REASON
FROM GOLD.FRAUD_RISK_SCORE WHERE RISK_CATEGORY = 'HIGH' ORDER BY TOTAL_SCORE DESC LIMIT 5;
```

✔ **Done when:**

- FRAUD_RISK_SCORE has one row per claim in FACT_CLAIM.
- Two runs → 0 differences. The INR 25,000,000 outlier claim has AMOUNT_SCORE = 30.
- SCORE_REASON explains every HIGH claim in plain words.

Phase 13 – Security and ★ Region-Based Access

In simple words: three layers. **Roles** decide which objects you can open. **Masking** hides personal details inside a column. The **row access policy** decides which *rows* you see, based on region. Business users only ever read **secure views** in CONSUMPTION.

13.1 Roles

sql/70_roles.sql

```
USE ROLE SECURITYADMIN;
CREATE ROLE IF NOT EXISTS INSURANCE_ANALYST_ROLE;
CREATE ROLE IF NOT EXISTS CLAIMS_ANALYST_ROLE;
CREATE ROLE IF NOT EXISTS FRAUD_MANAGER_ROLE;
CREATE ROLE IF NOT EXISTS CG_APP_ROLE;                      -- owns the Streamlit app
-- regional demo roles inherit everything from the analyst role
CREATE ROLE IF NOT EXISTS INSURANCE_ANALYST_NORTH_ROLE;
CREATE ROLE IF NOT EXISTS INSURANCE_ANALYST_SOUTH_ROLE;
GRANT ROLE INSURANCE_ANALYST_ROLE TO ROLE INSURANCE_ANALYST_NORTH_ROLE;
GRANT ROLE INSURANCE_ANALYST_ROLE TO ROLE INSURANCE_ANALYST_SOUTH_ROLE;

GRANT ROLE INSURANCE_ANALYST_NORTH_ROLE, INSURANCE_ANALYST_SOUTH_ROLE TO ROLE SYSADMIN;
GRANT ROLE CLAIMS_ANALYST_ROLE, FRAUD_MANAGER_ROLE, CG_APP_ROLE    TO ROLE SYSADMIN;

-- for testing, give yourself the business roles
GRANT ROLE INSURANCE_ANALYST_NORTH_ROLE TO USER <your_user>;
GRANT ROLE INSURANCE_ANALYST_SOUTH_ROLE TO USER <your_user>;
GRANT ROLE CLAIMS_ANALYST_ROLE          TO USER <your_user>;
GRANT ROLE FRAUD_MANAGER_ROLE           TO USER <your_user>;

-- basic access (database + warehouse) for every business role
GRANT USAGE ON WAREHOUSE CG_WH TO ROLE INSURANCE_ANALYST_ROLE;
GRANT USAGE ON WAREHOUSE CG_WH TO ROLE CLAIMS_ANALYST_ROLE;
GRANT USAGE ON WAREHOUSE CG_WH TO ROLE FRAUD_MANAGER_ROLE;
GRANT USAGE ON WAREHOUSE CG_WH TO ROLE CG_APP_ROLE;
GRANT USAGE ON DATABASE INSURANCE_DB TO ROLE INSURANCE_ANALYST_ROLE;
GRANT USAGE ON DATABASE INSURANCE_DB TO ROLE CLAIMS_ANALYST_ROLE;
GRANT USAGE ON DATABASE INSURANCE_DB TO ROLE FRAUD_MANAGER_ROLE;
GRANT USAGE ON DATABASE INSURANCE_DB TO ROLE CG_APP_ROLE;
GRANT USAGE ON SCHEMA INSURANCE_DB.CONSUMPTION TO ROLE INSURANCE_ANALYST_ROLE;
GRANT USAGE ON SCHEMA INSURANCE_DB.CONSUMPTION TO ROLE CLAIMS_ANALYST_ROLE;
GRANT USAGE ON SCHEMA INSURANCE_DB.CONSUMPTION TO ROLE FRAUD_MANAGER_ROLE;
GRANT USAGE ON SCHEMA INSURANCE_DB.CONSUMPTION TO ROLE CG_APP_ROLE;
-- Bronze / Silver / Gold / Control: NO grants to business roles (engineering only)
```

13.2 Masking policies (PII)

sql/71_masking.sql

```
USE ROLE CG_ENGINEER_ROLE;  USE SCHEMA INSURANCE_DB.CONTROL;

CREATE OR REPLACE MASKING POLICY CONTROL.MP_EMAIL AS (VAL STRING) RETURNS STRING ->
  CASE WHEN IS_ROLE_IN_SESSION('FRAUD_MANAGER_ROLE') OR IS_ROLE_IN_SESSION('CG_ENGINEER_ROLE') THEN VAL
       WHEN VAL IS NULL THEN NULL
       ELSE REGEXP_REPLACE(VAL, '^[^@]+', '*****') END;             -- *****@gmail.com

CREATE OR REPLACE MASKING POLICY CONTROL.MP_PHONE AS (VAL STRING) RETURNS STRING ->
  CASE WHEN IS_ROLE_IN_SESSION('FRAUD_MANAGER_ROLE') OR IS_ROLE_IN_SESSION('CG_ENGINEER_ROLE') THEN VAL
       WHEN VAL IS NULL THEN NULL
       WHEN IS_ROLE_IN_SESSION('CLAIMS_ANALYST_ROLE') THEN '******' || RIGHT(VAL, 4)   -- last 4 digits
       ELSE '**********' END;

CREATE OR REPLACE MASKING POLICY CONTROL.MP_ADDRESS AS (VAL STRING) RETURNS STRING ->
  CASE WHEN IS_ROLE_IN_SESSION('FRAUD_MANAGER_ROLE') OR IS_ROLE_IN_SESSION('CG_ENGINEER_ROLE') THEN VAL
       WHEN VAL IS NULL THEN NULL
       ELSE '*** MASKED ***' END;

ALTER TABLE GOLD.DIM_CUSTOMER MODIFY COLUMN EMAIL        SET MASKING POLICY CONTROL.MP_EMAIL;
ALTER TABLE GOLD.DIM_CUSTOMER MODIFY COLUMN PHONE        SET MASKING POLICY CONTROL.MP_PHONE;
ALTER TABLE GOLD.DIM_CUSTOMER MODIFY COLUMN ADDRESS_LINE SET MASKING POLICY CONTROL.MP_ADDRESS;
```

Why is CG_ENGINEER_ROLE exempt?

Tasks and procedures run as CG_ENGINEER_ROLE. If the pipeline role saw masked values, it would **write the stars into the next table** and destroy the data. Mask only at the final Gold table and always exempt the pipeline role.

13.3 ★ Region row access policy

sql/72_row_access.sql

```
-- who sees which region (upper case region values as stored in Gold)
INSERT INTO CONTROL.USER_REGION_MAP (PRINCIPAL_TYPE, PRINCIPAL_NAME, REGION) VALUES
  ('ROLE', 'INSURANCE_ANALYST_NORTH_ROLE', 'NORTH'),
  ('ROLE', 'INSURANCE_ANALYST_SOUTH_ROLE', 'SOUTH'),
  ('ROLE', 'INSURANCE_ANALYST_ROLE',       'ALL'),
  ('ROLE', 'CLAIMS_ANALYST_ROLE',          'ALL'),
  ('ROLE', 'CG_APP_ROLE',                  'ALL'),     -- app filters per viewer itself (Phase 15)
  ('USER', '<YOUR_SNOWFLAKE_USER>',        'SOUTH');   -- used by the Streamlit app
-- check the real values first:  SELECT DISTINCT REGION FROM GOLD.DIM_CUSTOMER;

CREATE OR REPLACE ROW ACCESS POLICY CONTROL.RAP_REGION AS (P_REGION STRING) RETURNS BOOLEAN ->
     IS_ROLE_IN_SESSION('CG_ENGINEER_ROLE')            -- pipeline must see everything
  OR IS_ROLE_IN_SESSION('FRAUD_MANAGER_ROLE')          -- fraud works across regions
  OR EXISTS (SELECT 1 FROM CONTROL.USER_REGION_MAP m
             WHERE m.IS_ACTIVE
               AND m.PRINCIPAL_TYPE = 'ROLE'
               AND m.PRINCIPAL_NAME = CURRENT_ROLE()
               AND (m.REGION = 'ALL' OR m.REGION = UPPER(P_REGION)));

-- attach to every Gold object that has a REGION column
ALTER TABLE GOLD.DIM_CUSTOMER                   ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.DIM_POLICY                     ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.FACT_PREMIUM_PAYMENT           ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.FACT_CLAIM                     ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.FACT_CLAIM_SETTLEMENT          ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.FACT_FRAUD_CASE                ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.MART_POLICY_PORTFOLIO          ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.MART_CLAIMS_PERFORMANCE        ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.MART_SETTLEMENT_RECONCILIATION ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.MART_FRAUD_RISK                ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
ALTER TABLE GOLD.FRAUD_RISK_SCORE               ADD ROW ACCESS POLICY CONTROL.RAP_REGION ON (REGION);
```

Onboarding a new regional analyst

No code change: create the role (or reuse one), then one INSERT into USER_REGION_MAP. Removing access = set IS_ACTIVE = FALSE. Show this live – reviewers love it.

13.4 Secure views (the only thing business roles can read)

sql/73_secure_views.sql

```
USE SCHEMA INSURANCE_DB.CONSUMPTION;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_KPI_SUMMARY        AS SELECT * FROM GOLD.V_KPI_SUMMARY;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_POLICY_PORTFOLIO   AS SELECT * FROM GOLD.MART_POLICY_PORTFOLIO;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_CLAIMS_PERFORMANCE AS SELECT * FROM GOLD.MART_CLAIMS_PERFORMANCE;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_SETTLEMENT_RECON   AS
  SELECT * FROM GOLD.MART_SETTLEMENT_RECONCILIATION;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_FRAUD_RISK         AS SELECT * FROM GOLD.FRAUD_RISK_SCORE;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_FRAUD_CASES        AS SELECT * FROM GOLD.FACT_FRAUD_CASE;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_CUSTOMER AS
  SELECT CUSTOMER_ID, CUSTOMER_NAME, GENDER, EMAIL, PHONE, ADDRESS_LINE, CITY, STATE, REGION,
         CUSTOMER_RISK_CATEGORY FROM GOLD.DIM_CUSTOMER;
-- pipeline health for the app
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_CONTROL_CHECKS AS SELECT * FROM CONTROL.CONTROL_FILE_CHECK;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_LOAD_AUDIT     AS SELECT * FROM CONTROL.LOAD_AUDIT;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_REJECT_REGISTER AS SELECT * FROM CONTROL.REJECT_REGISTER;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_ALERT_HISTORY  AS SELECT * FROM CONTROL.ALERT_HISTORY;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_USER_REGION_MAP AS SELECT * FROM CONTROL.USER_REGION_MAP;
CREATE OR REPLACE SECURE VIEW CONSUMPTION.SV_DQ_SCORECARD AS
  SELECT SOURCE_NAME, ROUTE, COUNT(*) AS RECORDS FROM (
    SELECT 'customers' AS SOURCE_NAME, DQ_STATUS AS ROUTE FROM BRONZE.CUSTOMERS
    UNION ALL SELECT 'insurance_products', DQ_STATUS FROM BRONZE.INSURANCE_PRODUCTS
    UNION ALL SELECT 'policies',           DQ_STATUS FROM BRONZE.POLICIES
    UNION ALL SELECT 'premium_payments',   DQ_STATUS FROM BRONZE.PREMIUM_PAYMENTS
    UNION ALL SELECT 'claims',             DQ_STATUS FROM BRONZE.CLAIMS
    UNION ALL SELECT 'claim_settlements',  DQ_STATUS FROM BRONZE.CLAIM_SETTLEMENTS
    UNION ALL SELECT 'fraud_cases',        DQ_STATUS FROM BRONZE.FRAUD_CASES
```

```
  UNION ALL SELECT SOURCE_NAME, 'DUPLICATE' FROM CONTROL.DUPLICATE_LOG
  UNION ALL SELECT SOURCE_NAME, 'REJECT' FROM CONTROL.REJECT_LOG WHERE ATTEMPT_TYPE = 'INITIAL')
GROUP BY 1, 2;
```

13.5 Grants per role

Secure view	INSURANCE_ANALYST	CLAIMS_ANALYST	FRAUD_MANAGER	CG_APP_ROLE
SV_KPI_SUMMARY	✔	✔	✔	✔
SV_POLICY_PORTFOLIO	✔		✔	✔
SV_CLAIMS_PERFORMANCE, SV_SETTLEMENT_RECON		✔	✔	✔
SV_FRAUD_RISK, SV_FRAUD_CASES			✔	✔
SV_CUSTOMER (masked)	✔	✔	✔	✔
SV_CONTROL_CHECKS, SV_LOAD_AUDIT, SV_REJECT_REGISTER, SV_ALERT_HISTORY, SV_DQ_SCORECARD, SV_USER_REGION_MAP				✔

sql/74_grants.sql

```
USE ROLE SECURITYADMIN;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_KPI_SUMMARY TO ROLE INSURANCE_ANALYST_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_KPI_SUMMARY TO ROLE CLAIMS_ANALYST_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_KPI_SUMMARY TO ROLE FRAUD_MANAGER_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_CUSTOMER    TO ROLE INSURANCE_ANALYST_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_CUSTOMER    TO ROLE CLAIMS_ANALYST_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_CUSTOMER    TO ROLE FRAUD_MANAGER_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_POLICY_PORTFOLIO   TO ROLE INSURANCE_ANALYST_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_POLICY_PORTFOLIO   TO ROLE FRAUD_MANAGER_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_CLAIMS_PERFORMANCE TO ROLE CLAIMS_ANALYST_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_CLAIMS_PERFORMANCE TO ROLE FRAUD_MANAGER_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_SETTLEMENT_RECON   TO ROLE CLAIMS_ANALYST_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_SETTLEMENT_RECON   TO ROLE FRAUD_MANAGER_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_FRAUD_RISK  TO ROLE FRAUD_MANAGER_ROLE;
GRANT SELECT ON VIEW INSURANCE_DB.CONSUMPTION.SV_FRAUD_CASES TO ROLE FRAUD_MANAGER_ROLE;
GRANT SELECT ON ALL VIEWS IN SCHEMA INSURANCE_DB.CONSUMPTION TO ROLE CG_APP_ROLE;
GRANT CREATE STREAMLIT, CREATE STAGE ON SCHEMA INSURANCE_DB.CONSUMPTION TO ROLE CG_APP_ROLE;
```

13.6 Test every role with USE ROLE

tests/security_tests.sql

```
USE SECONDARY ROLES NONE;     -- IMPORTANT: otherwise your other roles leak into the test

USE ROLE INSURANCE_ANALYST_SOUTH_ROLE;
SELECT REGION, COUNT(*) FROM CONSUMPTION.SV_CUSTOMER GROUP BY 1;   -- only SOUTH
SELECT EMAIL, PHONE, ADDRESS_LINE FROM CONSUMPTION.SV_CUSTOMER LIMIT 3;  -- masked
SELECT * FROM GOLD.FACT_CLAIM LIMIT 1;                              -- error: no privilege
SELECT * FROM CONSUMPTION.SV_FRAUD_RISK LIMIT 1;                    -- error: not granted

USE ROLE INSURANCE_ANALYST_NORTH_ROLE;
SELECT REGION, COUNT(*) FROM CONSUMPTION.SV_CUSTOMER GROUP BY 1;   -- only NORTH

USE ROLE CLAIMS_ANALYST_ROLE;
SELECT PHONE FROM CONSUMPTION.SV_CUSTOMER LIMIT 3;                  -- ******1234

USE ROLE FRAUD_MANAGER_ROLE;
SELECT REGION, COUNT(*) FROM CONSUMPTION.SV_FRAUD_RISK GROUP BY 1; -- all regions, unmasked PII

USE ROLE CG_ENGINEER_ROLE;
```

Secondary roles

Newer Snowflake accounts activate **all** your roles as secondary roles by default. IS_ROLE_IN_SESSION then returns TRUE for roles you are not “using”, and every test looks unmasked. Always run `USE SECONDARY ROLES NONE;` before security tests.

✔ Done when:

- Screenshot per role: rows, masking and the expected “insufficient privileges” errors.
- SOUTH and NORTH roles see different region counts from the same view.

Phase 14 – ★ Threshold Alerts

In simple words: business limits live in a table. After every Gold refresh, a procedure measures each metric, compares it with its limit, and if the limit is crossed it writes ALERT_HISTORY and sends an email. Each metric alerts at most once per day for the same value, so nobody gets spammed. A separate Snowflake **Alert** warns when no file arrived by 09:00.

14.1 Email integration (ACCOUNTADMIN, once)

sql/80_email_integration.sql

```
USE ROLE ACCOUNTADMIN;
CREATE NOTIFICATION INTEGRATION IF NOT EXISTS CG_EMAIL_INT
  TYPE = EMAIL
  ENABLED = TRUE
  ALLOWED_RECIPIENTS = ('<you>@ltimindtree.com', '<teammate>@ltimindtree.com');
GRANT USAGE ON INTEGRATION CG_EMAIL_INT TO ROLE CG_ENGINEER_ROLE;

-- quick test
CALL SYSTEM$SEND_EMAIL('CG_EMAIL_INT', '<you>@ltimindtree.com',
                       'ClaimGuard360 test', 'If you can read this, alerts will work.');
```

Email not arriving?

Recipients must be **verified** email addresses of users in this Snowflake account (Snowsight → your name → Settings → Profile → verify email). Also check the spam folder.

14.2 Seed the thresholds

sql/81_thresholds_seed.sql

```
USE ROLE CG_ENGINEER_ROLE;
INSERT INTO CONTROL.ALERT_THRESHOLDS
  (METRIC_NAME, DESCRIPTION, METRIC_SQL, OPERATOR, THRESHOLD_VALUE, SEVERITY, RECIPIENTS)
SELECT * FROM VALUES
 ('HIGH_RISK_CLAIMS', 'Number of HIGH risk claims',
  'SELECT COUNT(*) FROM GOLD.FRAUD_RISK_SCORE WHERE RISK_CATEGORY = ''HIGH''',
  '>', 20, 'HIGH', '<you>@ltimindtree.com'),
 ('LOSS_RATIO_PCT', 'Loss ratio proxy (%) across all regions',
  'SELECT LOSS_RATIO_PCT FROM GOLD.V_KPI_SUMMARY WHERE REGION = ''ALL REGIONS''',
  '>', 70, 'HIGH', '<you>@ltimindtree.com'),
 ('MAX_SINGLE_CLAIM_INR', 'Largest single claim amount',
  'SELECT MAX(CLAIM_AMOUNT) FROM GOLD.FACT_CLAIM',
  '>', 10000000, 'MEDIUM', '<you>@ltimindtree.com'),
 ('AGED_OPEN_FRAUD_CASES', 'Fraud investigations open for more than 30 days',
  'SELECT COUNT(*) FROM GOLD.FACT_FRAUD_CASE WHERE IS_OPEN AND INVESTIGATION_DAYS > 30',
  '>', 0, 'MEDIUM', '<you>@ltimindtree.com'),
 ('CONTROL_FILE_MISMATCH_24H', 'Control file mismatches in the last 24 hours',
  'SELECT COUNT(*) FROM CONTROL.CONTROL_FILE_CHECK WHERE STATUS = ''MISMATCH''
     AND CHECK_TS >= DATEADD(hour, -24, CURRENT_TIMESTAMP())',
  '>', 0, 'HIGH', '<you>@ltimindtree.com'),
 ('OPEN_REJECTS', 'Rejected records waiting for a fix',
  'SELECT COUNT(*) FROM CONTROL.REJECT_REGISTER WHERE STATUS IN (''OPEN'', ''RESUBMITTED_FAILED'')',
  '>', 5, 'LOW', '<you>@ltimindtree.com'),
 ('OVER_SETTLED_CLAIMS', 'Claims paid more than approved',
  'SELECT COUNT(*) FROM GOLD.MART_SETTLEMENT_RECONCILIATION WHERE RECON_STATUS = ''OVER_SETTLED''',
  '>', 0, 'HIGH', '<you>@ltimindtree.com');
```

Changing a limit is a one-line UPDATE – no code change, no redeploy:

```
UPDATE CONTROL.ALERT_THRESHOLDS SET THRESHOLD_VALUE = 15 WHERE METRIC_NAME = 'HIGH_RISK_CLAIMS';
```

14.3 The checking procedure

sql/82_sp_check_thresholds.sql

```
CREATE OR REPLACE PROCEDURE CONTROL.SP_CHECK_THRESHOLDS()
RETURNS STRING
LANGUAGE PYTHON
RUNTIME_VERSION = '3.11'
PACKAGES = ('snowflake-snowpark-python')
HANDLER = 'run'
AS
$$
```

```
import operator

OPS = {'>': operator.gt, '>=': operator.ge, '<': operator.lt, '<=': operator.le, '=': operator.eq}


def q(text):                        # escape for a SQL string literal
    return str(text).replace("'", "''")


def run(session):
    rules = session.sql(
        "SELECT METRIC_NAME, DESCRIPTION, METRIC_SQL, OPERATOR, THRESHOLD_VALUE, SEVERITY, RECIPIENTS "
        "FROM CONTROL.ALERT_THRESHOLDS WHERE IS_ACTIVE").collect()
    fired, errors = 0, 0
    for r in rules:
        name, op, limit = r['METRIC_NAME'], r['OPERATOR'], float(r['THRESHOLD_VALUE'])
        try:
            value = session.sql(r['METRIC_SQL']).collect()[0][0]
        except Exception as e:
            errors += 1
            continue
        if value is None or op not in OPS or not OPS[op](float(value), limit):
            continue
        value = float(value)
        already = session.sql(
            "SELECT COUNT(*) FROM CONTROL.ALERT_HISTORY WHERE METRIC_NAME = ? "
            "AND ALERT_TS::DATE = CURRENT_DATE() AND METRIC_VALUE = ?", params=[name, value]).collect()[0][0]
        if already:
            continue                                    # same breach already reported today
        subject = f"[ClaimGuard360][{r['SEVERITY']}] {name} breached"
        body = (f"{r['DESCRIPTION']}\nCurrent value: {value:,.2f}\nLimit: {op} {limit:,.2f}\n"
                f"Time: see ALERT_HISTORY. Open the ClaimGuard360 dashboard for details.")
        sent = True
        try:
            session.sql(f"CALL SYSTEM$SEND_EMAIL('CG_EMAIL_INT', '{q(r['RECIPIENTS'])}', "
                        f"'{q(subject)}', '{q(body)}')").collect()
        except Exception:
            sent = False
        session.sql(
            "INSERT INTO CONTROL.ALERT_HISTORY (METRIC_NAME, METRIC_VALUE, OPERATOR, THRESHOLD_VALUE, "
            "SEVERITY, MESSAGE, EMAIL_SENT) VALUES (?, ?, ?, ?, ?, ?, ?)",
            params=[name, value, op, limit, r['SEVERITY'], subject, sent]).collect()
        fired += 1
    return f"{fired} alert(s) fired, {errors} metric error(s)"
$$;

CALL CONTROL.SP_CHECK_THRESHOLDS();
SELECT * FROM CONTROL.ALERT_HISTORY ORDER BY ALERT_TS DESC;
```

This procedure is already the last step of the task graph (T_CHECK_THRESHOLDS, Phase 10.2), so it runs after every refresh automatically.

14.4 Late-file alert with a Snowflake ALERT object

sql/83_alert_late_files.sql

```
CREATE OR REPLACE ALERT CONTROL.ALR_LATE_FILES
  WAREHOUSE = CG_WH
  SCHEDULE  = 'USING CRON 0 9 * * MON-FRI Asia/Kolkata'
  IF (EXISTS (
        SELECT 1
        WHERE (SELECT COUNT(*) FROM CONTROL.CONTROL_FILE_CHECK
               WHERE CHECK_TS::DATE = CURRENT_DATE()) = 0))
  THEN
    CALL SYSTEM$SEND_EMAIL('CG_EMAIL_INT', '<you>@ltimindtree.com',
         '[ClaimGuard360][HIGH] No files received today',
         'No control file has been checked today by 09:00 IST. Please contact the source system team.');

ALTER ALERT CONTROL.ALR_LATE_FILES RESUME;
EXECUTE ALERT CONTROL.ALR_LATE_FILES;          -- run it now for the demo
SELECT * FROM TABLE(INFORMATION_SCHEMA.ALERT_HISTORY()) ORDER BY SCHEDULED_TIME DESC;
```

✔ Done when:

- ■ Test email received.
- ■ Loading the INR 25,000,000 claim fires MAX_SINGLE_CLAIM_INR once (row in ALERT_HISTORY, EMAIL_SENT = TRUE).

- Running the graph again on the same day does **not** send a second email for the same value.
- Late-file alert email received after EXECUTE ALERT on a day with no files.

Phase 15 – Streamlit in Snowflake Dashboard

In simple words: a web app that lives inside Snowflake. It reads only the secure views, shows five tabs (Portfolio, Claims, Fraud, Pipeline Health, Reconciliation) and filters regions for whoever is looking at it.

> **Important: how security works inside Streamlit**
>
> A Streamlit in Snowflake app runs with the **owner's role** (CG_APP_ROLE), not the viewer's role. So: (1) masking still applies – CG_APP_ROLE is not exempt, PII stays masked; (2) for regions, the app reads the viewer's user name and looks it up in USER_REGION_MAP (PRINCIPAL_TYPE = 'USER'), then filters every query. Direct SQL access is still protected by the row access policy from Phase 13.

15.1 Create the app

1. Snowsight → switch role to **CG_APP_ROLE** (top-left role picker).
2. **Projects → Streamlit → + Streamlit App**. Name **CLAIMGUARD360_APP**, location **INSURANCE_DB.CONSUMPTION**, warehouse **CG_WH** → Create.
3. Delete the sample code, paste the code below, click **Run**.
4. **Share** (top right) → add INSURANCE_ANALYST_ROLE, CLAIMS_ANALYST_ROLE, FRAUD_MANAGER_ROLE.
5. Add each viewer to USER_REGION_MAP with PRINCIPAL_TYPE = 'USER' (ALL for managers).

15.2 streamlit_app.py

streamlit/streamlit_app.py

```
"""ClaimGuard 360 - Insurance, Claims & Fraud dashboard (Streamlit in Snowflake)."""
import re
import pandas as pd
import streamlit as st
from snowflake.snowpark.context import get_active_session

st.set_page_config(page_title="ClaimGuard 360", layout="wide")
session = get_active_session()
V = "INSURANCE_DB.CONSUMPTION."


@st.cache_data(ttl=300)
def q(sql: str) -> pd.DataFrame:
    return session.sql(sql).to_pandas()


def viewer_name() -> str:
    user = getattr(st, "user", None) or getattr(st, "experimental_user", None)
    try:
        name = user.get("user_name") or user.get("login_name") or ""
    except Exception:
        name = ""
    return re.sub(r"[^A-Z0-9_.@-]", "", str(name).upper())       # safe for SQL


# ---------------- who is looking, which regions may they see ----------------
me = viewer_name()
mapped = q(f"""SELECT REGION FROM {V}SV_USER_REGION_MAP
               WHERE PRINCIPAL_TYPE = 'USER' AND IS_ACTIVE AND PRINCIPAL_NAME = '{me}'""")
all_regions = q(f"SELECT DISTINCT REGION FROM {V}SV_POLICY_PORTFOLIO ORDER BY 1")["REGION"].tolist()
if mapped.empty:
    st.error(f"User {me or '(unknown)'} is not mapped to any region. Ask the data team for access.")
    st.stop()
allowed = all_regions if "ALL" in mapped["REGION"].tolist() else \
    [r for r in all_regions if r in mapped["REGION"].tolist()]

st.sidebar.title("ClaimGuard 360")
st.sidebar.caption(f"Signed in as {me}")
regions = st.sidebar.multiselect("Region", allowed, default=allowed)
if not regions:
    st.warning("Select at least one region.")
    st.stop()
IN = "(" + ",".join("'" + r.replace("'", "''") + "'" for r in regions) + ")"
```

```
if st.sidebar.button("Refresh data"):
    st.cache_data.clear()

st.title("ClaimGuard 360 - Insurance, Claims & Fraud")
t1, t2, t3, t4, t5 = st.tabs(["Portfolio", "Claims", "Fraud", "Pipeline Health", "Reconciliation"])

# ---------------- PORTFOLIO ----------------
with t1:
    pf = q(f"SELECT * FROM {V}SV_POLICY_PORTFOLIO WHERE REGION IN {IN}")
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Active policies", f"{int(pf.ACTIVE_POLICY_COUNT.sum()):,}")
    c2.metric("Active sum insured", f"INR {pf.ACTIVE_SUM_INSURED.sum():,.0f}")
    c3.metric("Written premium", f"INR {pf.WRITTEN_PREMIUM.sum():,.0f}")
    c4.metric("Premium collected", f"INR {pf.PREMIUM_COLLECTED.sum():,.0f}")
    st.subheader("Premium collected by insurance type")
    st.bar_chart(pf.groupby("INSURANCE_TYPE")["PREMIUM_COLLECTED"].sum())
    st.subheader("Active policies by region")
    st.bar_chart(pf.groupby("REGION")["ACTIVE_POLICY_COUNT"].sum())
    st.dataframe(pf, use_container_width=True)

# ---------------- CLAIMS ----------------
with t2:
    cp = q(f"SELECT * FROM {V}SV_CLAIMS_PERFORMANCE WHERE REGION IN {IN}")
    rc = q(f"SELECT * FROM {V}SV_SETTLEMENT_RECON WHERE REGION IN {IN}")
    claims = cp.CLAIM_COUNT.sum()
    premium = pf.PREMIUM_COLLECTED.sum()
    net = rc.TOTAL_NET_SETTLED.sum()

    c1, c2, c3, c4, c5 = st.columns(5)
    c1.metric("Claims", f"{int(claims):,}")
    c2.metric("Average claim", f"INR {cp.TOTAL_CLAIM_AMOUNT.sum() / max(claims, 1):,.0f}")
    c3.metric("Approval ratio", f"{100 * cp.APPROVED_CLAIM_COUNT.sum() / max(claims, 1):.1f}%")
    c4.metric("Avg settlement TAT", f"{rc.AVG_TURNAROUND_DAYS.mean():.1f} days")
    c5.metric("Loss ratio proxy", f"{100 * net / premium:.1f}%" if premium else "n/a")
    st.subheader("Claims per month")
    st.line_chart(cp.groupby("CLAIM_MONTH")["CLAIM_COUNT"].sum())
    st.subheader("Settlement reconciliation status")
    st.bar_chart(rc.groupby("RECON_STATUS")["CLAIM_ID"].count())
    st.subheader("Exceptions: not settled / over settled")
    st.dataframe(rc[rc.RECON_STATUS.isin(["NOT_SETTLED", "OVER_SETTLED"])], use_container_width=True)

# ---------------- FRAUD ----------------
with t3:
    fr = q(f"SELECT * FROM {V}SV_FRAUD_RISK WHERE REGION IN {IN}")
    fc = q(f"SELECT * FROM {V}SV_FRAUD_CASES WHERE REGION IN {IN}")
    c1, c2, c3 = st.columns(3)
    c1.metric("High-risk claims", int((fr.RISK_CATEGORY == "HIGH").sum()))
    c2.metric("Open investigations", int(fc.IS_OPEN.sum()))
    c3.metric("Confirmed fraud rate", f"{100 * fc.IS_CONFIRMED.sum() / max(len(fc), 1):.1f}%")
    st.subheader("Risk mix")
    st.bar_chart(fr.groupby("RISK_CATEGORY")["CLAIM_ID"].count())
    st.subheader("Why is a claim high risk?")
    high = fr[fr.RISK_CATEGORY == "HIGH"].sort_values("TOTAL_SCORE", ascending=False)
    if high.empty:
        st.info("No HIGH risk claims for the selected regions.")
    else:
        pick = st.selectbox("Choose a HIGH risk claim", high.CLAIM_ID.tolist())
        row = high[high.CLAIM_ID == pick].iloc[0]
        st.bar_chart(pd.Series({"Amount": row.AMOUNT_SCORE, "Early claim": row.EARLY_CLAIM_SCORE,
                                "History": row.HISTORY_SCORE, "Investigation": row.INVESTIGATION_SCORE}))
        st.write(f"**Total {int(row.TOTAL_SCORE)} / 100** - {row.SCORE_REASON}")
        st.caption(f"Model {row.MODEL_VERSION}, scored {row.SCORING_TS}")
    st.dataframe(high, use_container_width=True)

# ---------------- PIPELINE HEALTH ----------------
with t4:
    st.subheader("Control file checks")
    st.dataframe(q(f"SELECT * FROM {V}SV_CONTROL_CHECKS ORDER BY CHECK_TS DESC LIMIT 50"),
                 use_container_width=True)
    st.subheader("Data-quality scorecard (records per route)")
    dq = q(f"SELECT * FROM {V}SV_DQ_SCORECARD")
    st.dataframe(dq.pivot_table(index="SOURCE_NAME", columns="ROUTE", values="RECORDS",
                                aggfunc="sum", fill_value=0), use_container_width=True)
    st.subheader("Reject register")
    rr = q(f"SELECT * FROM {V}SV_REJECT_REGISTER ORDER BY UPDATED_TS DESC")
    st.bar_chart(rr.groupby("STATUS")["REJECT_ID"].count())
    st.dataframe(rr, use_container_width=True)
    st.subheader("Alerts")
    st.dataframe(q(f"SELECT * FROM {V}SV_ALERT_HISTORY ORDER BY ALERT_TS DESC LIMIT 50"),
                 use_container_width=True)
    st.subheader("Latest loads")
    st.dataframe(q(f"SELECT * FROM {V}SV_LOAD_AUDIT ORDER BY LOGGED_TS DESC LIMIT 50"),
                 use_container_width=True)
```

```
# ---------------- RECONCILIATION (Streamlit vs Gold) ----------------
with t5:
    st.write("Totals shown in this app compared with the Gold KPI view, for the selected regions.")
    kpi = q(f"SELECT * FROM {V}SV_KPI_SUMMARY WHERE REGION IN {IN}")
    checks = pd.DataFrame([
        ["Active policies", pf.ACTIVE_POLICY_COUNT.sum(), kpi.ACTIVE_POLICIES.sum()],
        ["Premium collected", round(pf.PREMIUM_COLLECTED.sum(), 2), round(kpi.PREMIUM_COLLECTED.sum(), 2)],
        ["Claims", cp.CLAIM_COUNT.sum(), kpi.CLAIM_COUNT.sum()],
        ["High-risk claims", int((fr.RISK_CATEGORY == "HIGH").sum()), kpi.HIGH_RISK_CLAIMS.sum()],
    ], columns=["Measure", "Dashboard", "Gold"])
    checks["Result"] = ["MATCH" if abs(float(a) - float(b)) < 0.01 else "MISMATCH"
                        for a, b in zip(checks.Dashboard, checks.Gold)]
    st.dataframe(checks, use_container_width=True)
    if (checks.Result == "MATCH").all():

        st.success("Dashboard reconciles with Gold.")
    else:
        st.error("Dashboard does not reconcile with Gold - investigate before sharing.")
```

If a chart is empty

Usually a status value differs from the ones in Phase 9 (e.g. “Settled” vs “SETTLED”). Fix the IN (…) lists in the Gold views – not in the app – so every consumer gets the same answer.

✔ **Done when:**

- ■ App opens for each shared role; masked PII visible as stars.
- ■ Your user mapped to SOUTH sees only SOUTH; mapped to ALL sees every region.
- ■ Reconciliation tab shows MATCH for all four measures (screenshot).

Phase 16 – Deployment, Git, Runbook and Recovery

16.1 Repository layout

```
claimguard360/
├── README.md                    <- what it is, how to deploy, team, decisions
├── sql/
│   ├── 01_foundation.sql  02_integration.sql  03_stages.sql  04_idmc_user.sql
│   ├── 05_control_tables.sql  06_sp_control_check.sql
│   ├── 10_bronze_tables.sql  11_sp_load_bronze.sql
│   ├── 20_silver_views.sql  21_silver_tables.sql  22_silver_dq.sql
│   ├── 30_dim_date.sql  31_gold_dimensions.sql  32_gold_facts.sql  33_gold_marts.sql
│   ├── 34_kpi_view.sql  35_sp_refresh_gold.sql
│   ├── 40_stream_merge.sql  41_task_graph.sql  50_reject_register.sql  60_sp_fraud_score.sql
│   ├── 70_roles.sql  71_masking.sql  72_row_access.sql  73_secure_views.sql  74_grants.sql
│   └── 80_email_integration.sql  81_thresholds_seed.sql  82_sp_check_thresholds.sql  83_alert_late_files.sql
├── idmc/        <- exported mappings, tasks, taskflows, listeners (.zip)
├── scripts/     <- cg_move.sh, deploy.sh
├── streamlit/   <- streamlit_app.py
├── tests/       <- *.sql test scripts + expected results
└── docs/        <- architecture, data dictionary, runbook, test report, screenshots
```

16.2 One-command deployment with SnowSQL

SnowSQL connection

```
# ~/.snowsql/config
[connections.cg]
accountname = <orgname-accountname>
username    = <your_user>
authenticator = externalbrowser
rolename    = CG_ENGINEER_ROLE
warehousename = CG_WH
dbname      = INSURANCE_DB
```

scripts/deploy.sh

```
#!/bin/bash
# scripts/deploy.sh  - runs every engineer-owned script in order, stops on first error
set -e
for f in 05 06 10 11 20 21 22 30 31 32 33 34 35 40 50 60 71 72 73 81 82 83 41; do
  file=$(ls sql/${f}_*.sql)
  echo ">>> $file"
  snowsql -c cg -f "$file" -o exit_on_error=true -o friendly=false -o quiet=true
done
echo "Deployment complete"
# 01-04, 70, 74, 80 need ACCOUNTADMIN / SECURITYADMIN - run them once by hand.
# 41 (task graph) runs last because it enables tasks that call every procedure.
```

Run as CG_APP_ROLE

```
-- deploy the Streamlit app from Git instead of pasting in Snowsight
CREATE STAGE IF NOT EXISTS INSURANCE_DB.CONSUMPTION.APP_STAGE;
PUT file://streamlit/streamlit_app.py @INSURANCE_DB.CONSUMPTION.APP_STAGE/claimguard/
    AUTO_COMPRESS = FALSE OVERWRITE = TRUE;
CREATE OR REPLACE STREAMLIT INSURANCE_DB.CONSUMPTION.CLAIMGUARD360_APP
  FROM '@INSURANCE_DB.CONSUMPTION.APP_STAGE/claimguard'
  MAIN_FILE = 'streamlit_app.py'
  QUERY_WAREHOUSE = CG_WH;
```

16.3 IDMC export

Explore → select the project folder → **Export** → include dependencies → download the .zip into **idmc/**.
To rebuild in another org: Explore → **Import** → map the connections CONN_S3_CG and CONN_SF_CG.

16.4 Recovery (Time Travel and Zero-Copy Clone)

tests/recovery.sql

```
-- before any risky rerun: free, instant backup
CREATE OR REPLACE SCHEMA INSURANCE_DB.GOLD_BKP CLONE INSURANCE_DB.GOLD;

-- someone dropped a table
DROP TABLE GOLD.FACT_CLAIM;          -- (demo)
UNDROP TABLE GOLD.FACT_CLAIM;

-- a bad load changed data: put the table back as it was 30 minutes ago
INSERT OVERWRITE INTO GOLD.FACT_CLAIM
  SELECT * FROM GOLD.FACT_CLAIM AT (OFFSET => -60 * 30);

-- or go back to just before a specific statement
SELECT QUERY_ID, QUERY_TEXT, START_TIME FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
WHERE QUERY_TEXT ILIKE '%FACT_CLAIM%' ORDER BY START_TIME DESC LIMIT 10;
INSERT OVERWRITE INTO GOLD.FACT_CLAIM
  SELECT * FROM GOLD.FACT_CLAIM BEFORE (STATEMENT => '<query_id>');
```

Restore into the same table

INSERT OVERWRITE … AT(…) restores the data while keeping grants, masking and row access policies. A clone swap would need the policies re-attached.

16.5 Runbook (operations cheat sheet)

Situation	What to do
Start a daily batch	Upload data files → .ctl files → _BATCH_READY_yyyymmdd.done (all to S3). Nothing else.
Run the Snowflake side now	EXECUTE TASK CONTROL.T_ROOT_LOAD_BRONZE;
A source was quarantined	Read CONTROL_FILE_CHECK.MESSAGE → get a corrected file/ctl from the source team → upload with a **new** date in the name → new .done file.
Fix rejected rows	Follow Phase 11.5.
Change an alert limit	UPDATE CONTROL.ALERT_THRESHOLDS … (no redeploy).
Give an analyst a region	INSERT INTO CONTROL.USER_REGION_MAP (USER and/or ROLE row).
Task failed	TASK_HISTORY → ERROR_MESSAGE → fix → EXECUTE TASK root (all steps are idempotent).
Pause everything	ALTER TASK CONTROL.T_ROOT_LOAD_BRONZE SUSPEND; stop both File Listeners in IDMC.
Bad data reached Gold	Clone GOLD → Time Travel restore (16.4) → rerun graph.

✔ Done when:

- ■ deploy.sh runs end-to-end on a fresh database without errors.
- ■ IDMC export zip committed; README explains the run order and the manual admin scripts.
- ■ Recovery demo: drop + UNDROP, and an AT(OFFSET) restore.

Phase 17 – Testing, Reconciliation and Demo

17.1 Test cases mapped to the document's testing requirements

#	Requirement	How to test	Pass when
1	CSV load and header validation	LOAD_AUDIT; file format ERROR_ON_COLUMN_COUNT_MISMATCH	0 errors; a file with an extra column fails
2	Datatype and mandatory fields	Silver: COUNT_IF(date IS NULL AND bronze value not null)	0
3	Duplicate / null / anomaly / outlier routing	SV_DQ_SCORECARD + REJECT_LOG reasons	3 dup, 3 reject, 10 warning
4	Relationship and sequence validation	DQ_EXCEPTIONS ERROR rows; orphan queries on Gold	0 orphans in Gold
5	Source → Bronze → Silver → Gold reconciliation	17.2 query	every layer explained row-for-row
6	KPI calculation	Recalculate 3 KPIs by hand in Excel from Silver extracts	same values
7	Incremental load and replay	Phase 10.3	insert/update correct; replay unchanged
8	Roles and masking	Phase 13.6	expected rows / stars / errors
9	Snowpark score	Phase 12.3	0 differences between runs
10	Streamlit-to-Gold	Reconciliation tab	all MATCH
11	Time Travel / clone recovery	Phase 16.4	data restored
★	File Listener	Upload .done, watch Monitor	taskflow starts unattended
★	Control file check	Wrong count in .ctl	MISMATCH, quarantined, alert email
★	Reject reprocessing	Fix + resubmit	OPEN → RESOLVED; 3 fails → CLOSED
★	Threshold alerts	Load outlier claim	one email, one history row
★	Region access	NORTH vs SOUTH role / user	different rows, same view

17.2 Layer-to-layer reconciliation query

tests/reconciliation.sql

```
WITH src AS (SELECT SOURCE_NAME, SUM(EXPECTED_ROW_COUNT) AS SOURCE_ROWS
             FROM CONTROL.CONTROL_FILE_CHECK WHERE STATUS = 'MATCH' GROUP BY 1),
dup AS (SELECT SOURCE_NAME, COUNT(*) AS DUPLICATES FROM CONTROL.DUPLICATE_LOG GROUP BY 1),
rej AS (SELECT SOURCE_NAME, COUNT(*) AS REJECTS FROM CONTROL.REJECT_LOG
        WHERE ATTEMPT_TYPE = 'INITIAL' GROUP BY 1),
brz AS (SELECT 'claims' AS SOURCE_NAME, COUNT(*) AS BRONZE_ROWS FROM BRONZE.CLAIMS
        UNION ALL SELECT 'policies', COUNT(*) FROM BRONZE.POLICIES),          -- add the other 5
slv AS (SELECT 'claims' AS SOURCE_NAME, COUNT(*) AS SILVER_ROWS FROM SILVER.CLAIMS
        UNION ALL SELECT 'policies', COUNT(*) FROM SILVER.POLICIES),
gld AS (SELECT 'claims' AS SOURCE_NAME, COUNT(*) AS GOLD_ROWS FROM GOLD.FACT_CLAIM
        UNION ALL SELECT 'policies', COUNT(*) FROM GOLD.DIM_POLICY)
SELECT src.SOURCE_NAME, SOURCE_ROWS, COALESCE(DUPLICATES,0) DUPLICATES, COALESCE(REJECTS,0) REJECTS,
       BRONZE_ROWS, SOURCE_ROWS - COALESCE(DUPLICATES,0) - COALESCE(REJECTS,0) - BRONZE_ROWS AS SRC_VS_BRONZE_GAP,
       SILVER_ROWS, GOLD_ROWS
FROM src LEFT JOIN dup USING (SOURCE_NAME) LEFT JOIN rej USING (SOURCE_NAME)
         LEFT JOIN brz USING (SOURCE_NAME) LEFT JOIN slv USING (SOURCE_NAME) LEFT JOIN gld USING (SOURCE_NAME)
ORDER BY 1;     -- SRC_VS_BRONZE_GAP must be 0; Silver < Bronze only by corrected versions (incremental)
```

17.3 First full run – the exact order

1. Deploy all SQL (Phase 16.2) and the admin scripts. Task graph enabled.
2. Start both File Listeners. Publish both taskflows.
3. **Wave 1**: upload customers, insurance_products, policies (+ their .ctl) and a .done file.
4. Wait for IDMC to finish → `EXECUTE TASK CONTROL.T_ROOT_LOAD_BRONZE;` → wait until TASK_HISTORY shows T_CHECK_THRESHOLDS SUCCEEDED (Silver policies now exist for the claims lookup).
5. **Wave 2**: upload premium_payments, claims, claim_settlements, fraud_cases (+ .ctl) and a new .done file.
6. IDMC runs → execute the task graph again → check Phase 17.2 and the dashboard.
7. From now on every batch is fully automatic.

17.4 Ten-minute demo script

Min	Show	Say
0–1	Empty landing/ in S3 + Pipeline Health tab	"Nobody will press Run today."
1–2	Upload claims file + .ctl + .done → IDMC Monitor	File Listener started the taskflow by itself.
2–3	CONTROL_FILE_CHECK: MATCH. Then a wrong .ctl → MISMATCH + quarantine + email	We prove every file is complete.
3–4	DQ scorecard: 3 duplicates, 3 rejects, 10 warnings	Every documented DQ case routed exactly.
4–5	Fix a reject → resubmit → OPEN becomes RESOLVED	Bad rows get a second chance, fully traced.
5–6	Second claims file → MERGE counts; replay → HASH_AGG unchanged	Incremental and idempotent.
6–7	Fraud tab → pick a HIGH claim → score bars + reason	Explainable, reproducible scoring.
7–8	Alert email for the INR 25,000,000 claim	The platform warns the business itself.
8–9	SOUTH analyst vs NORTH analyst; masked email/phone	Right data for the right person.
9–10	DROP + UNDROP / Time Travel restore; Reconciliation tab all MATCH	Recoverable and trustworthy.

17.5 Troubleshooting

Symptom	Likely cause → fix
LIST @stage: Access Denied	Trust policy external ID / ARN wrong → redo Phase 2.3.
IDMC taskflow never starts	Listener not started, taskflow not published, or file not matching *.done in control/.
Gate always 0 (NO_FILE)	.ctl name not <source>_yyyymmdd.ctl, or the same name was already checked.
COPY loads 0 rows	File unchanged since last load (expected) – or the path/PATTERN is wrong; run LIST on the stage.
Silver dates NULL	A 5th date format exists – add it to EXP_STANDARDISE / the UDF.
Task "insufficient privileges"	Object not owned by CG_ENGINEER_ROLE, or EXECUTE TASK not granted.
Masking/RAP disappeared	Someone used CREATE OR REPLACE TABLE – re-run 71/72 and use INSERT OVERWRITE.
Everyone sees all regions	Secondary roles active → USE SECONDARY ROLES NONE; or role mapped to ALL.
SYSTEM$SEND_EMAIL fails	Recipient not verified or not in ALLOWED_RECIPIENTS.

Symptom	Likely cause → fix
Streamlit: “not mapped to any region”	Add a USER row in USER_REGION_MAP with your Snowflake user name (upper case).

✔ **Done when:**

- ■ All test cases pass and are recorded in docs/test_report.xlsx with screenshots.
- ■ Demo rehearsed twice end-to-end within 10 minutes.
- ■ Acceptance criteria from the document ticked one by one.

Acceptance Criteria Checklist

Criterion (project document)	Where it is proven
All seven files and 6,080 records are processed	17.2 reconciliation (source = Bronze + duplicates + rejects)
Every documented column is present in the correct order	Bronze DDL 7.1, clean templates 5.2
Controlled DQ records detected and routed exactly	SV_DQ_SCORECARD, REJECT_LOG, DUPLICATE_LOG
Relationships, grain, KPIs and reconciliations correct	8.3, 9 checklist, 9.6, 17.2
Incremental processing is idempotent	10.3 replay A and B
Roles and masking are demonstrated	13.6 test script
Snowpark score is reproducible	12.3
Streamlit reconciles with Gold	Reconciliation tab
Recovery and rerun procedures work	16.4, runbook 16.5
★ Beyond scope	File Listener, Control File Check, Reject Reprocessing, Threshold Alerts, Region-Based Access

Before you present

Replace every <placeholder>; confirm the 4 date formats, status values and region values with profiling; keep thresholds realistic for your data; and suspend the task graph after the demo.